

An Introduction to Optimization

Thomas Weise (汤卫思)

tweise@hfu.edu.cn · <https://thomasweise.github.io>

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Outline



1. Introduction
2. Problems that we can solve with equations
3. Problems that algorithms can solve fast and efficiently
4. Problems that algorithms cannot solve both efficiently *and* exactly
5. Randomly Guessing Solutions
6. Local Search: Using Information
7. Summary
8. Advertisement





Introduction



What is Optimization?



- There are **three** ways to approach this topic.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.
 2. Optimization is the art of solving **hard** problems.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.
 2. Optimization is the art of solving **hard** problems.
 3. Optimization allows us to find acceptable solutions for problems where our tools from high school maths *and* traditional algorithms from computer science are no longer enough, problems where exact solutions **cannot always be reached**.

Optimization is the search for superlatives

- Optimization means finding superlatives.



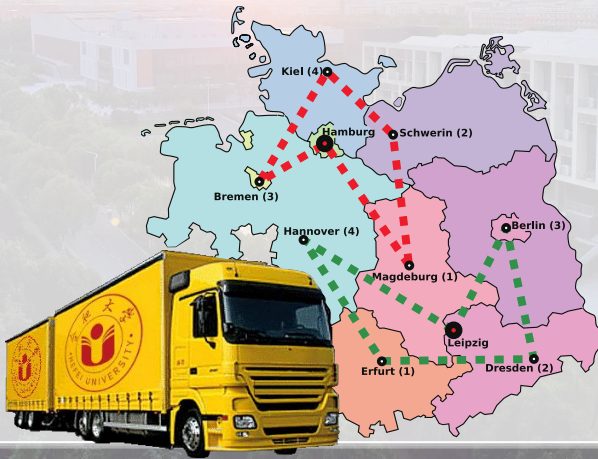
biggest ... with the least amount
of fuel...
...at the earliest possible time
...highest quality ...longest possible duration
most efficient ... most precise ... cheapest ... fastest...
fewest boxes ...with the highest score
...the longest possible duration most robust ...
...shortest path

The *superlative* form of an adjective is used to show that something has a quality to the greatest or least degree.

Optimization is the search for superlatives

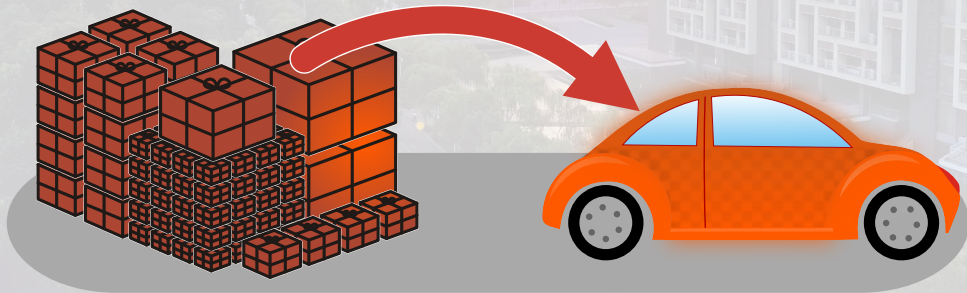
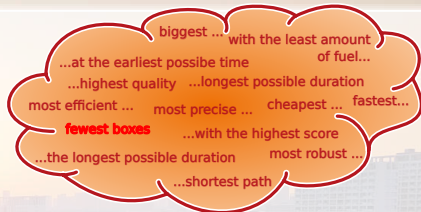
- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{73,93-95}.

biggest ... **with the least amount of fuel...**
...at the earliest possible time
...highest quality ...longest possible duration
most efficient ... most precise ... cheapest ... fastest...
fewest boxes ...with the highest score
...the longest possible duration most robust ...
...shortest path



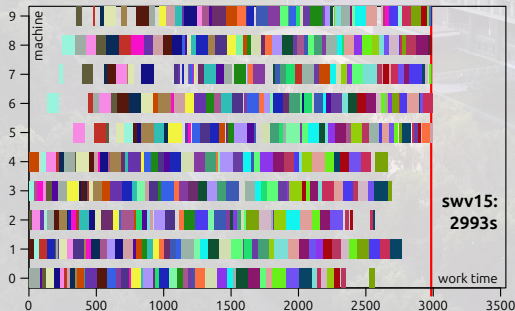
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{73,93-95}.
- Pack a set of things into the **fewest** boxes^{29,82,106,107}.



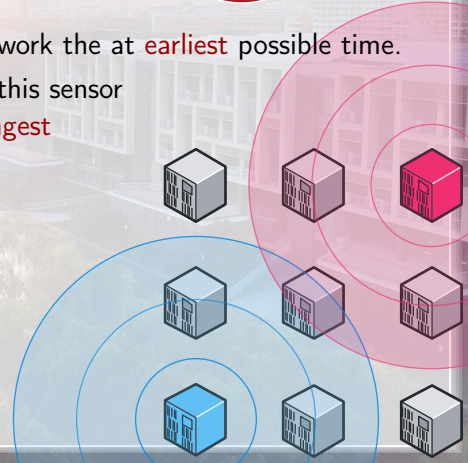
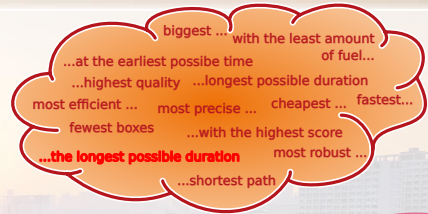
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{73,93-95}.
- Pack a set of things into the **fewest** boxes^{29,82,106,107}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.



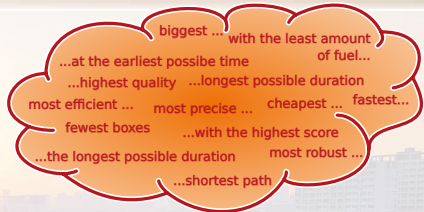
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{73,93–95}.
- Pack a set of things into the **fewest** boxes^{29,82,106,107}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.



Optimization is the search for superlatives

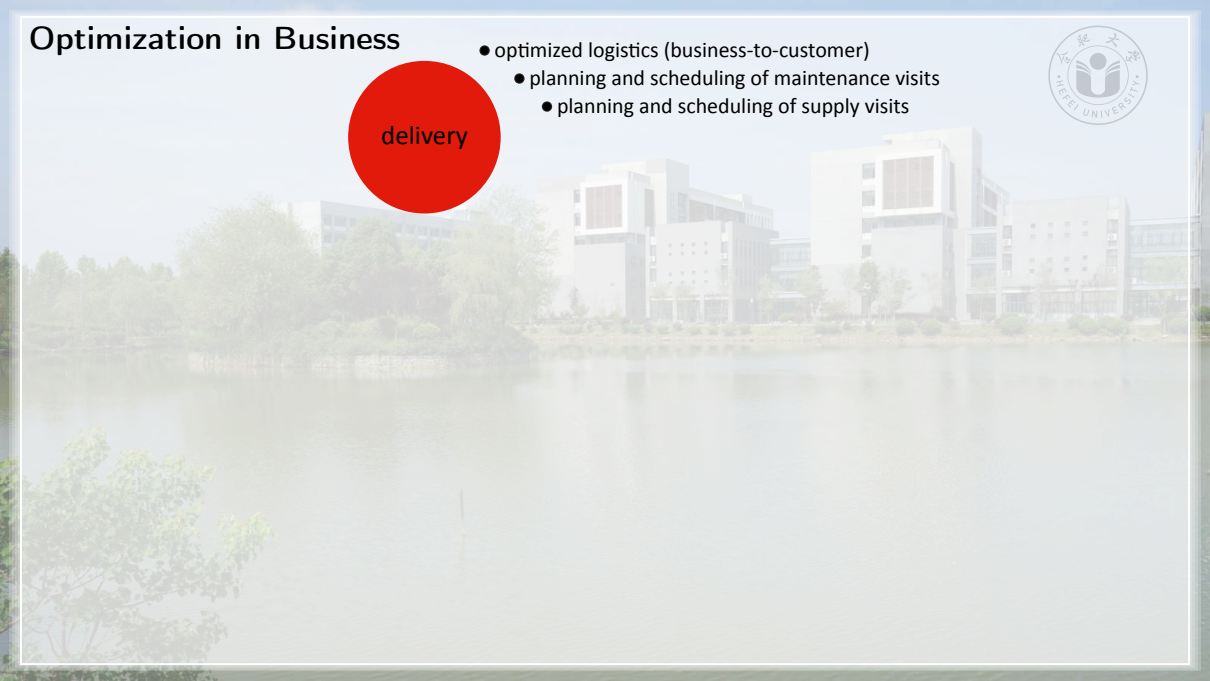
- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{73,93-95}.
- Pack a set of things into the **fewest** boxes^{29,82,106,107}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.
- And so on.



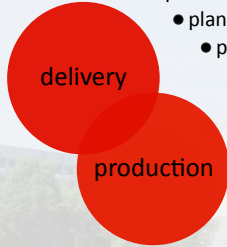
Optimization in Business

delivery

- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits

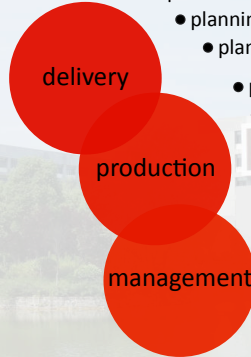


Optimization in Business



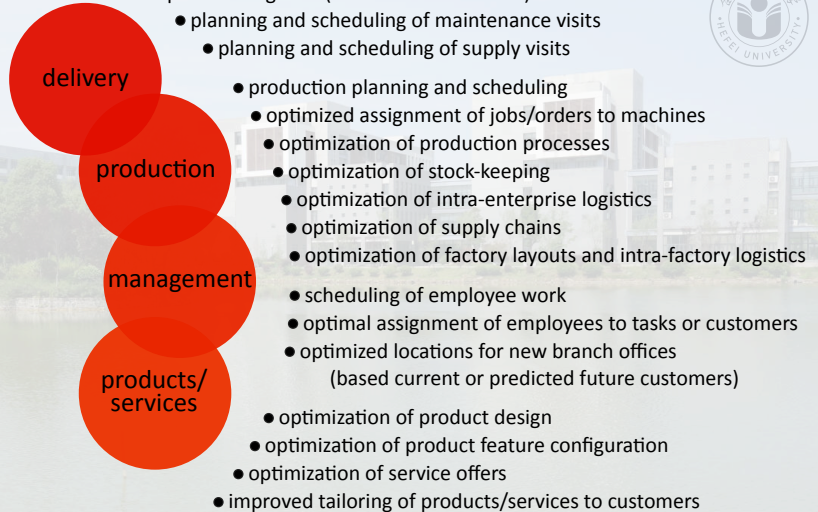
- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics

Optimization in Business



- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits
- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)

Optimization in Business



Optimization in Business



Optimization in Business



optimization
operations research
artificial intelligence (AI)
computational intelligence
machine learning
data mining

delivery

production

management

products/
services

sales

- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers
- optimization of pricing and offers
- mining of customer data for targeted offers

Optimization in Business



optimization
operations research
artificial intelligence (AI)
computational intelligence
machine learning
data mining

delivery

production

management

products/
services

sales

- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers
- optimization of pricing and offers
- mining of customer data for targeted offers

Optimization



- There are incredibly many problems from a very wide area where we can use optimization and Operations Research (OR).

Optimization



- There are incredibly many problems from a very wide area where we can use optimization and Operations Research (OR).
- How is this related to what you already learned?



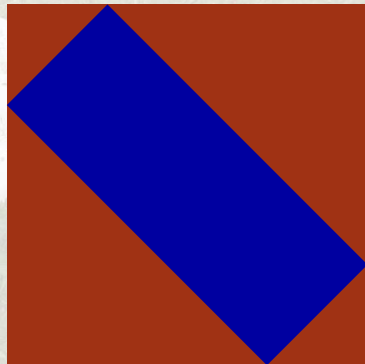
Problems that we can solve with equations



A Problem that we can solve with (high school Maths) equations



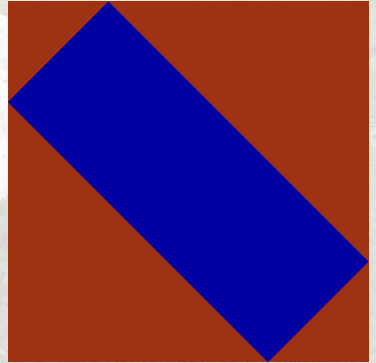
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

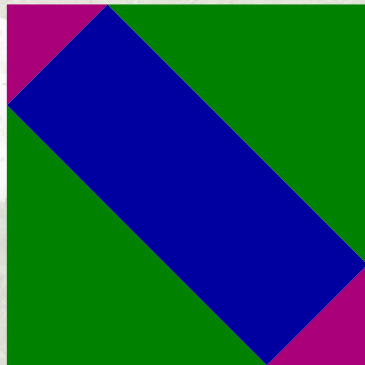


A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$



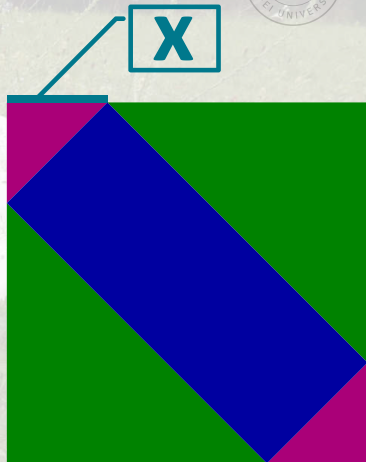
A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$



A Problem that we can solve with (high school Maths) equations

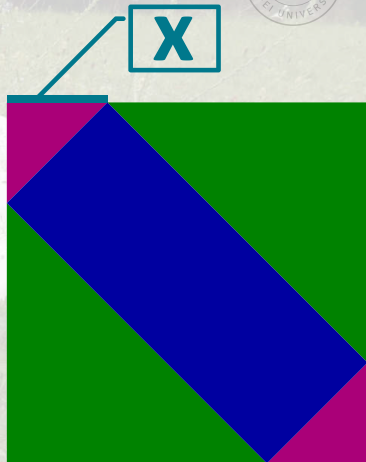


- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$



A Problem that we can solve with (high school Maths) equations



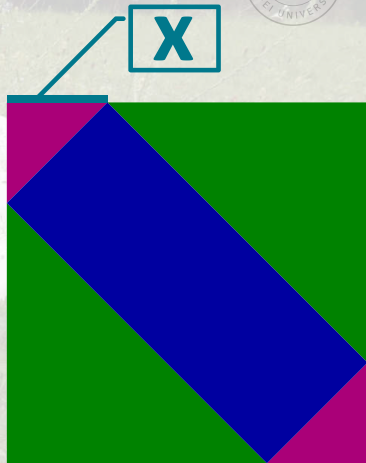
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

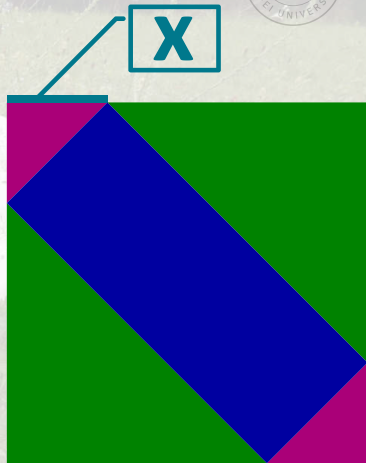
$$A_{\blacksquare} = A_{\square} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$



A Problem that we can solve with (high school Maths) equations



- A rectangle should be placed inside a square with side length 13cm as sketched.
- What area has the *largest* such rectangle?

$$A_{\blacksquare} = A_{\square} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

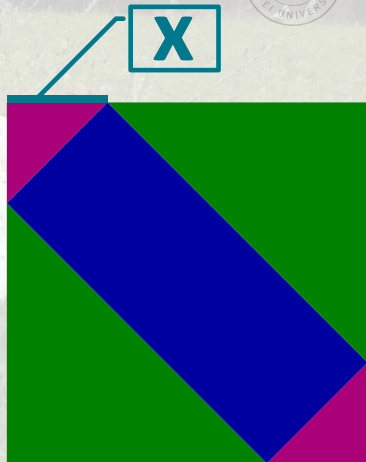
$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

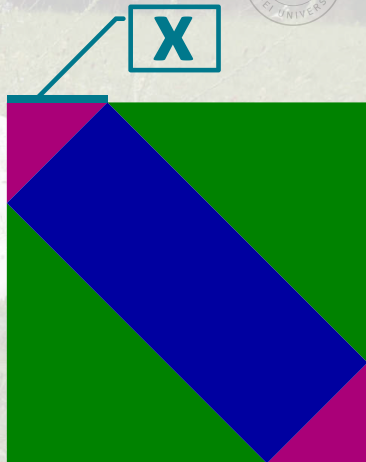
$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$

$$0 = -4\hat{x} + 26\text{cm}$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\blacksquare} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

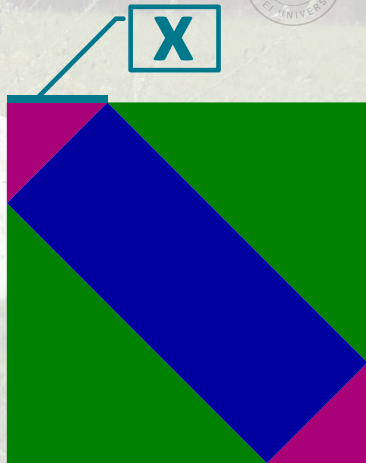
$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$

$$0 = -4\hat{x} + 26\text{cm}$$

$$4\hat{x} = 26\text{cm}$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare} = A_{\square} - 2A_{\blacktriangledown} - 2A_{\blacktriangle}$$

$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - (13\text{cm} - x)^2$$

$$A_{\blacksquare}(x) = 169\text{cm}^2 - x^2 - 169\text{cm}^2 + 26\text{cm} * x - x^2$$

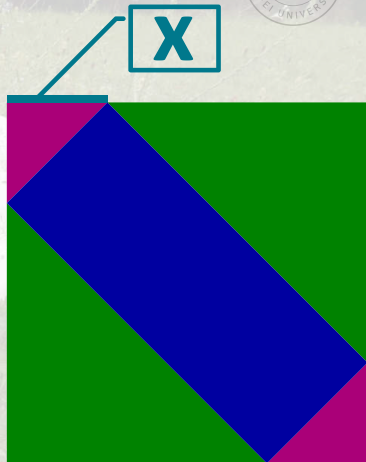
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$A'_{\blacksquare}(x) = -4x + 26\text{cm}$$

$$0 = -4\hat{x} + 26\text{cm}$$

$$4\hat{x} = 26\text{cm}$$

$$\hat{x} = 6.5\text{cm}$$



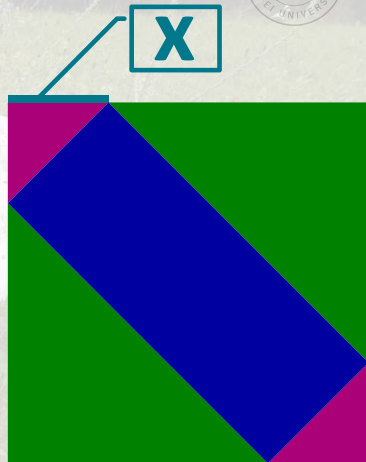
A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A(x) = -2x^2 + 26\text{cm} * x$$

$$\hat{x} = 6.5\text{cm}$$



A Problem that we can solve with (high school Maths) equations



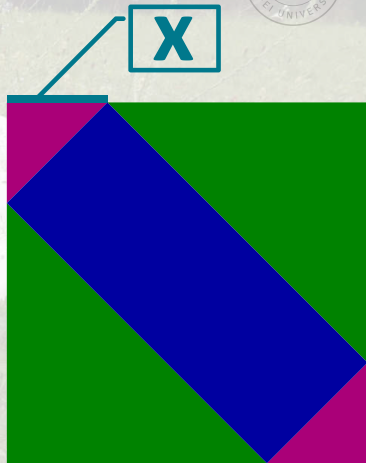
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$\hat{x} = 6.5\text{cm}$$

$$\widehat{A}_{\blacksquare} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$

$$\widehat{A}_{\blacksquare} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

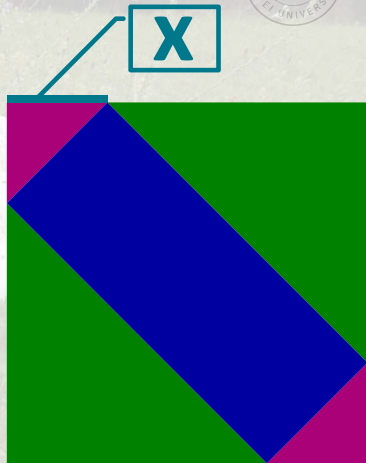
$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

$$\hat{x} = 6.5\text{cm}$$

$$\hat{A}_{\blacksquare} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$

$$\hat{A}_{\blacksquare} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$

$$\hat{A}_{\blacksquare} = 84.5\text{cm}^2$$



A Problem that we can solve with (high school Maths) equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

$$A_{\blacksquare}(x) = -2x^2 + 26\text{cm} * x$$

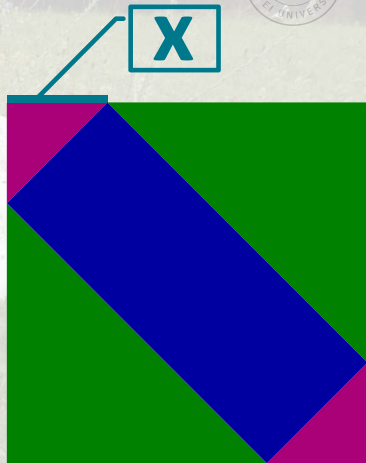
$$\hat{x} = 6.5\text{cm}$$

$$\widehat{A}_{\blacksquare} = A_{\blacksquare}(\hat{x}) = A_{\blacksquare}(6.5\text{cm})$$

$$\widehat{A}_{\blacksquare} = -2(6.5\text{cm})^2 + 26\text{cm} * 6.5\text{cm}$$

$$\widehat{A}_{\blacksquare} = 84.5\text{cm}^2$$

Solved.



Problems that we can solve with equations



- We can actually solve a lot of problems.

Problems that we can solve with equations



- We can actually solve a lot of problems.
- We just used an equation.

Problems that we can solve with equations



- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.

Problems that we can solve with equations



- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.
- Regardless how the previous problem would be parameterized (say, 16cm instead of 13cm), we could perform the exactly same steps.

Problems that we can solve with equations



- We can actually solve a lot of problems.
- We just used an equation.
- We did computations in multiple steps.
- Regardless how the previous problem would be parameterized (say, 16cm instead of 13cm), we could perform the exactly same steps.
- Can we always do that?



Problems that algorithms can solve fast and efficiently



Find the Shortest Path from **Start** to **Goal**



Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.



Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.



Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?

Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.

Find the Shortest Path from Start to Goal



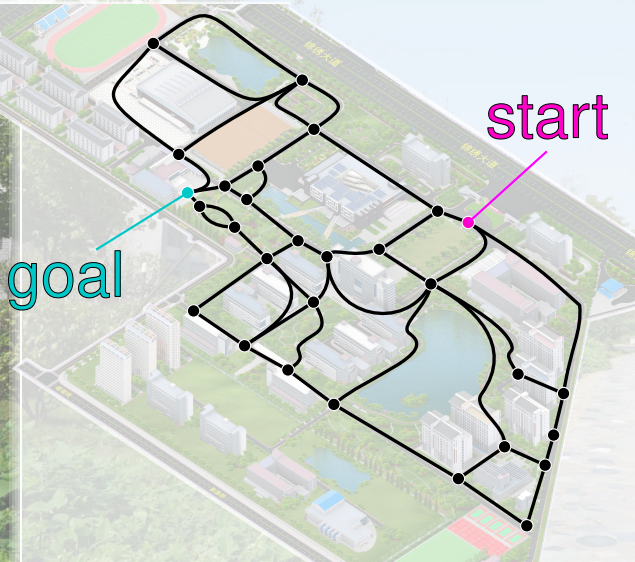
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).

Find the Shortest Path from Start to Goal



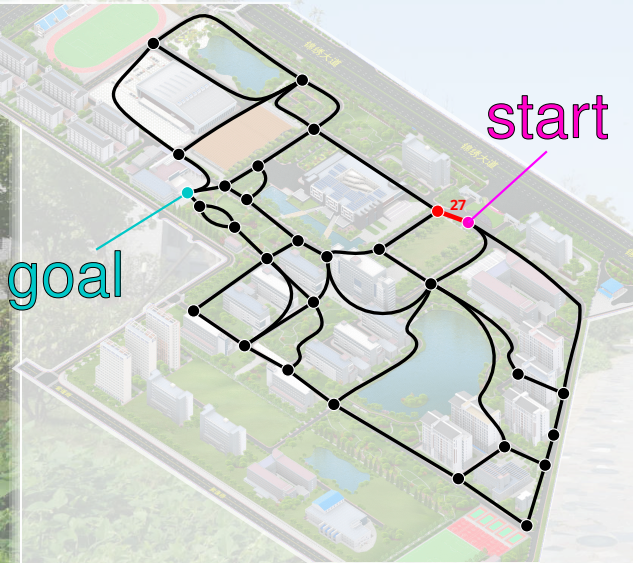
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.

Find the Shortest Path from Start to Goal



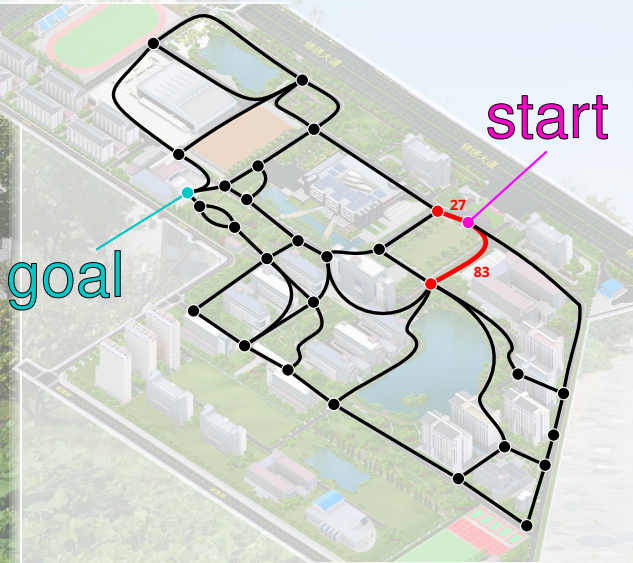
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.

Find the Shortest Path from Start to Goal



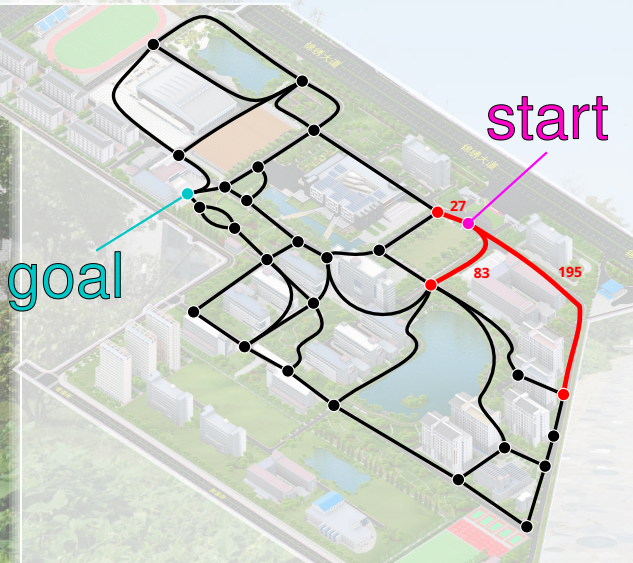
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27s to this intersection.

Find the Shortest Path from Start to Goal



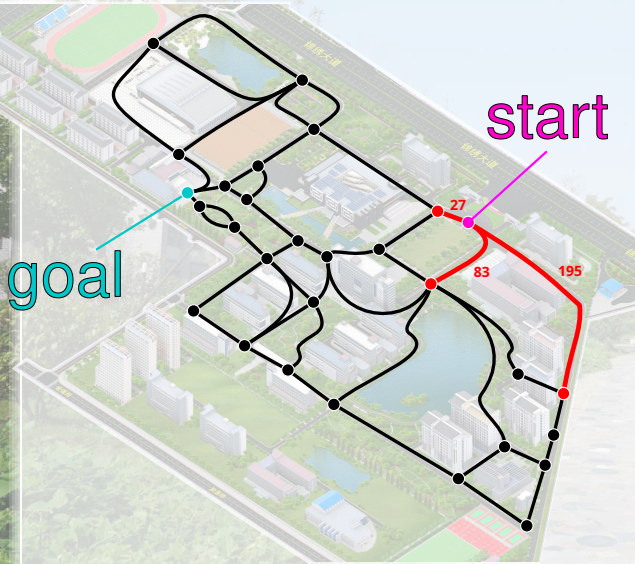
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27s to this intersection.
- Or for 83s to that one.

Find the Shortest Path from Start to Goal



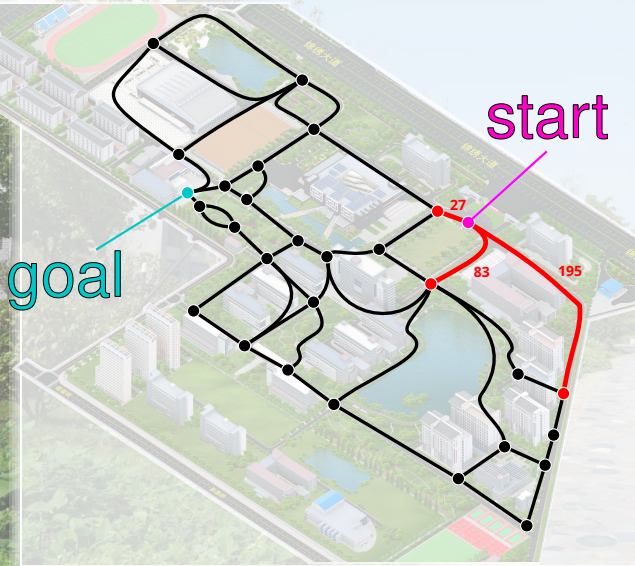
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27s to this intersection.
- Or for 83s to that one.
- Or for 195s to that one.

Find the Shortest Path from Start to Goal



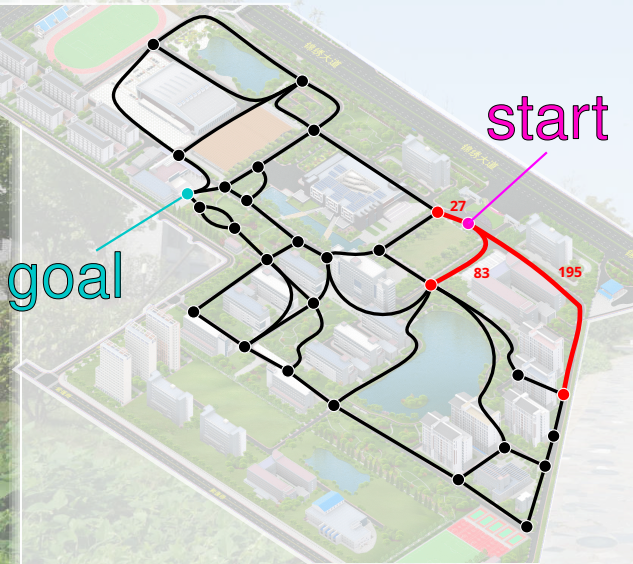
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- For example, I could walk for 27s to this intersection.
- Or for 83s to that one.
- Or for 195s to that one.
- Which one should we pick?

Find the Shortest Path from Start to Goal



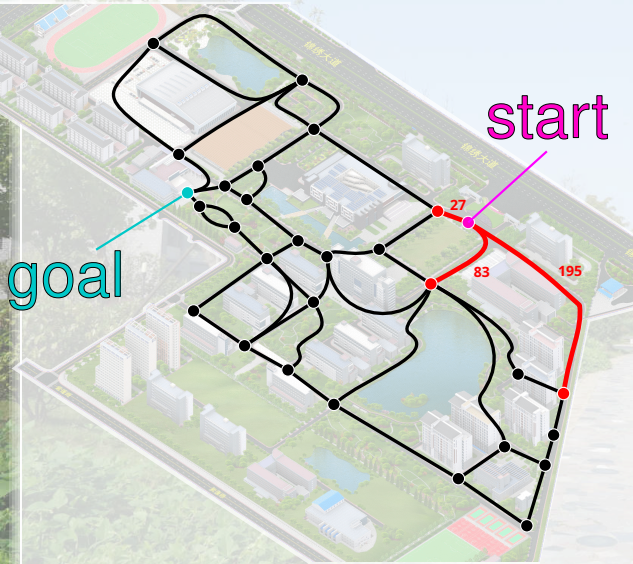
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) 27s, (b) 83s, (c) 195s.

Find the Shortest Path from Start to Goal



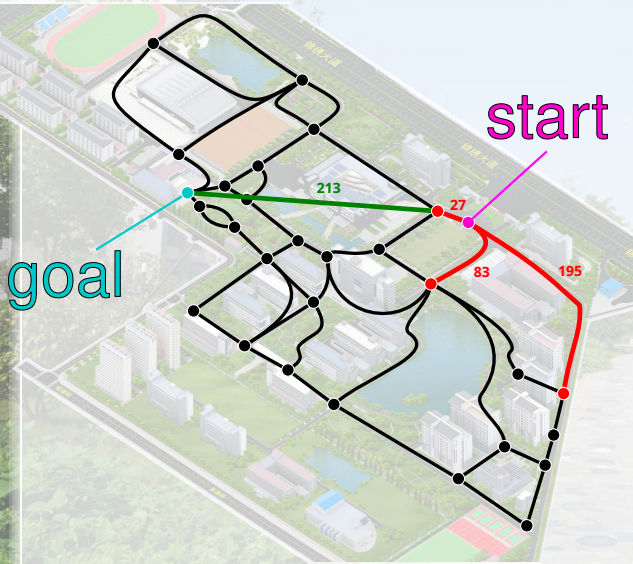
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) 27s, (b) 83s, (c) 195s.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.

Find the Shortest Path from Start to Goal



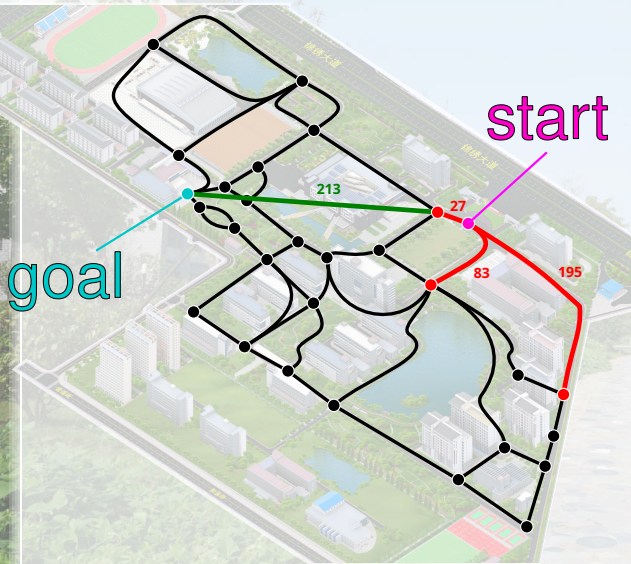
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) 27s, (b) 83s, (c) 195s.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



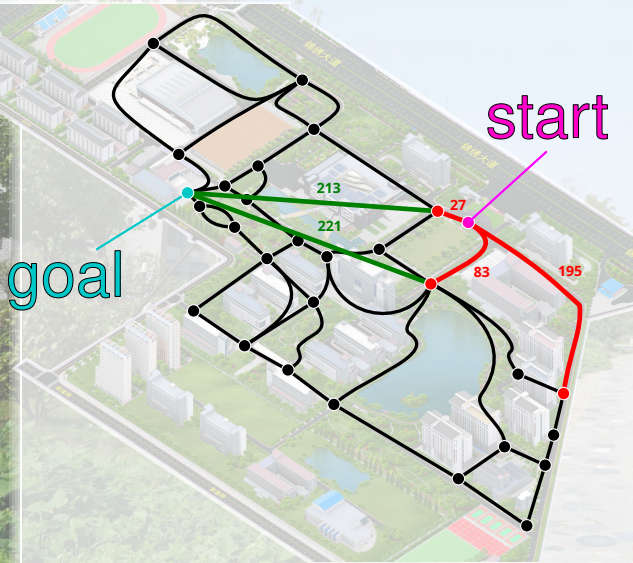
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) 27s, (b) 83s, (c) 195s.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



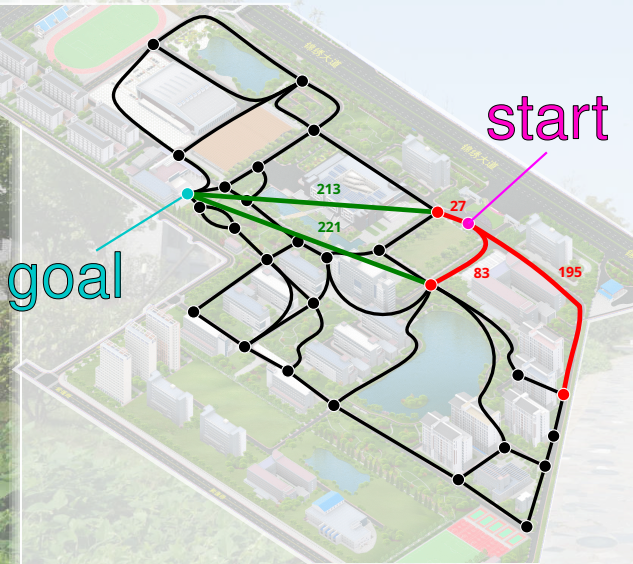
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



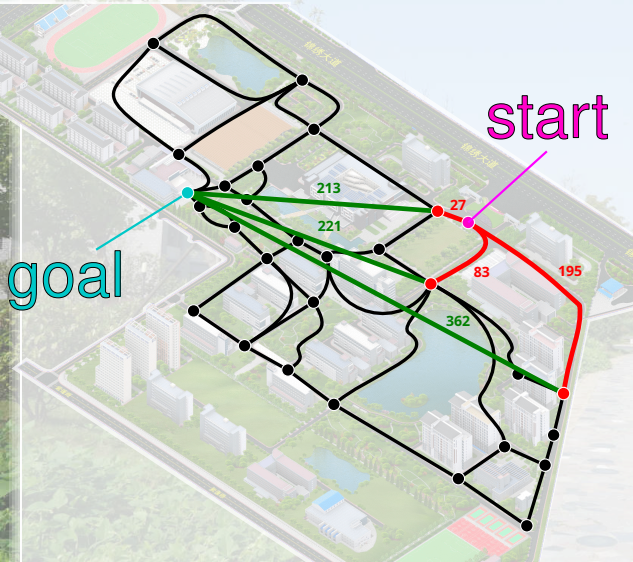
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



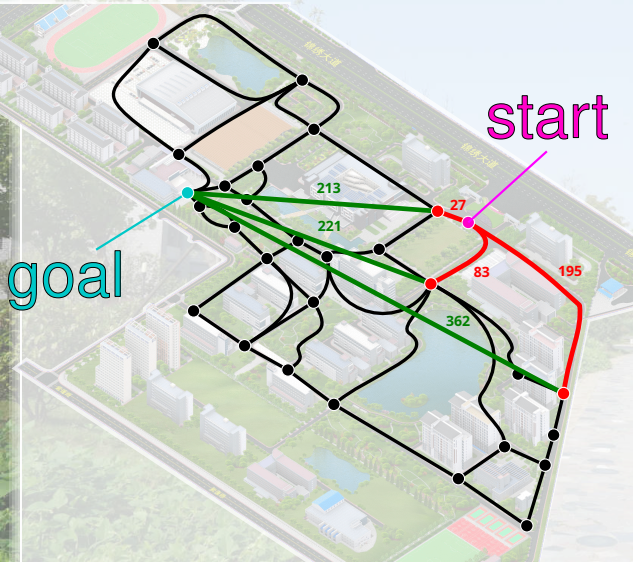
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



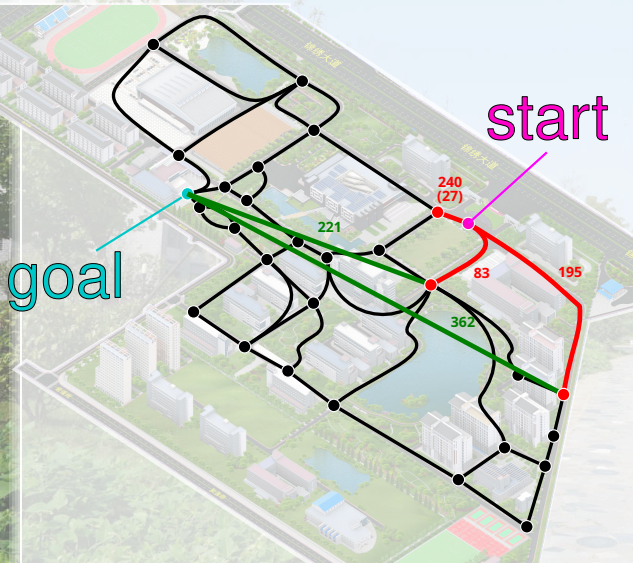
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



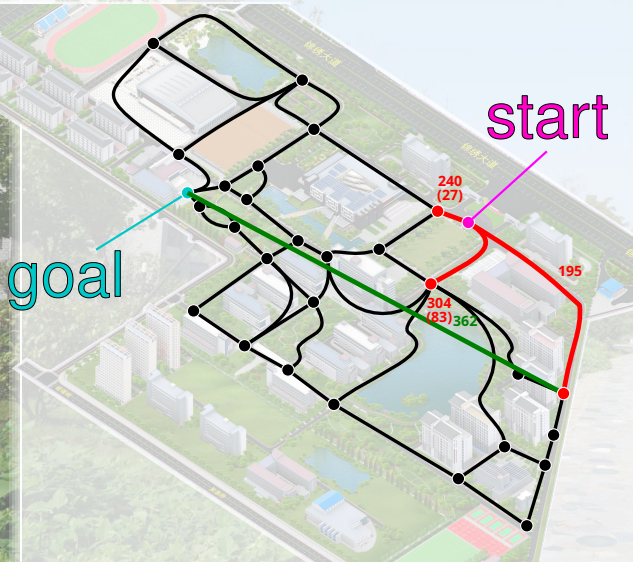
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (as the crow flies) to the 食堂.
- The actual walking distance can never be shorter than that.

Find the Shortest Path from Start to Goal



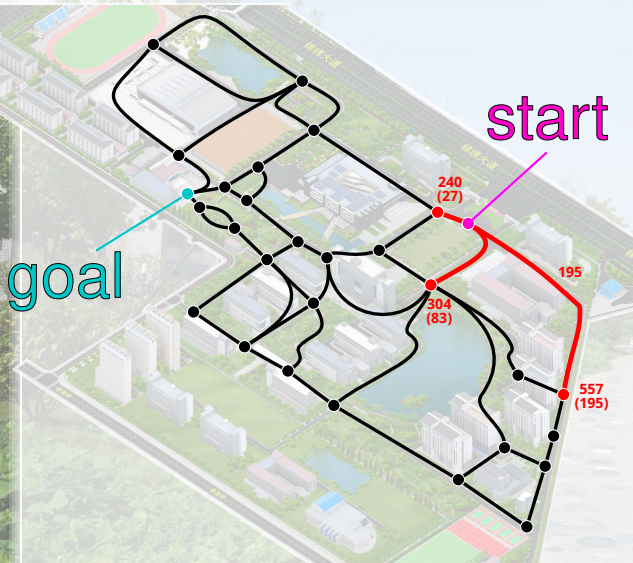
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
 - (a) $240s = 27s + 213s$, (b) $83s + 221s$,
 - (c) $195s + 362s$.

Find the Shortest Path from Start to Goal



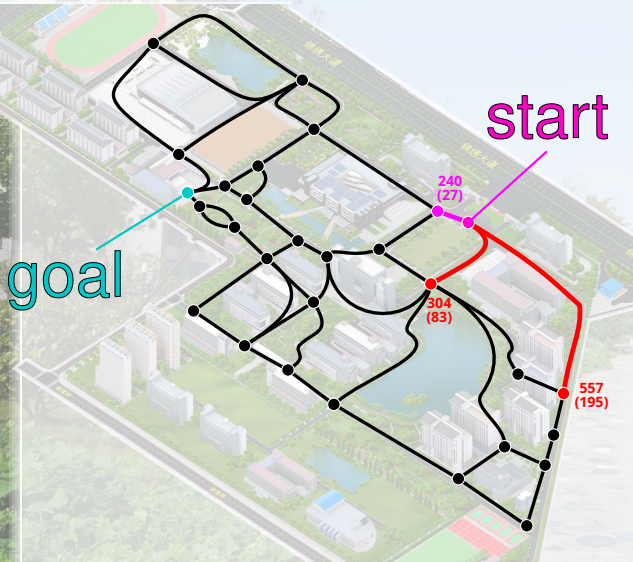
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
 - (a) $240s = 27s + 213s$,
 - (b) $304s = 83s + 221s$, (c) $195s + 362s$.

Find the Shortest Path from Start to Goal



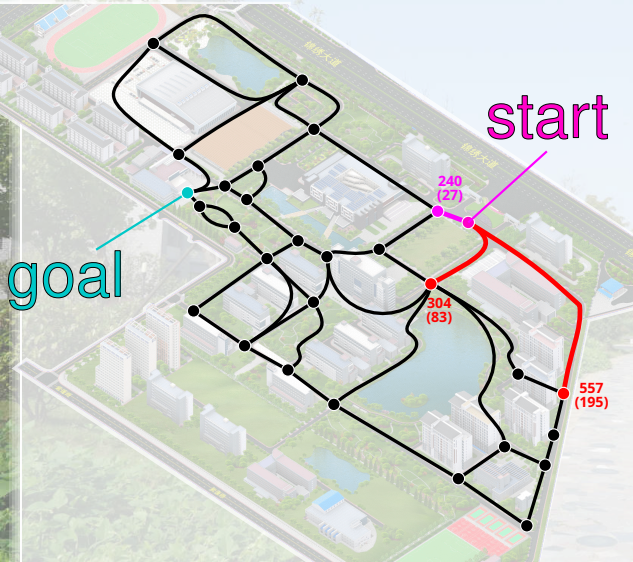
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
 - (a) $240s = 27s + 213s$,
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$.

Find the Shortest Path from Start to Goal



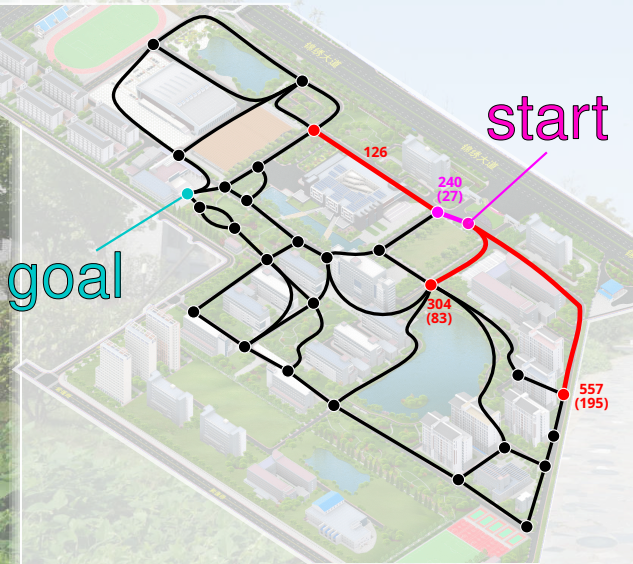
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 choices:
 - (a) $240s = 27s + 213s$,
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$.
- The most interesting candidate for the first step is clearly (a).

Find the Shortest Path from Start to Goal



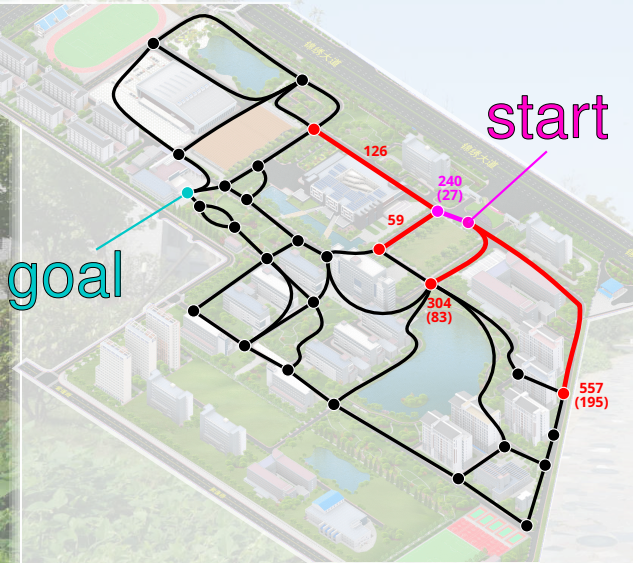
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$.

Find the Shortest Path from Start to Goal



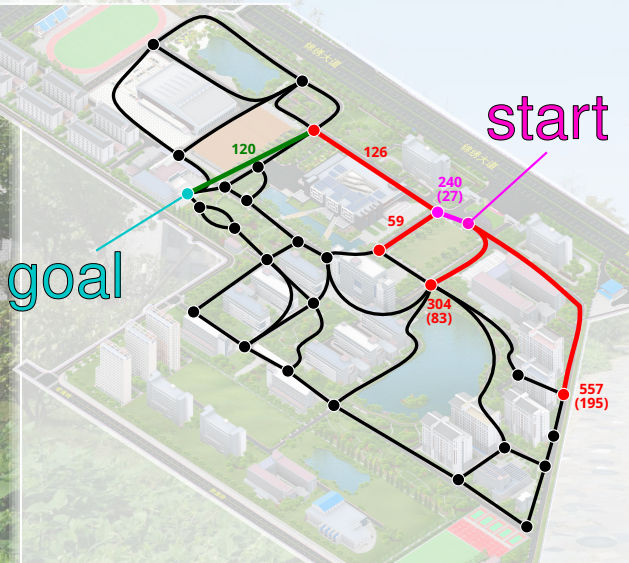
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
(b) $304s = 83s + 221s$,
(c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for $126s$.

Find the Shortest Path from Start to Goal



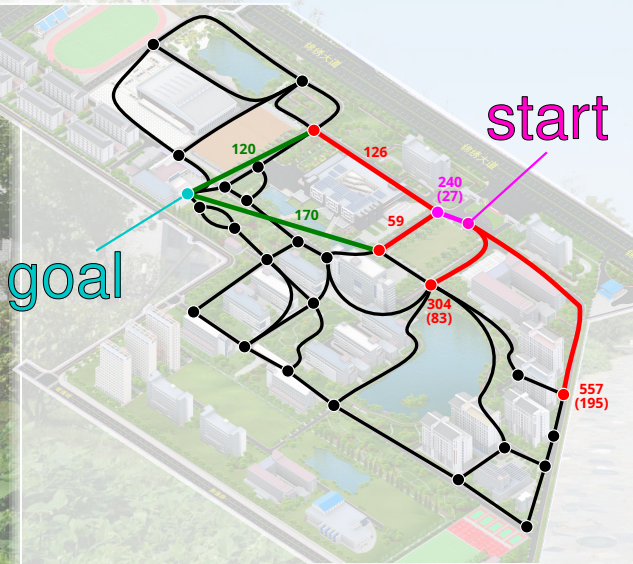
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
(b) $304s = 83s + 221s$,
(c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for 126s or to (e) by walking for 59s.

Find the Shortest Path from Start to Goal



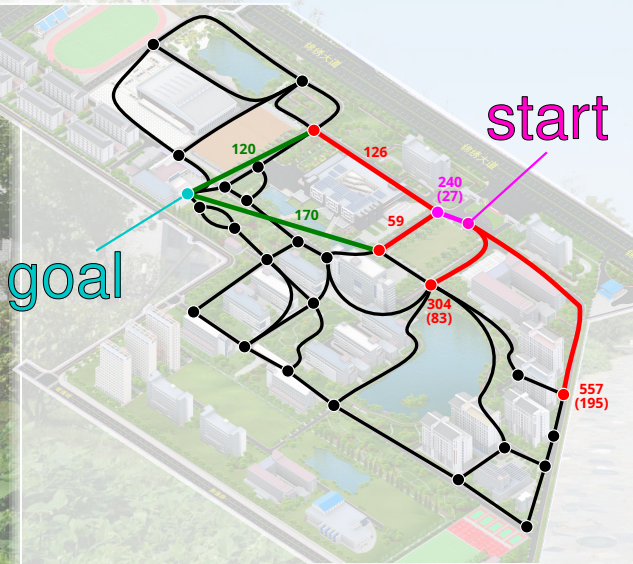
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- We have 2 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$.
- From (a), we can go to (d) by walking for $126s$ or to (e) by walking for $59s$.
- For both we compute the distance as the crow flies.

Find the Shortest Path from Start to Goal



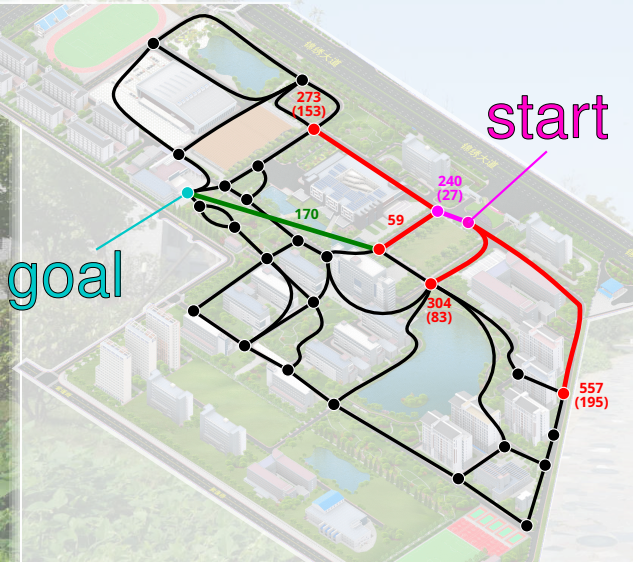
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for 126s or to (e) by walking for 59s.
- For both we compute the distance as the crow flies.

Find the Shortest Path from Start to Goal



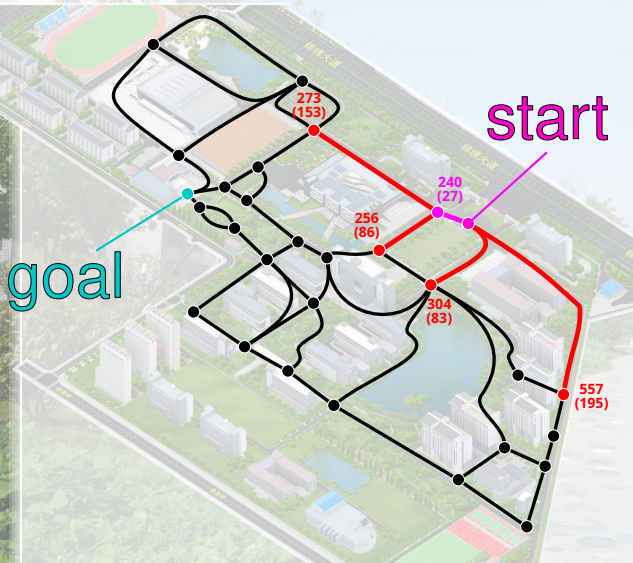
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for 126s or to (e) by walking for 59s.
- For both we compute the distance as the crow flies.
- We had 2 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$.

Find the Shortest Path from Start to Goal



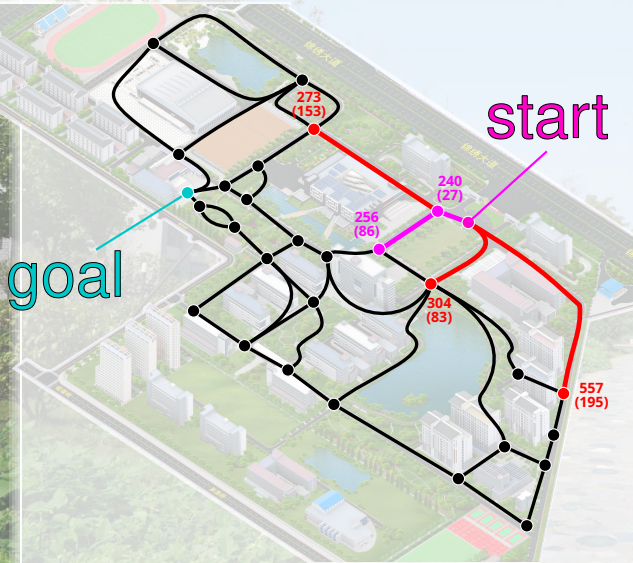
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for 126s or to (e) by walking for 59s.
- For both we compute the distance as the crow flies.
- We now have 3 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.

Find the Shortest Path from Start to Goal



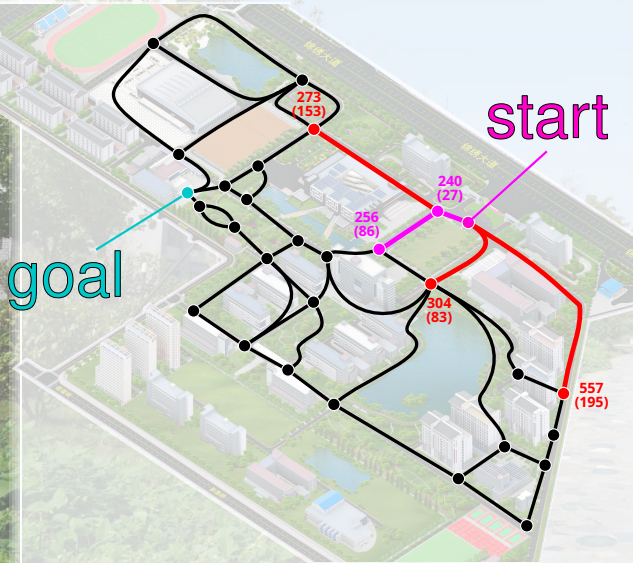
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- The most interesting candidate for the first step is clearly (a).
- From (a), we can go to (d) by walking for 126s or to (e) by walking for 59s.
- For both we compute the distance as the crow flies.
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (e) $256s = 86s + 170s$.

Find the Shortest Path from Start to Goal



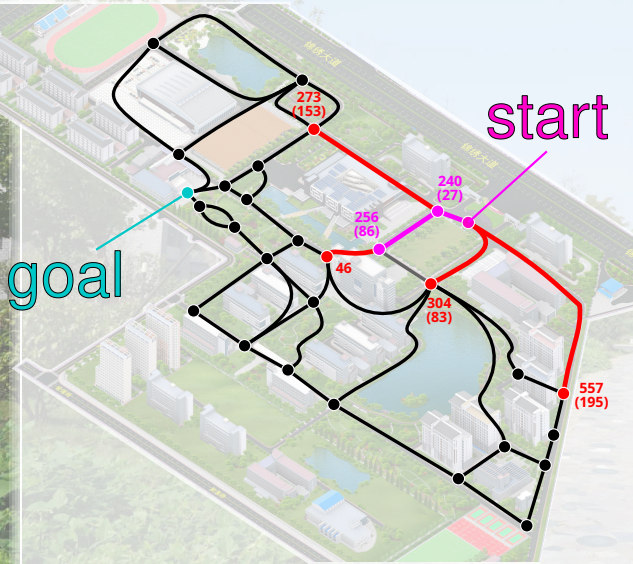
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (e) $256s = 86s + 170s$.
- (e) it is...

Find the Shortest Path from Start to Goal



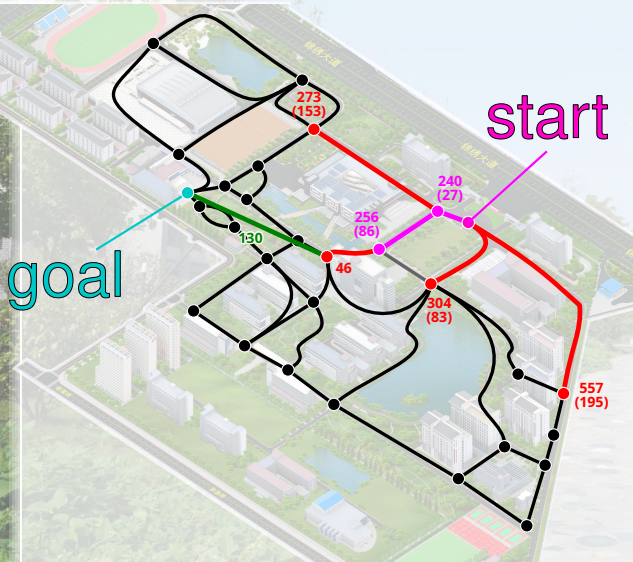
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (e) it is...
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.

Find the Shortest Path from Start to Goal



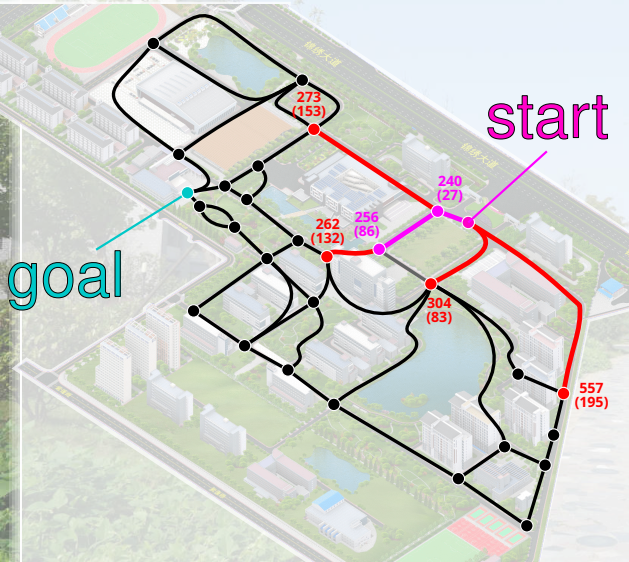
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From this point (e), we only have one reasonable choice where to go next.

Find the Shortest Path from Start to Goal



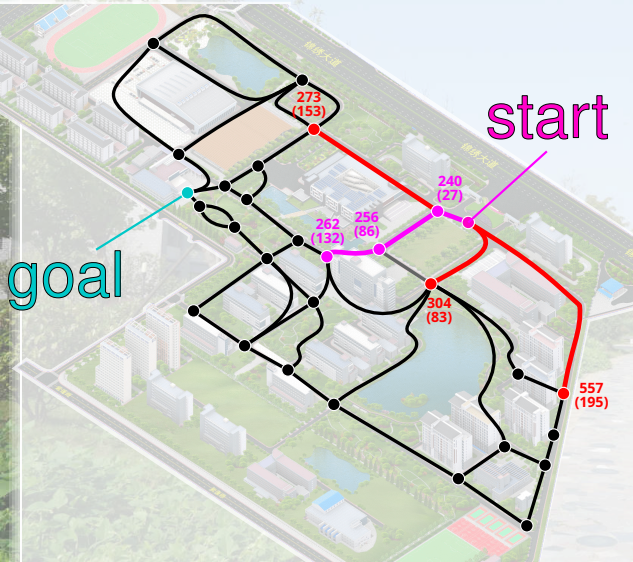
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From this point (e), we only have one reasonable choice where to go next.
- And we again compute the airline distance to the goal.

Find the Shortest Path from Start to Goal



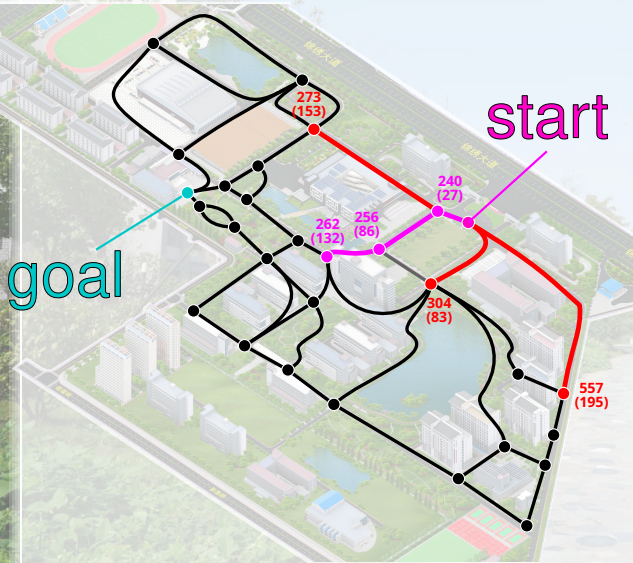
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From this point (e), we only have one reasonable choice where to go next.
- And we again compute the airline distance to the goal.
- We now have 4 choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (f) $262s = 132s + 130s$.

Find the Shortest Path from Start to Goal



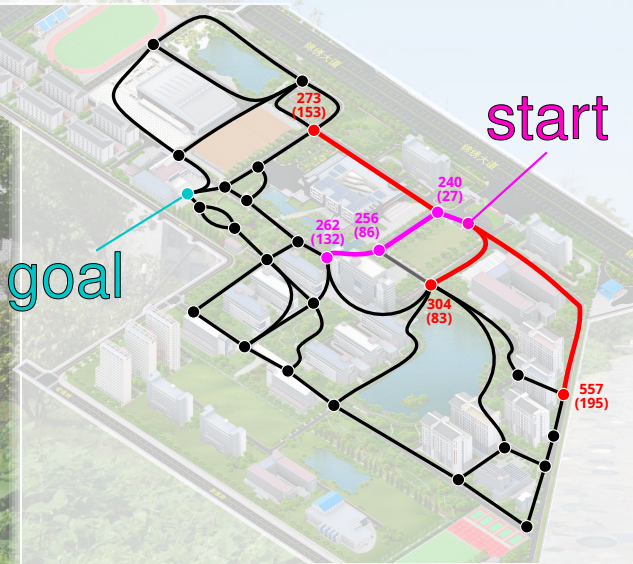
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- We now have 4 choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (f) $262s = 132s + 130s$.
- (f) it is...

Find the Shortest Path from Start to Goal



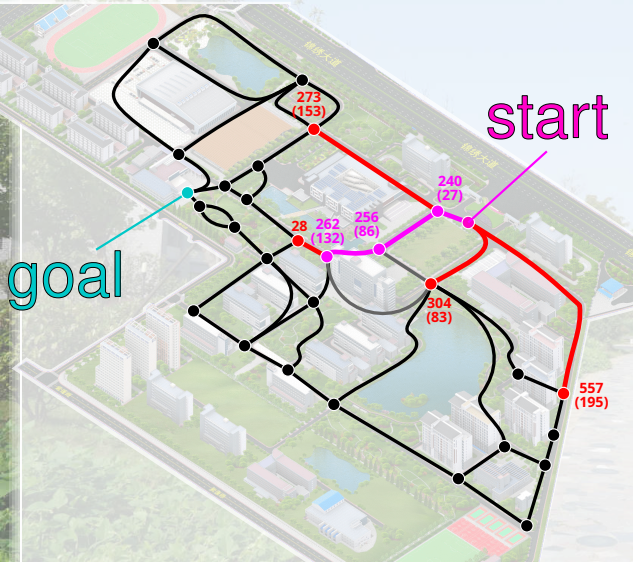
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.

Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is. . .
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

Find the Shortest Path from Start to Goal

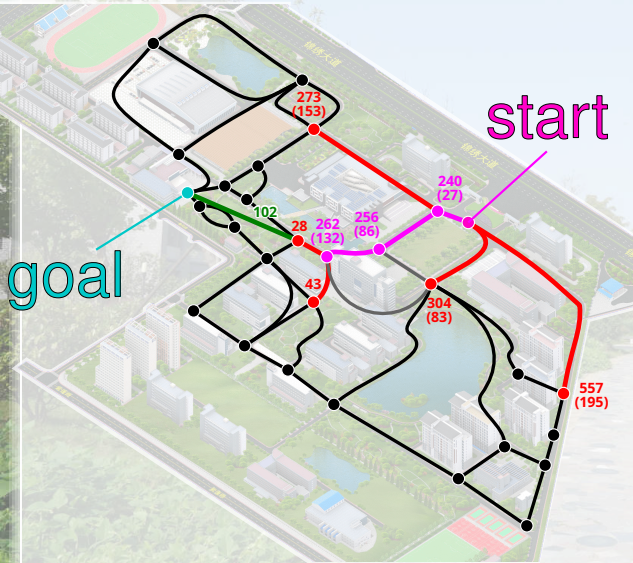


- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is. . .
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.



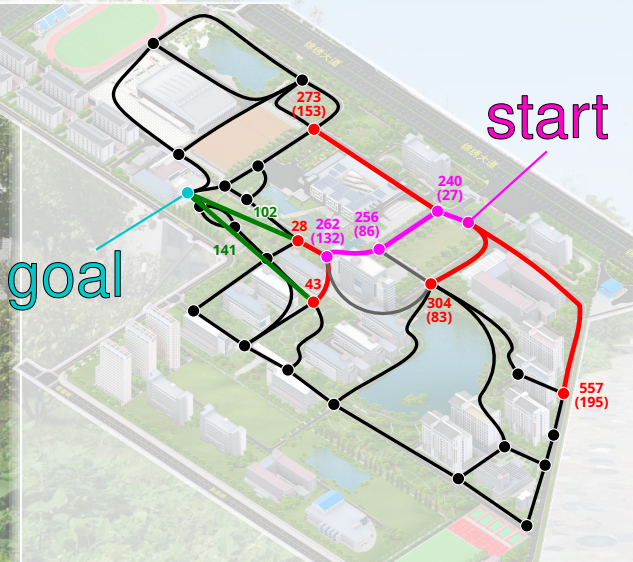
- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- And we again compute the **airline distance** to the **goal**.
- (f) it is...
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

Find the Shortest Path from Start to Goal



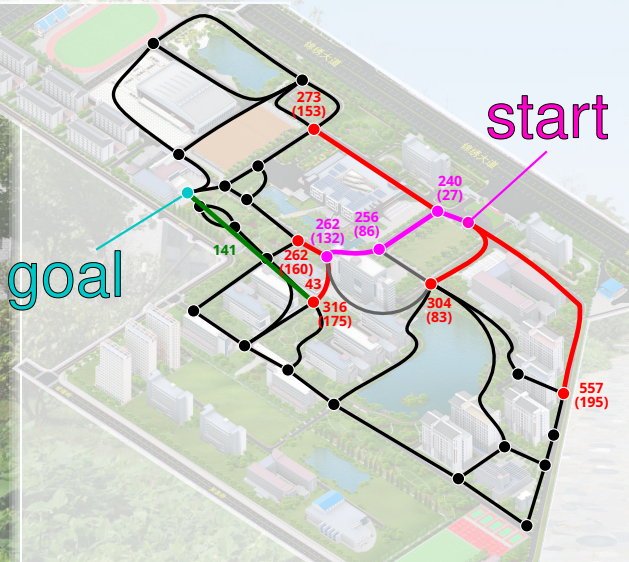
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

Find the Shortest Path from Start to Goal



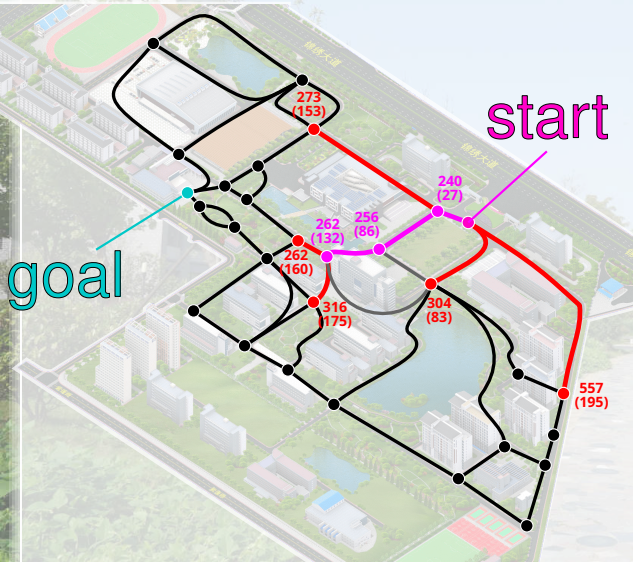
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- We have 3 remaining unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$.
- From (f), we have two possible choices to continue.

Find the Shortest Path from Start to Goal



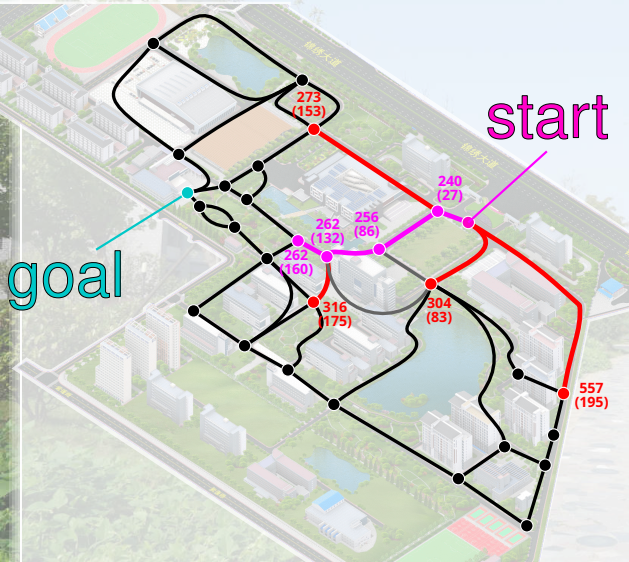
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- From (f), we have two possible choices to continue.
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (g) $262s = 160s + 102s$.

Find the Shortest Path from Start to Goal



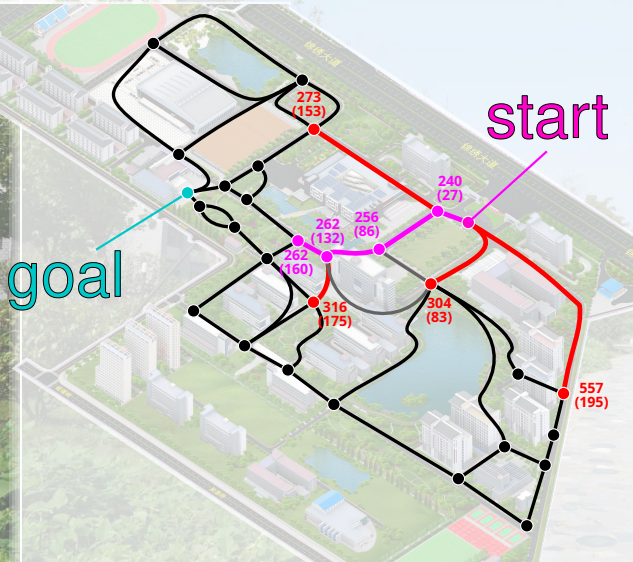
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- (f) it is...
- From (f), we have two possible choices to continue.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (g) $262s = 160s + 102s$,
 - (h) $316s = 175s + 141s$.

Find the Shortest Path from Start to Goal



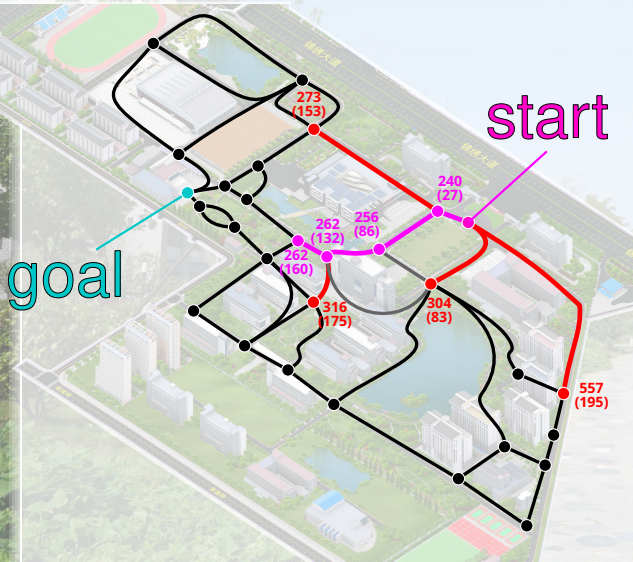
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we again compute the airline distance to the goal.
- From (f), we have two possible choices to continue.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (g) $262s = 160s + 102s$,
 - (h) $316s = 175s + 141s$.
- (g) it is ...

Find the Shortest Path from Start to Goal



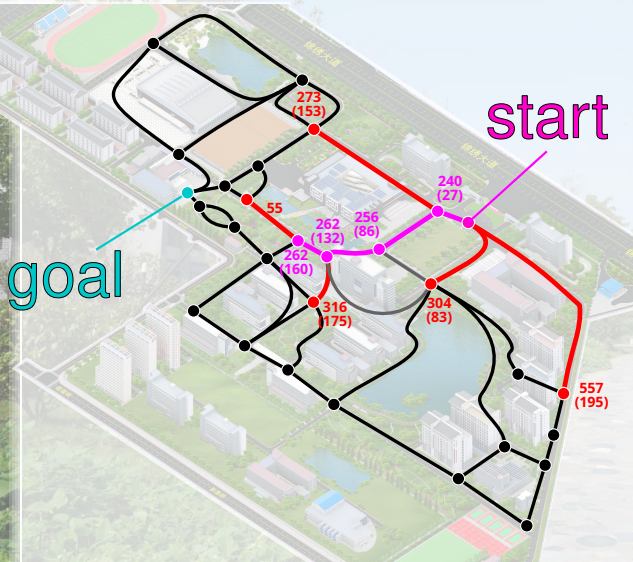
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.

Find the Shortest Path from Start to Goal



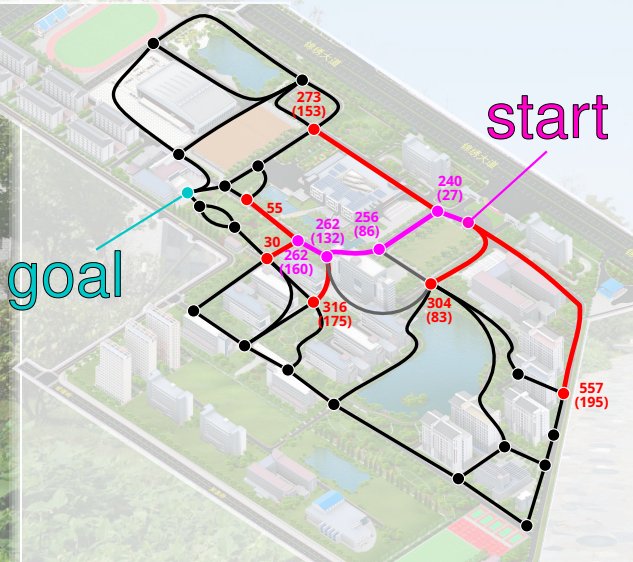
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.
- And we get two new choices.

Find the Shortest Path from Start to Goal



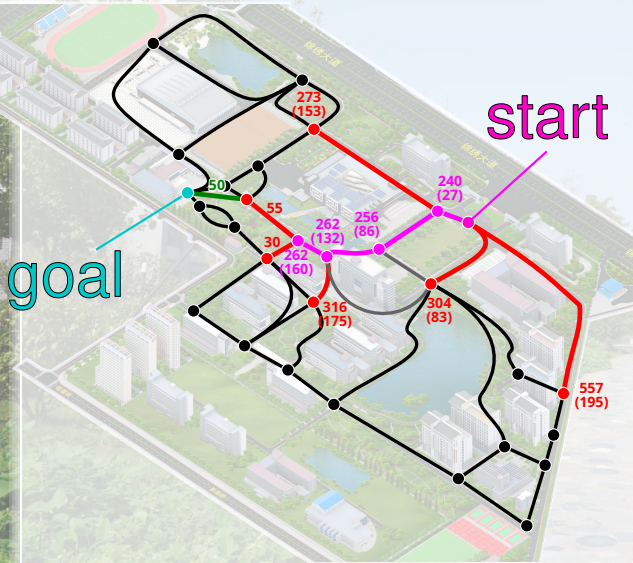
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.
- And we get two new choices.

Find the Shortest Path from Start to Goal



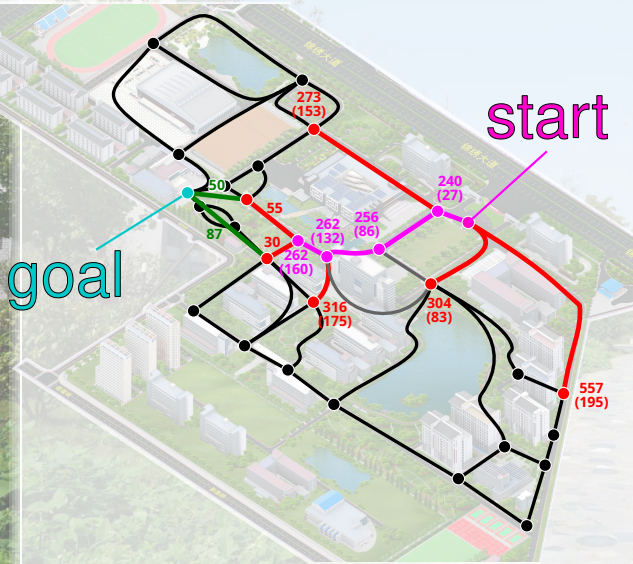
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.
- And we get two new choices.

Find the Shortest Path from Start to Goal



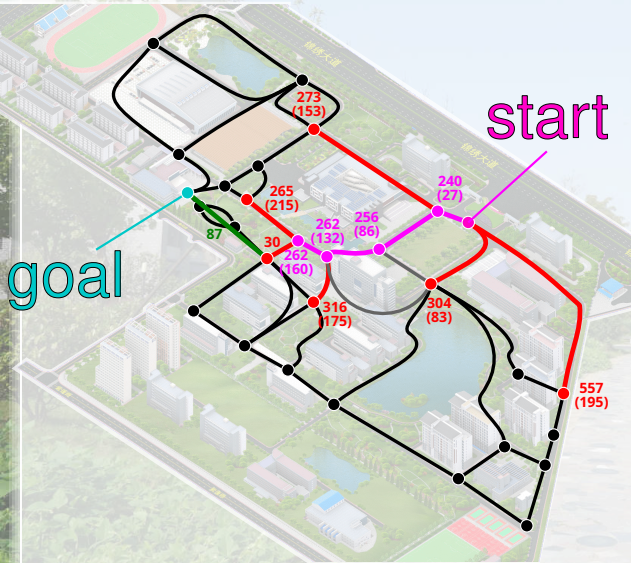
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.
- And we get two new choices.

Find the Shortest Path from Start to Goal



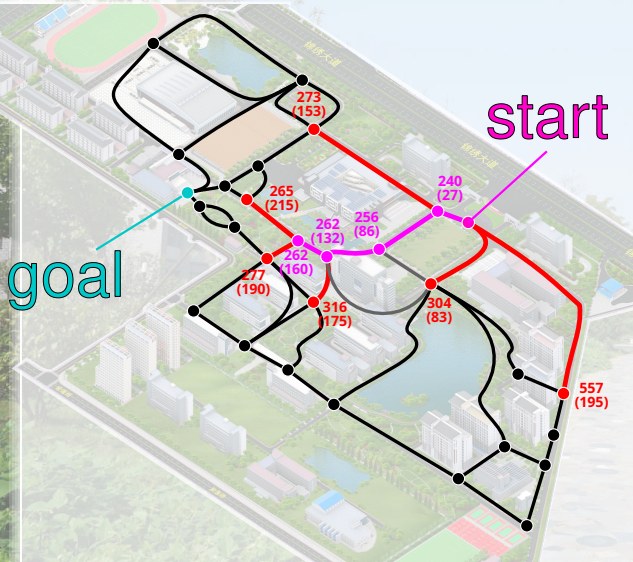
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- (g) it is...
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$.
- And we get two new choices.

Find the Shortest Path from Start to Goal



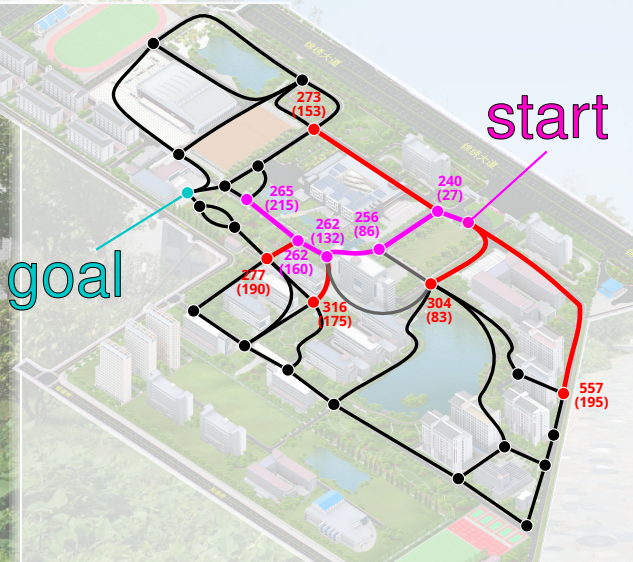
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (i) $265s = 215s + 50s$.

Find the Shortest Path from Start to Goal



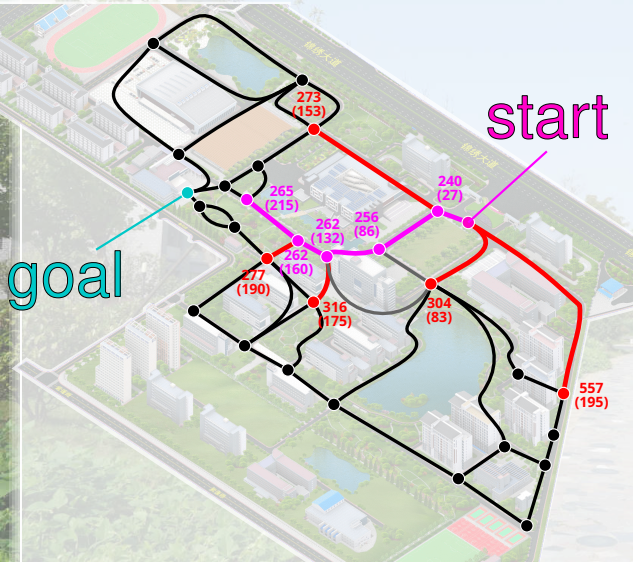
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (i) $265s = 215s + 50s$,
 - (j) $277s = 190s + 87s$.

Find the Shortest Path from Start to Goal



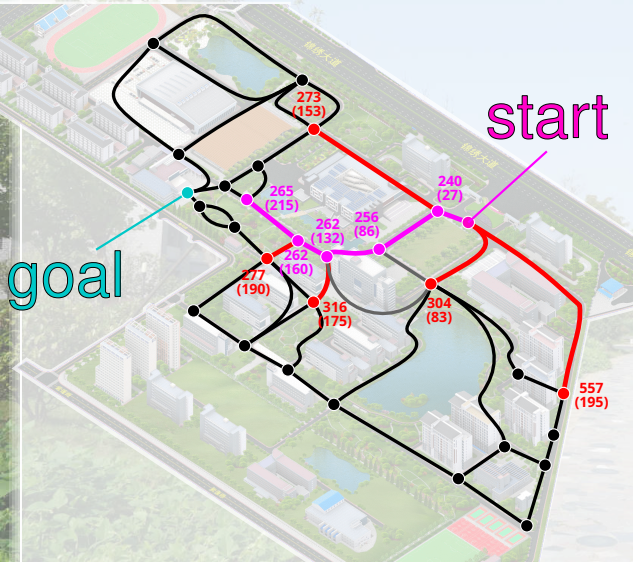
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- And we get two new choices.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (i) $265s = 215s + 50s$,
 - (j) $277s = 190s + 87s$.
- (i) it is...

Find the Shortest Path from Start to Goal



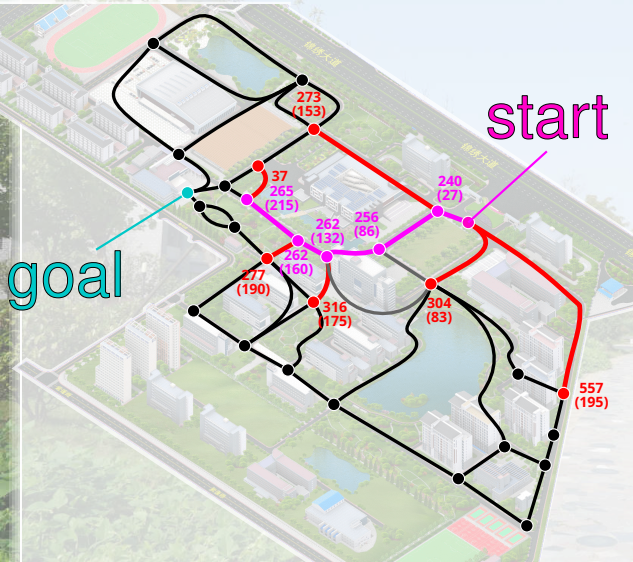
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.

Find the Shortest Path from Start to Goal



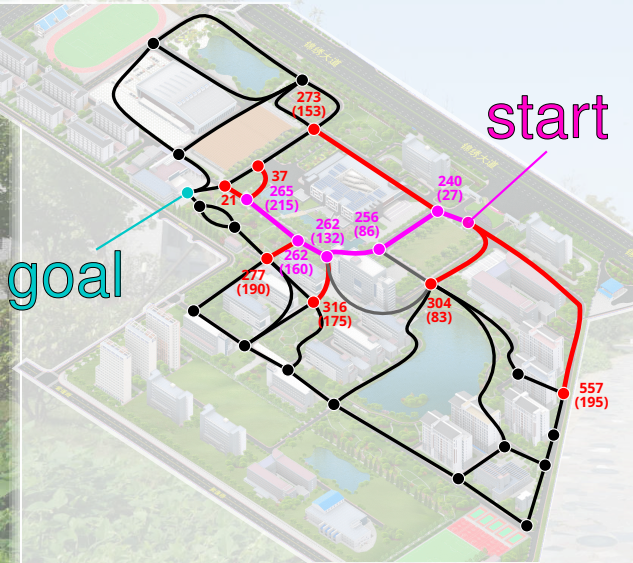
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

Find the Shortest Path from Start to Goal



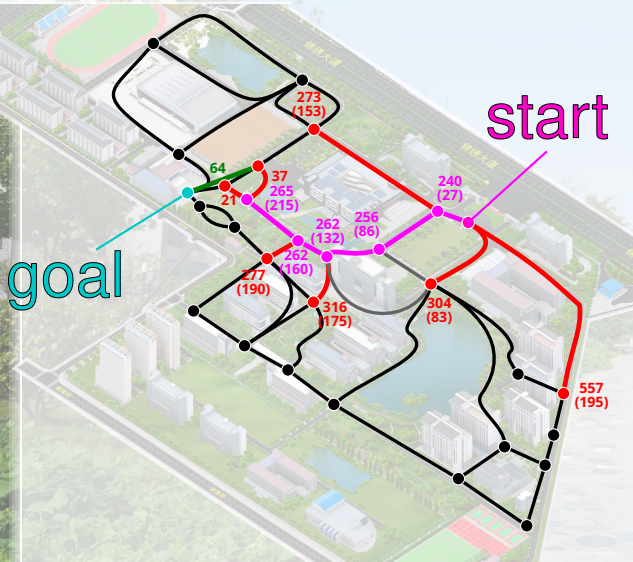
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

Find the Shortest Path from Start to Goal

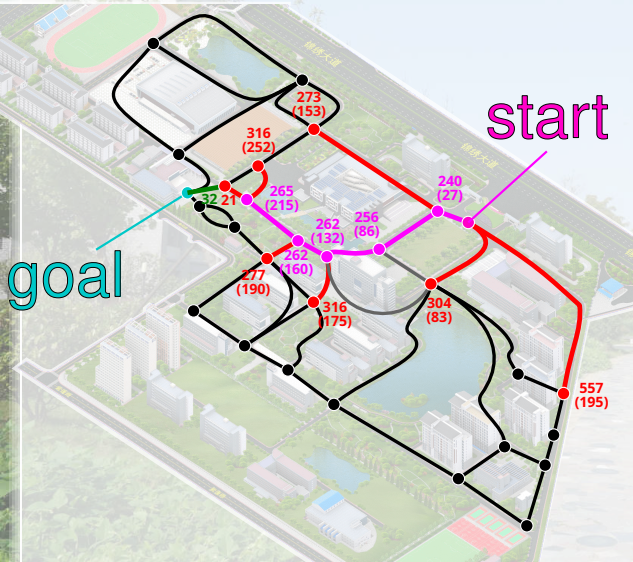


- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.



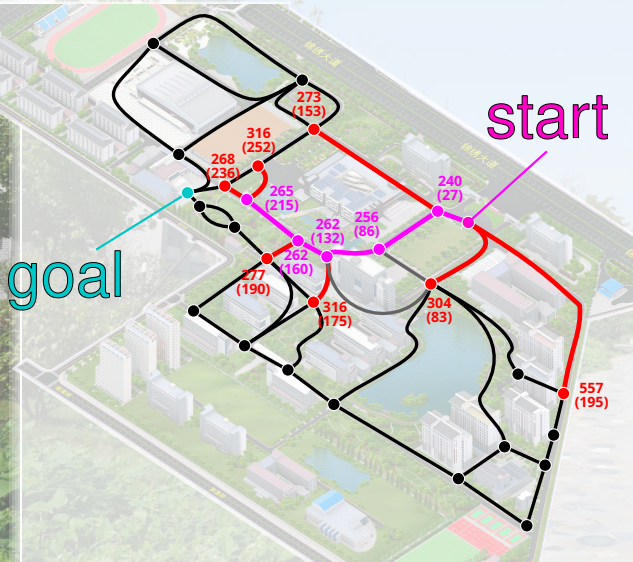
- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- We now have 5 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$.
- From (i), we again got two new choices.

Find the Shortest Path from Start to Goal



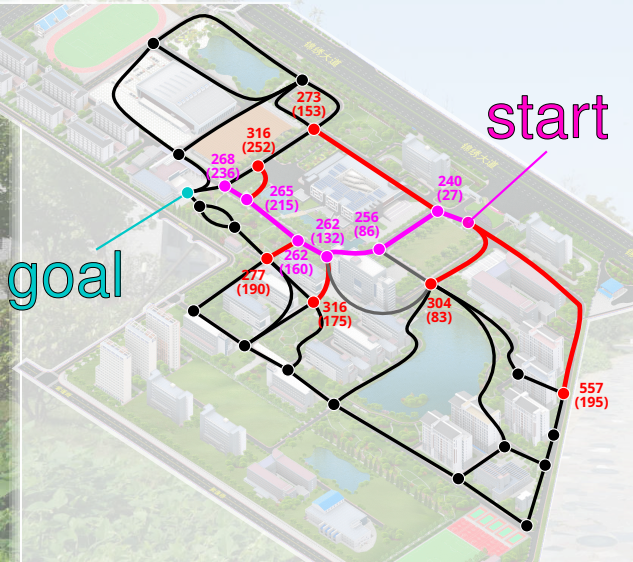
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.

Find the Shortest Path from Start to Goal



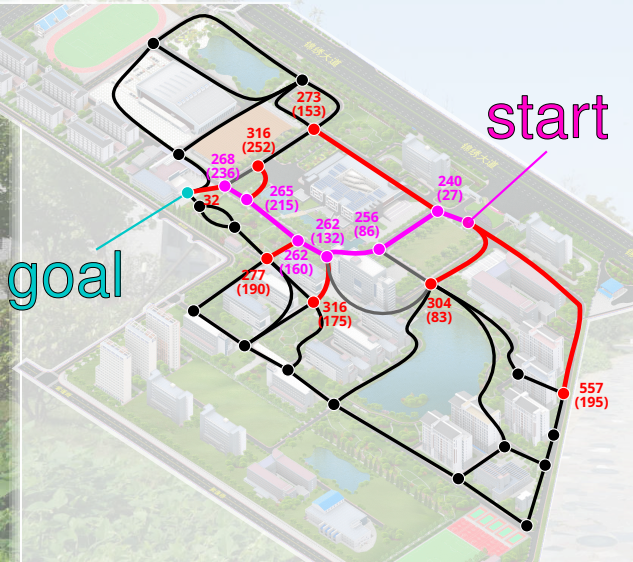
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 7 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$,
 - (l) $268s = 236s + 32s$.

Find the Shortest Path from Start to Goal



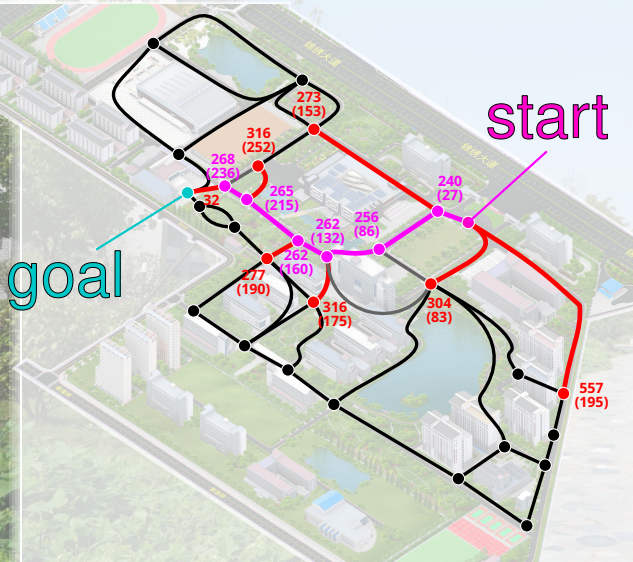
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- From (i), we again got two new choices.
- We now have 7 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$,
 - (l) $268s = 236s + 32s$.
- (l) it is ...

Find the Shortest Path from Start to Goal



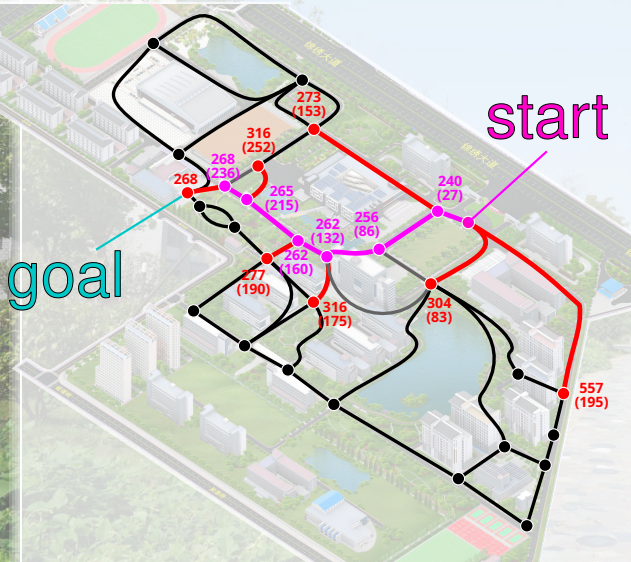
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (l) it is...
- There is only one choice to continue: go to the goal!

Find the Shortest Path from Start to Goal



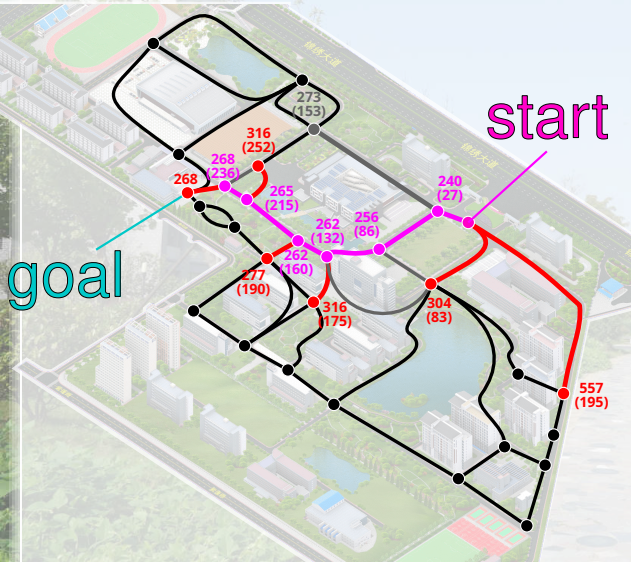
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (l) it is...
- There is only one choice to continue: go to the goal!
- We found a first complete path from start to goal.

Find the Shortest Path from Start to Goal



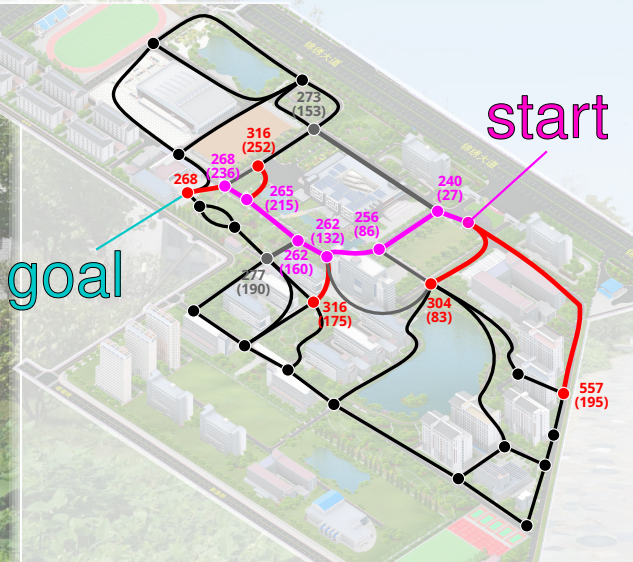
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 6 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- There is only one choice to continue: go to the goal!
- We found a first complete path from start to goal.
- It has the total length $268s$.

Find the Shortest Path from Start to Goal



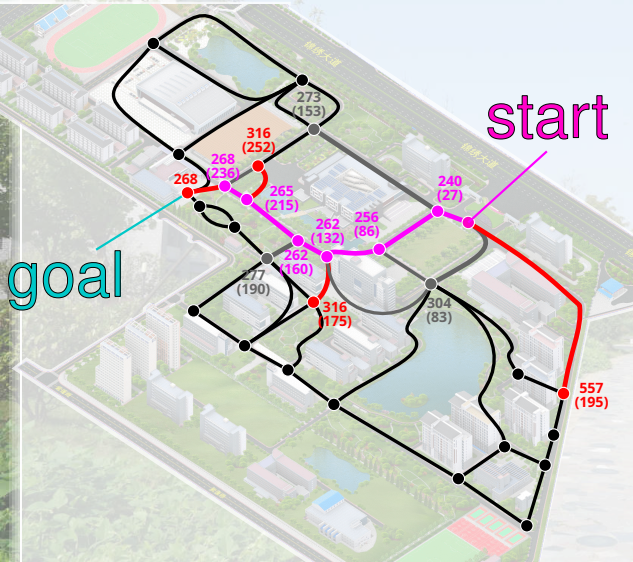
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 56 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (d) definitely will need more than $268s$.

Find the Shortest Path from Start to Goal



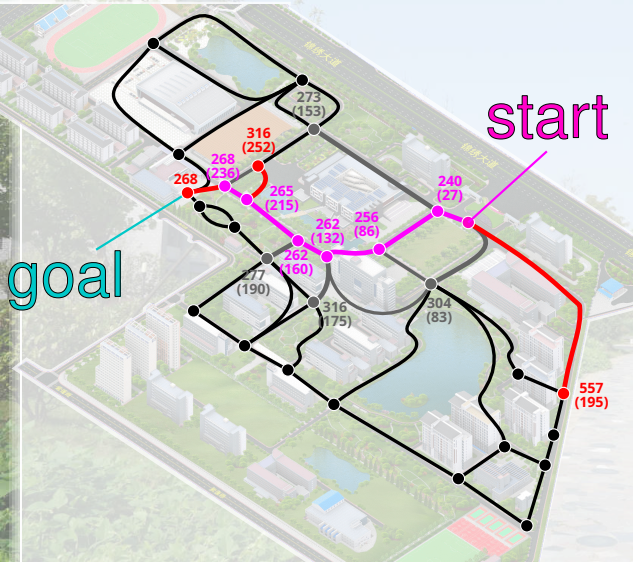
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 4 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (j) definitely will need more than $268s$.

Find the Shortest Path from Start to Goal



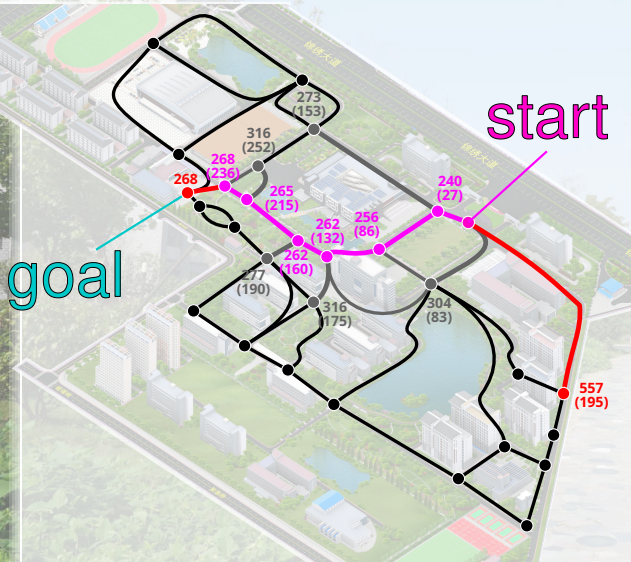
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 3 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (b) definitely will need more than 268s.

Find the Shortest Path from Start to Goal



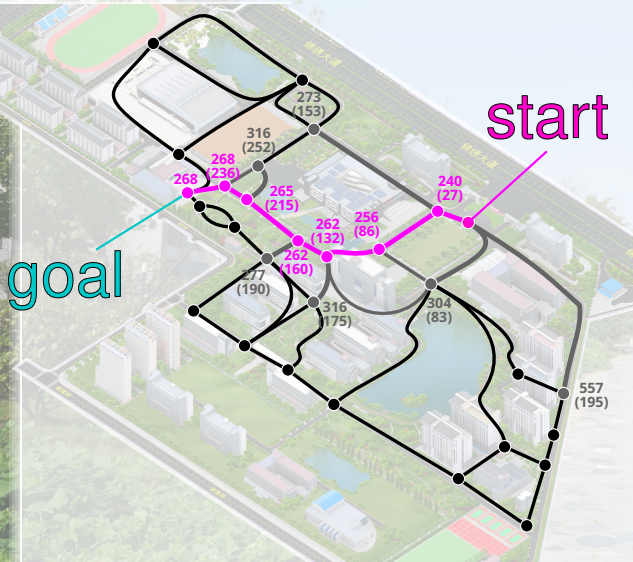
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 2 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (h) definitely will need more than $268s$.

Find the Shortest Path from Start to Goal



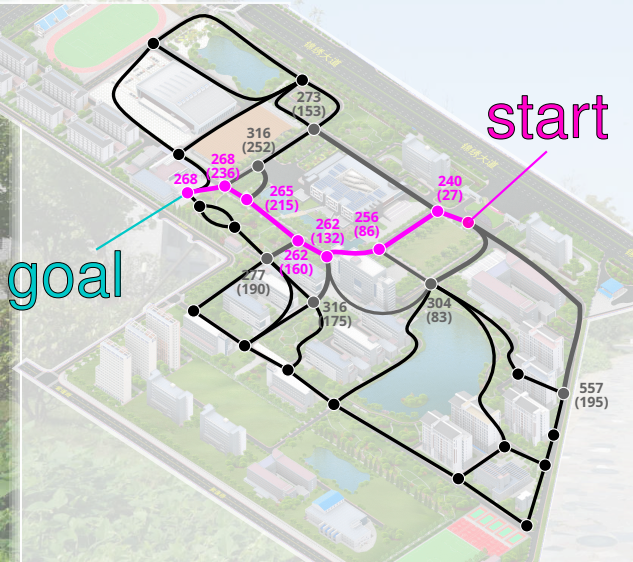
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 1 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (k) definitely will need more than $268s$.

Find the Shortest Path from Start to Goal



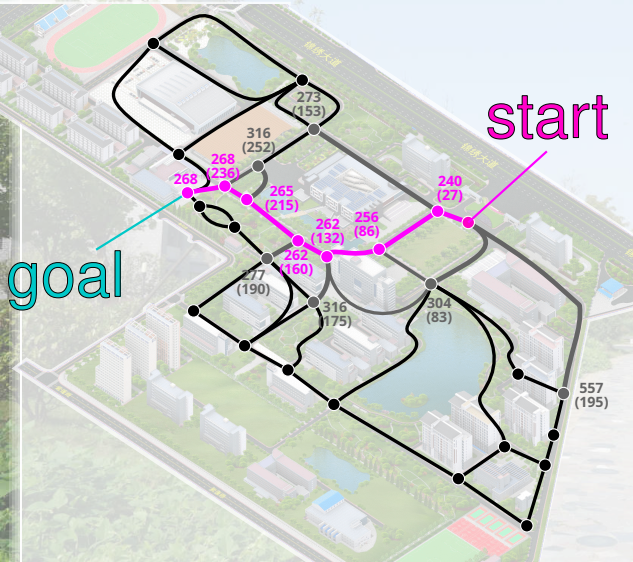
- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- (c) definitely will need more than $268s$.

Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- We found the shortest possible path and it takes 268s.

Find the Shortest Path from Start to Goal



- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- We now have 0 unexplored choices:
 - (b) $304s = 83s + 221s$,
 - (c) $557s = 195s + 362s$,
 - (d) $273s = 153s + 120s$,
 - (h) $316s = 175s + 141s$,
 - (j) $277s = 190s + 87s$,
 - (k) $316s = 252s + 64s$.
- We found the shortest possible path and it takes 268s.
- **Solved.**

The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.

The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.
- The A* Algorithm iteratively constructs a solution.

The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.

The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to “look” at all intersections (nodes) and streets (edges) once.

The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to “look” at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.

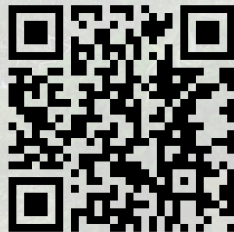
The A* Algorithm for Finding Shortest Paths



- We just applied the A* Algorithm^{40,71}.
- The A* Algorithm iteratively constructs a solution.
- It decides next step to test based on a combination of heuristic and cost.
- In the worst case, it needs to “look” at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.
- (If we look for shortest paths that do not visit any node twice, where the edge distances are not negative, where the number of choices per node is limited, and where the graph and path are not too big – then this algorithm is very efficient.)



Problems that algorithms cannot solve both efficiently *and* exactly



Hard Problems (1)



- There exists a group of problems that are *hard*.

Hard Problems (1)



- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.

Hard Problems (1)



- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.

Hard Problems (1)

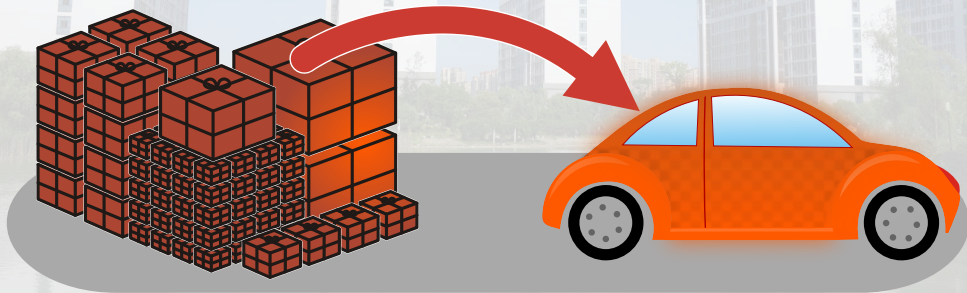


- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.
- Let's look at two quick examples.

Bin Packing Problem



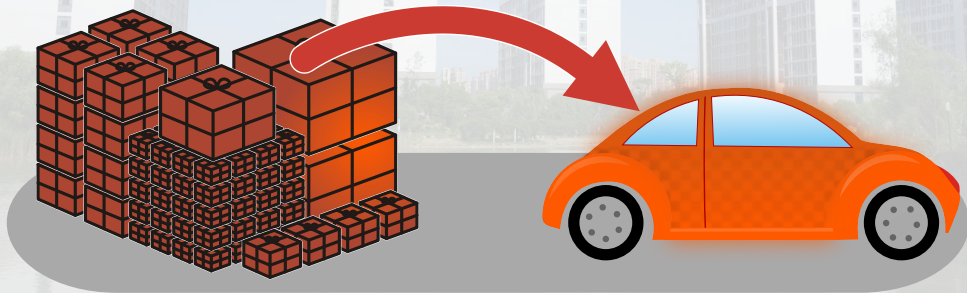
- My car can carry T kg of weight.



Bin Packing Problem



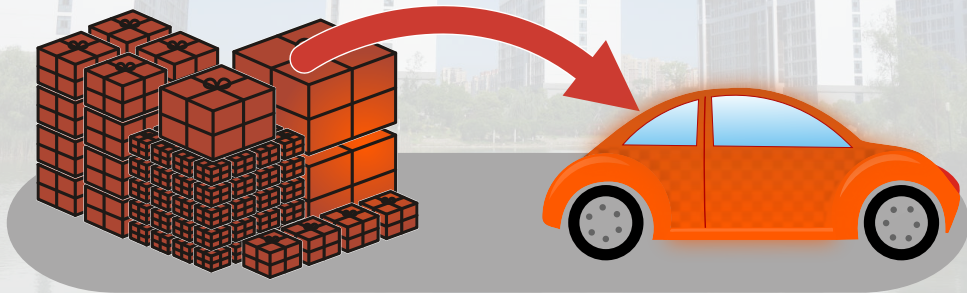
- My car can carry T kg of weight.
- I have n objects, each with weight w_i with i in 1 to n .



Bin Packing Problem

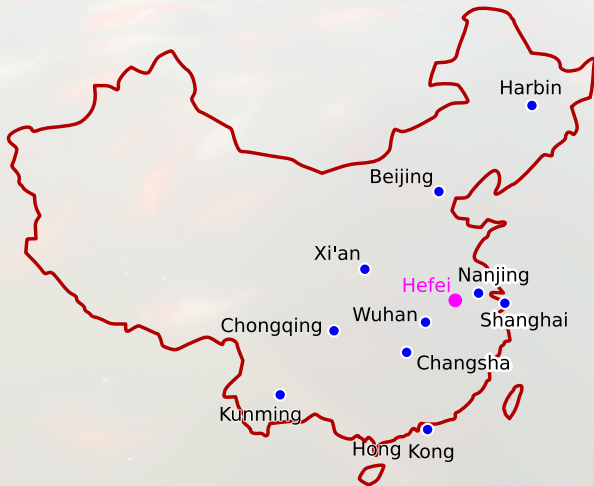


- My car can carry T kg of weight.
- I have n objects, each with weight w_i with i in 1 to n .
- How can I pack my car so that I can carry them from A to B with the fewest possible hauls?^{106,107}



Traveling Salesperson Problem

- Find the shortest path that visits n locations and returns back to its origin. ^{2,55}



Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.



Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?



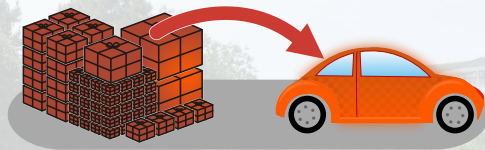
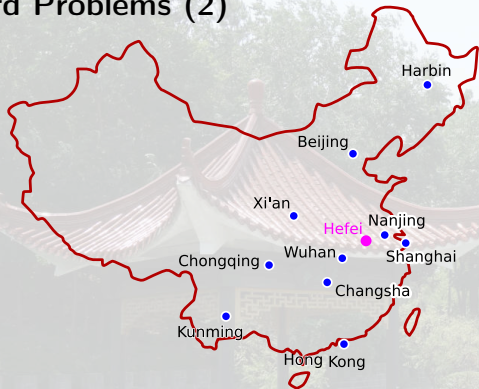
Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

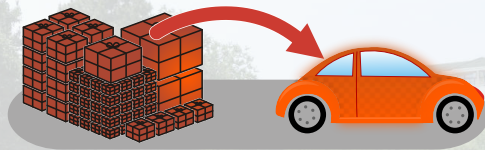
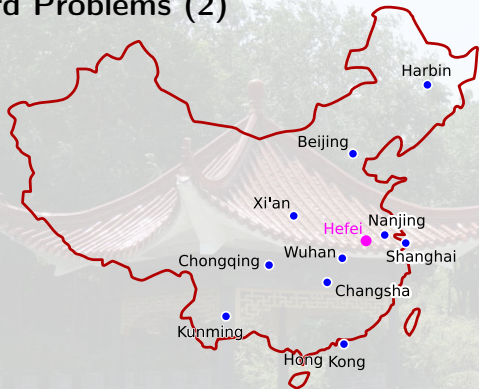
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



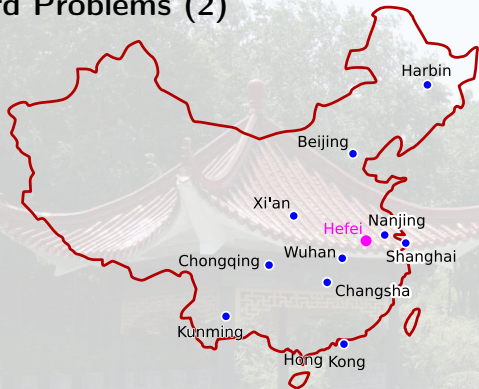
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)

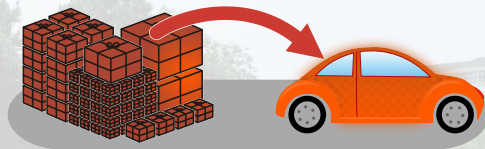


If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



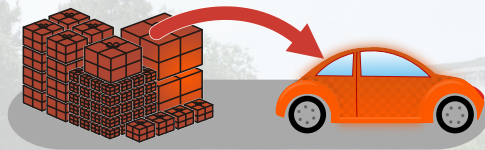
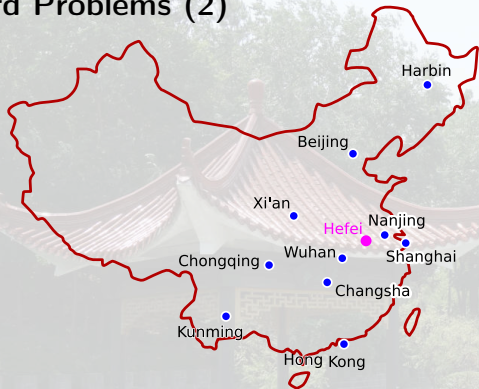
shortest round-trip tour



packing with fewest hauls

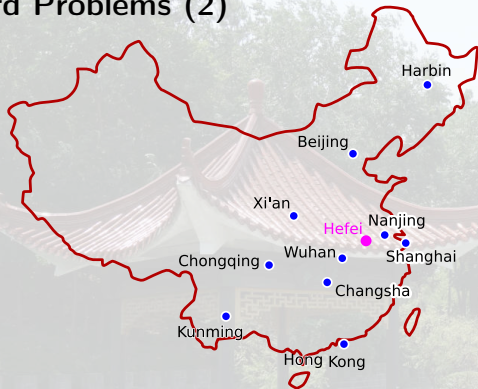
If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)

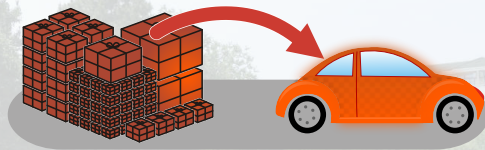


If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the **input size** n .

Hard Problems (2)



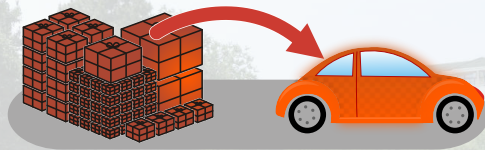
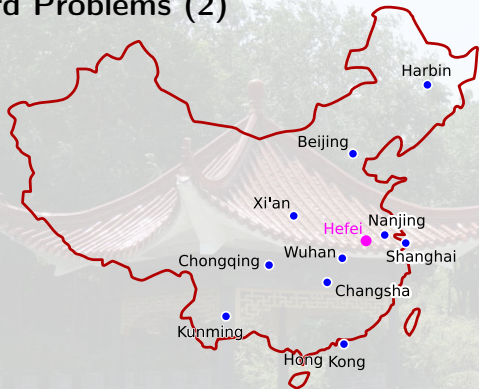
number n of cities to visit



number n of items to pack

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the **input size** n .

Hard Problems (2)

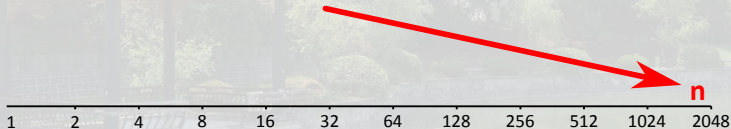


If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)

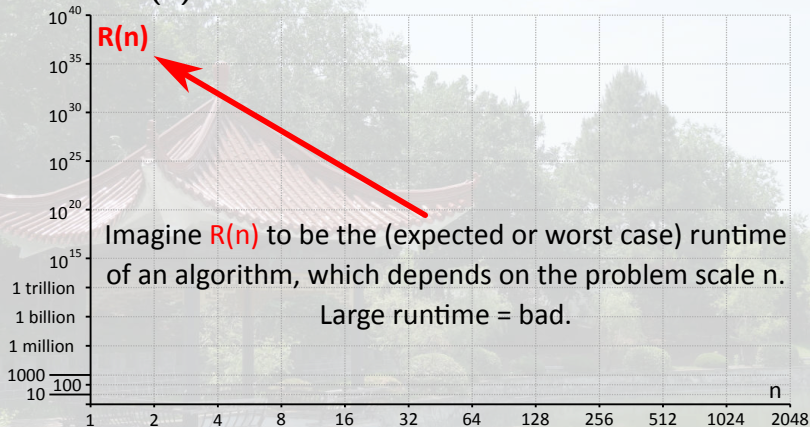


The problem scale n usually corresponding to the number of decision variables.



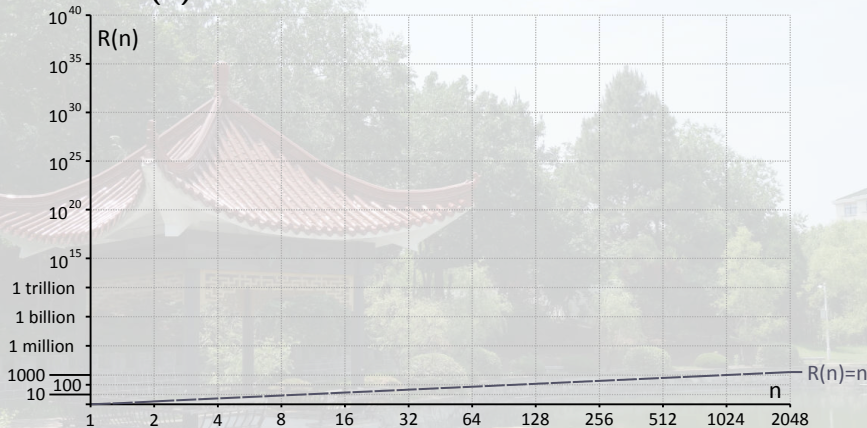
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



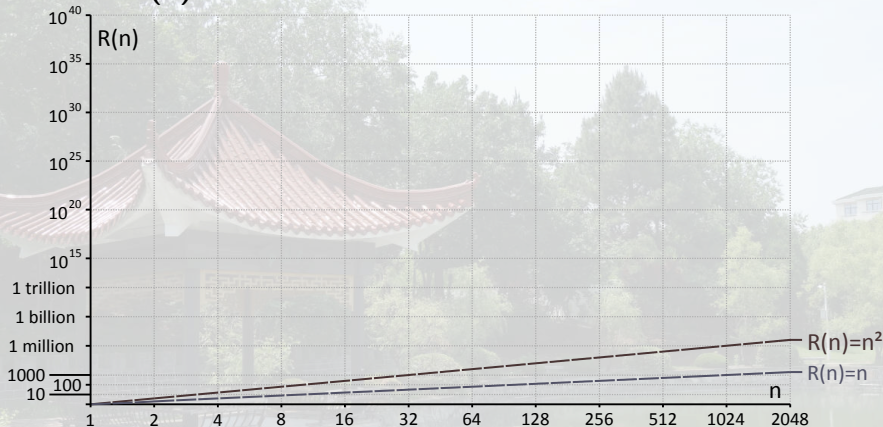
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



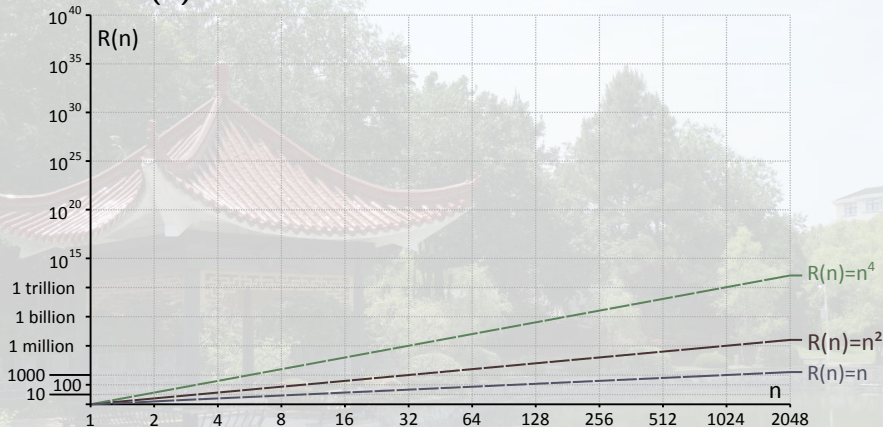
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



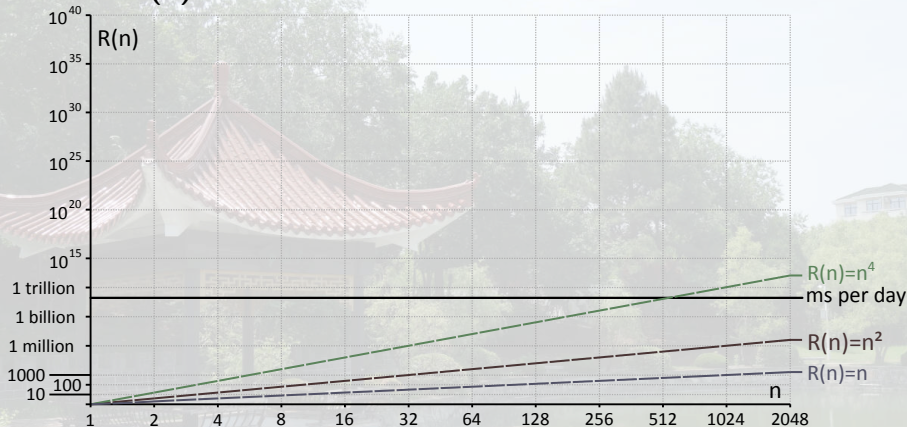
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



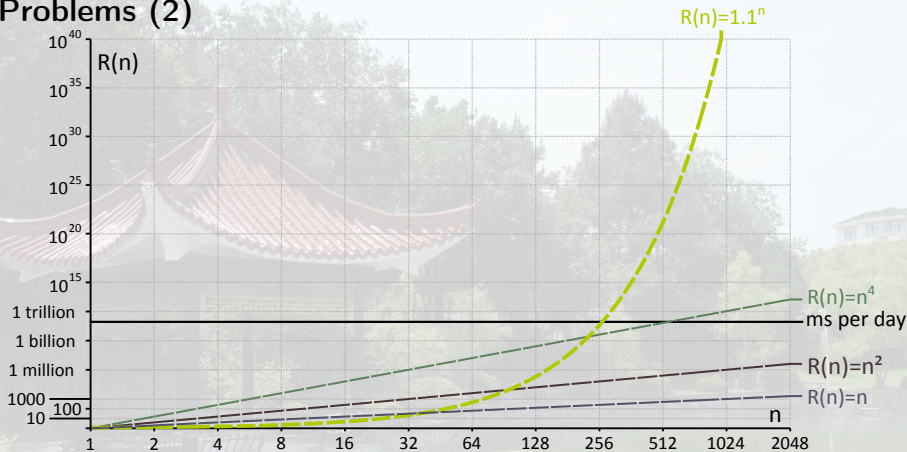
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



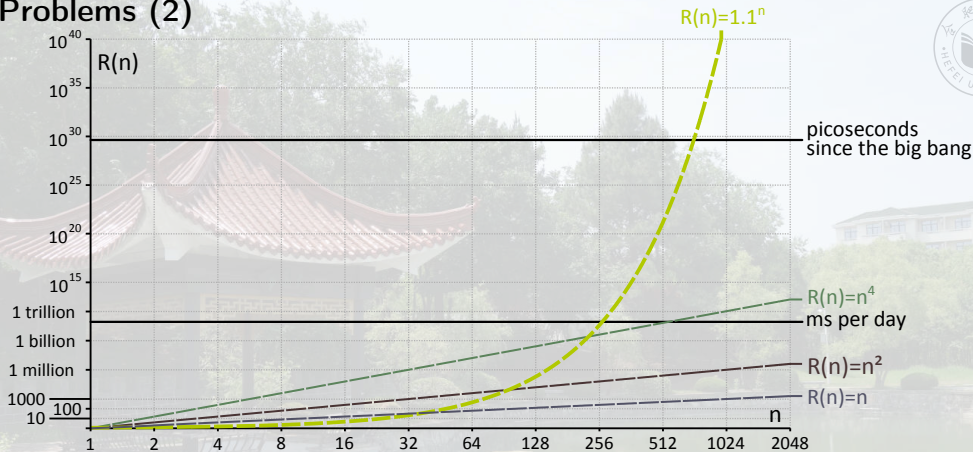
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)

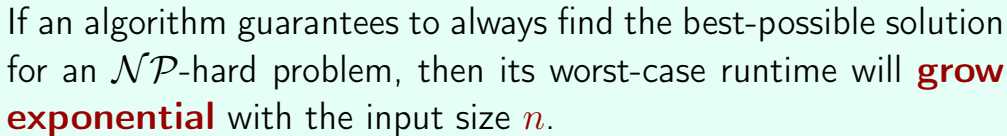


If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

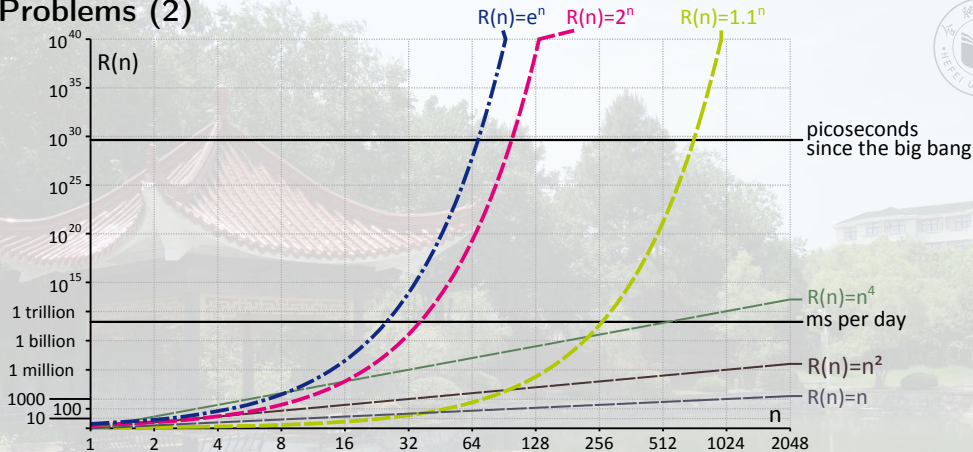
Hard Problems (2)



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

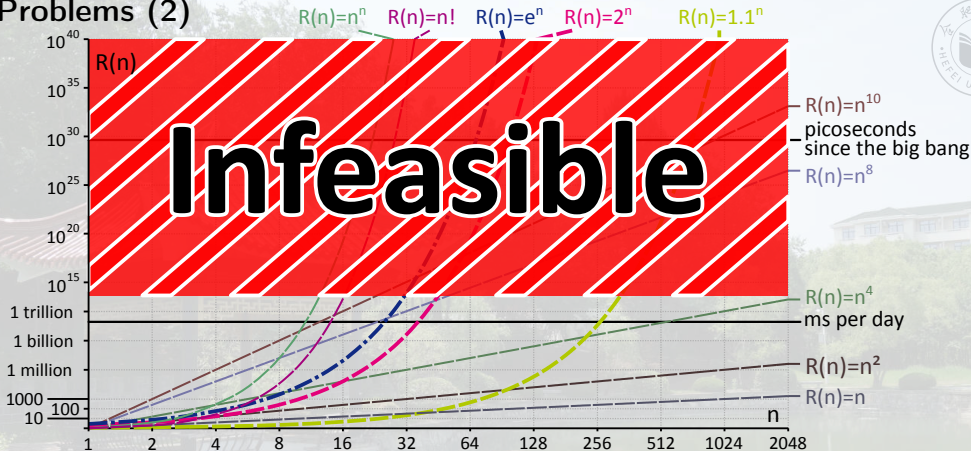


Hard Problems (2)



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

**If we want to get the optimal solution,
it will take too long.**

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

If we want to get the optimal solution,
it will **sometimes/often/always** take too long.

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its **worst-case** runtime will grow exponential with the input size n .

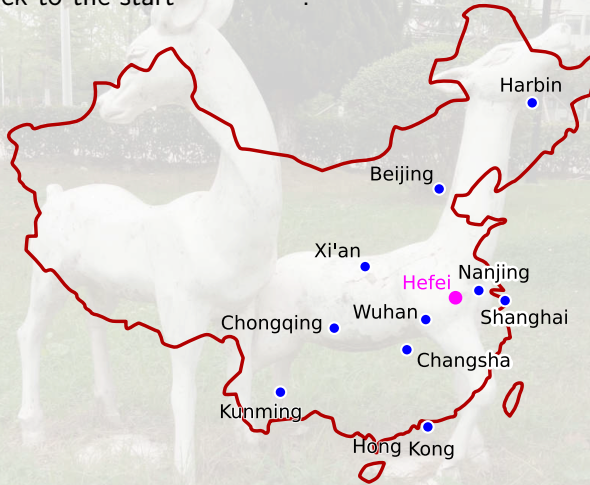
Traveling Salesperson Problem

- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start ^{2,39,55,57,91,100}.



Traveling Salesperson Problem

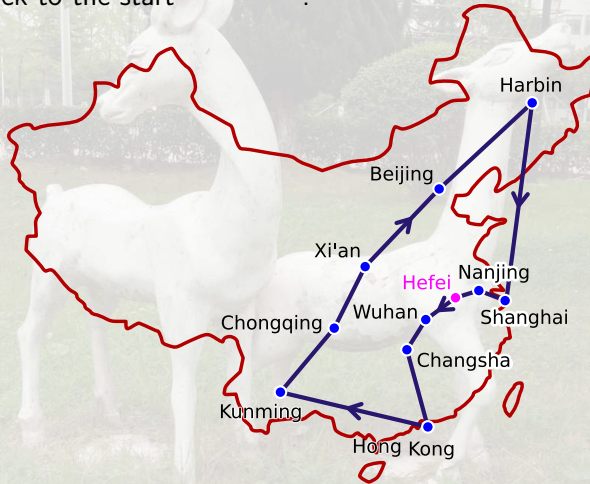
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



Traveling Salesperson Problem



- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.

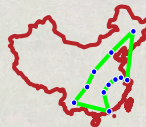


Traveling Salesperson Problem



- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start ^{2,39,55,57,91,100}.

getting the optimal solution
for a TSP

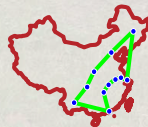


Traveling Salesperson Problem



- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start ^{2,39,55,57,91,100}.

getting the optimal solution
for a TSP may take too long



consumed runtime:

very much / too (?) long

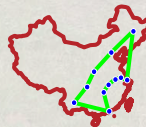
Traveling Salesperson Problem



- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



getting the optimal solution
for a TSP may take too long



very little / fast

consumed runtime

very much / too (?) long

Traveling Salesperson Problem



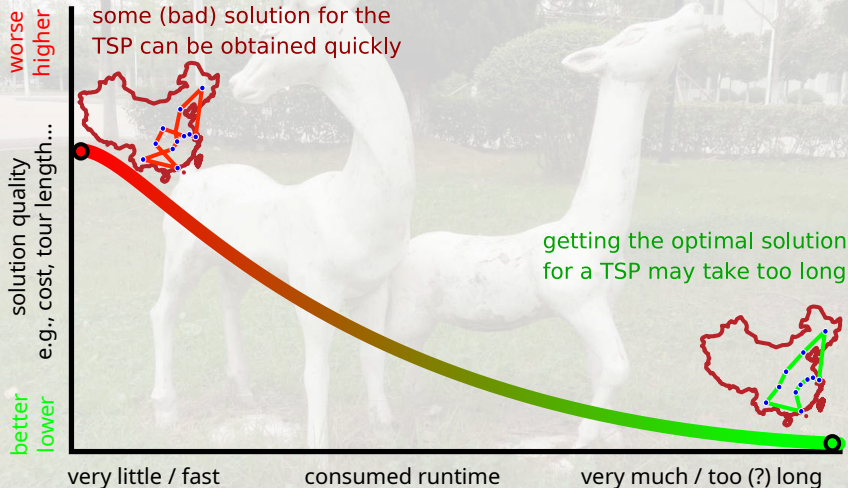
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



Traveling Salesperson Problem



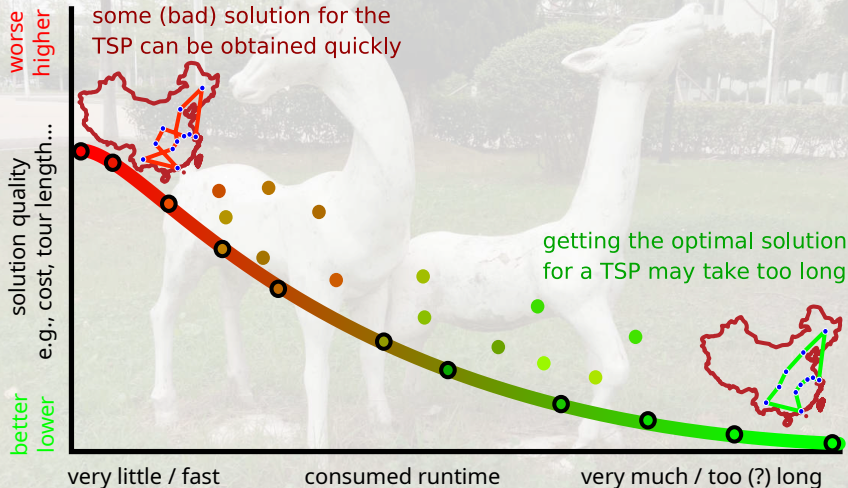
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



Traveling Salesperson Problem



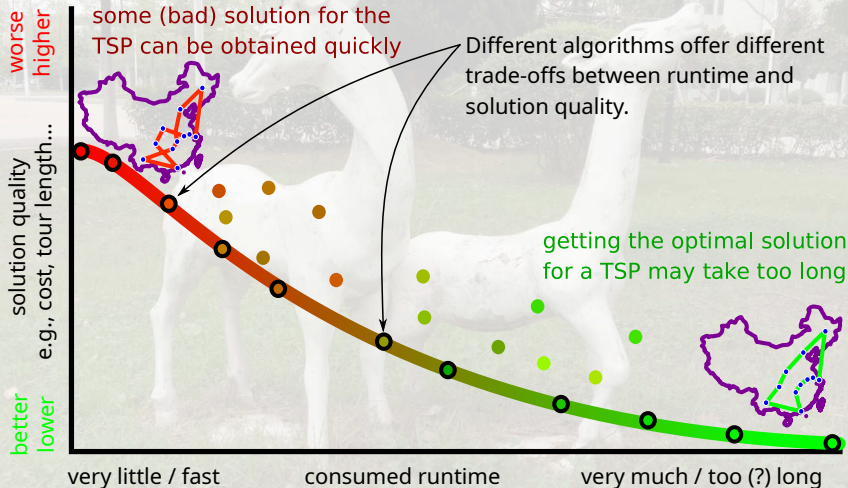
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



Traveling Salesperson Problem



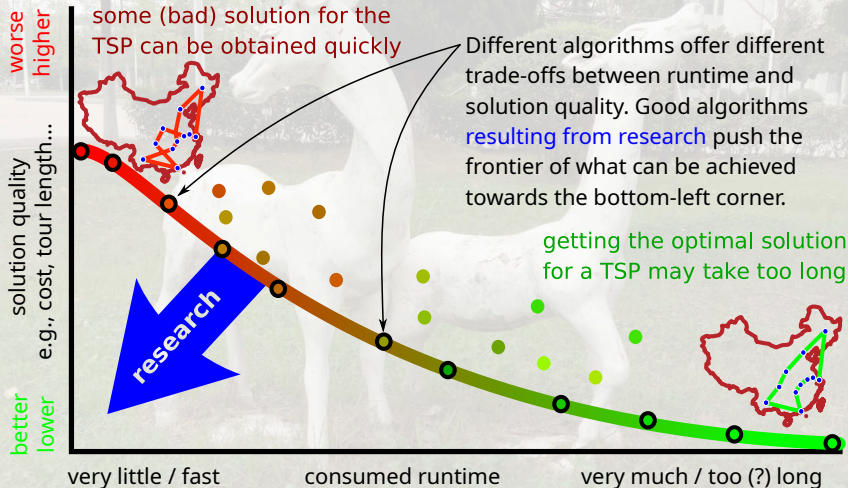
- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.



Traveling Salesperson Problem

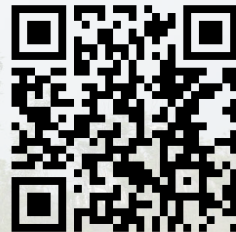


- In an Traveling Salesperson Problem (TSP), we want to find the shortest path through n locations and back to the start^{2,39,55,57,91,100}.





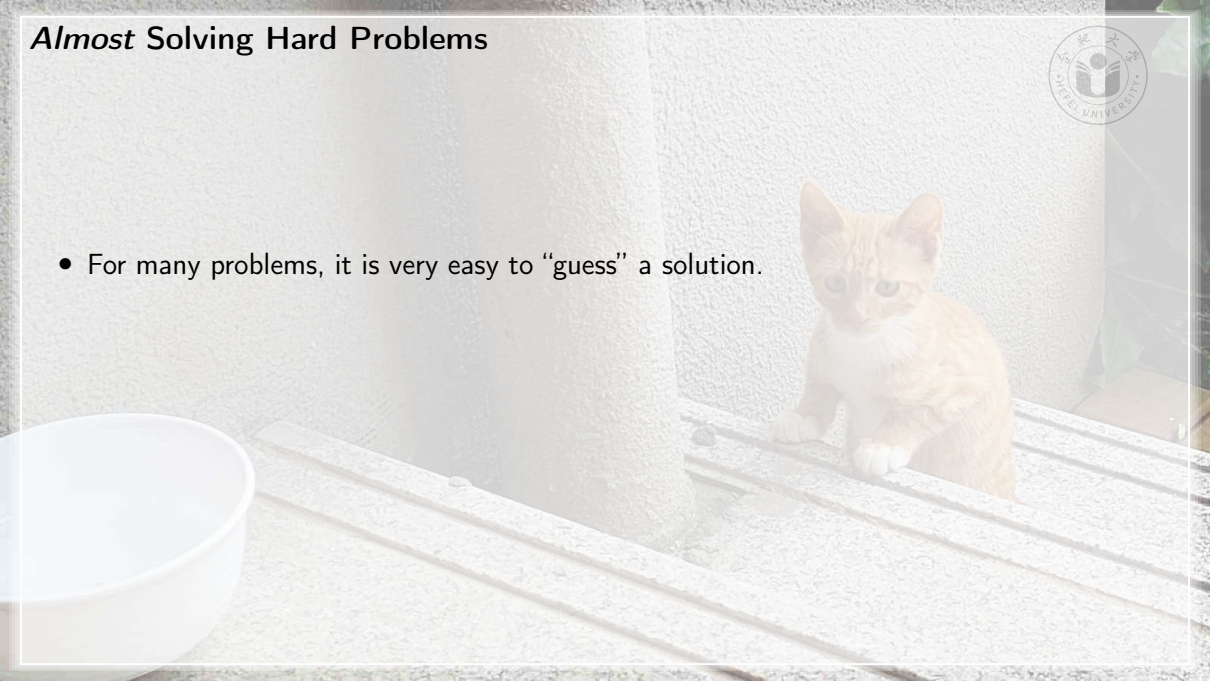
Randomly Guessing Solutions



Almost Solving Hard Problems



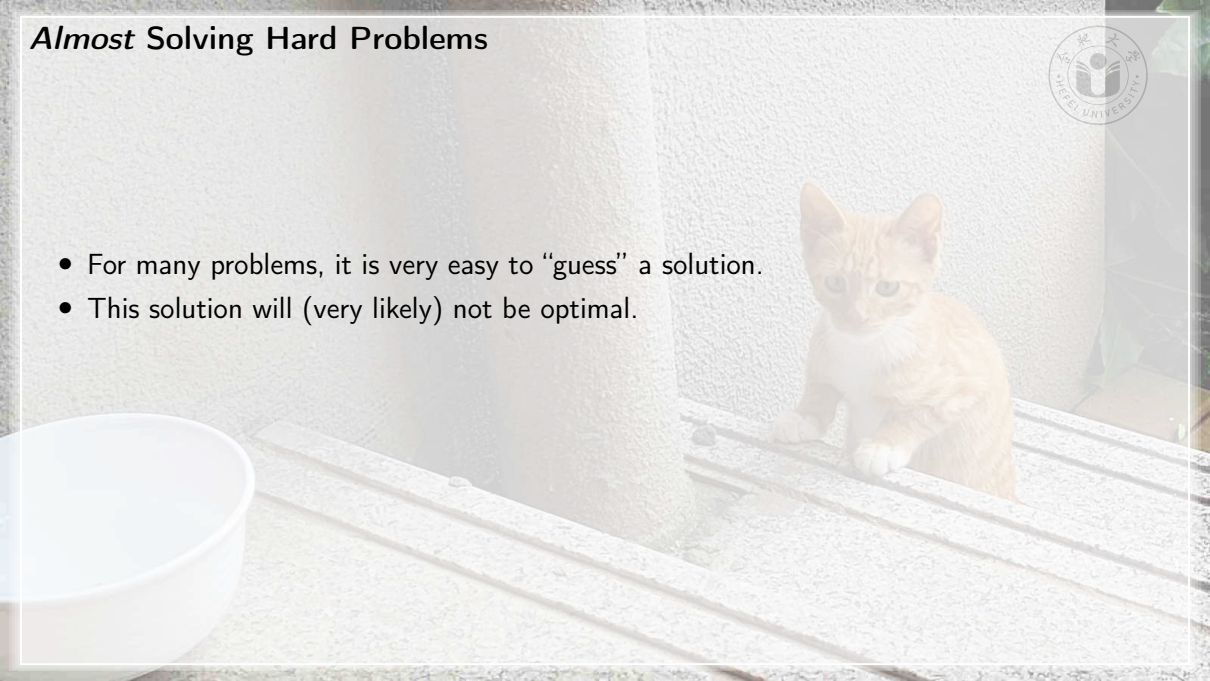
- For many problems, it is very easy to “guess” a solution.



Almost Solving Hard Problems



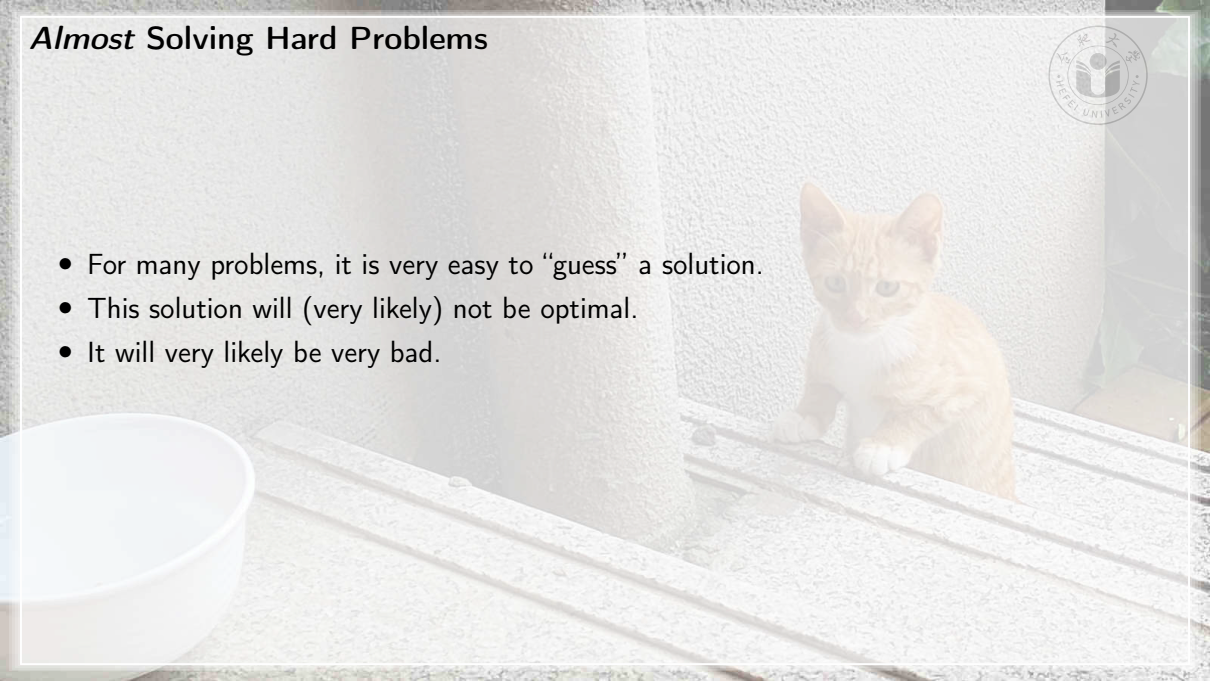
- For many problems, it is very easy to “guess” a solution.
- This solution will (very likely) not be optimal.



Almost Solving Hard Problems



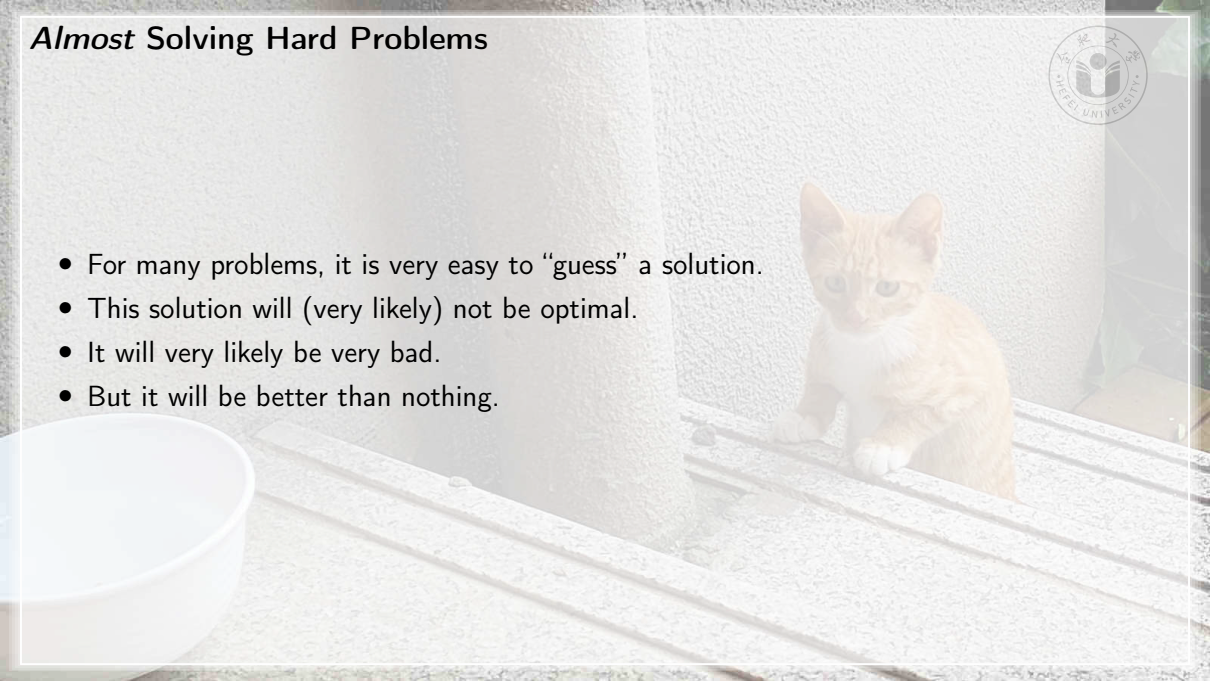
- For many problems, it is very easy to “guess” a solution.
- This solution will (very likely) not be optimal.
- It will very likely be very bad.



Almost Solving Hard Problems

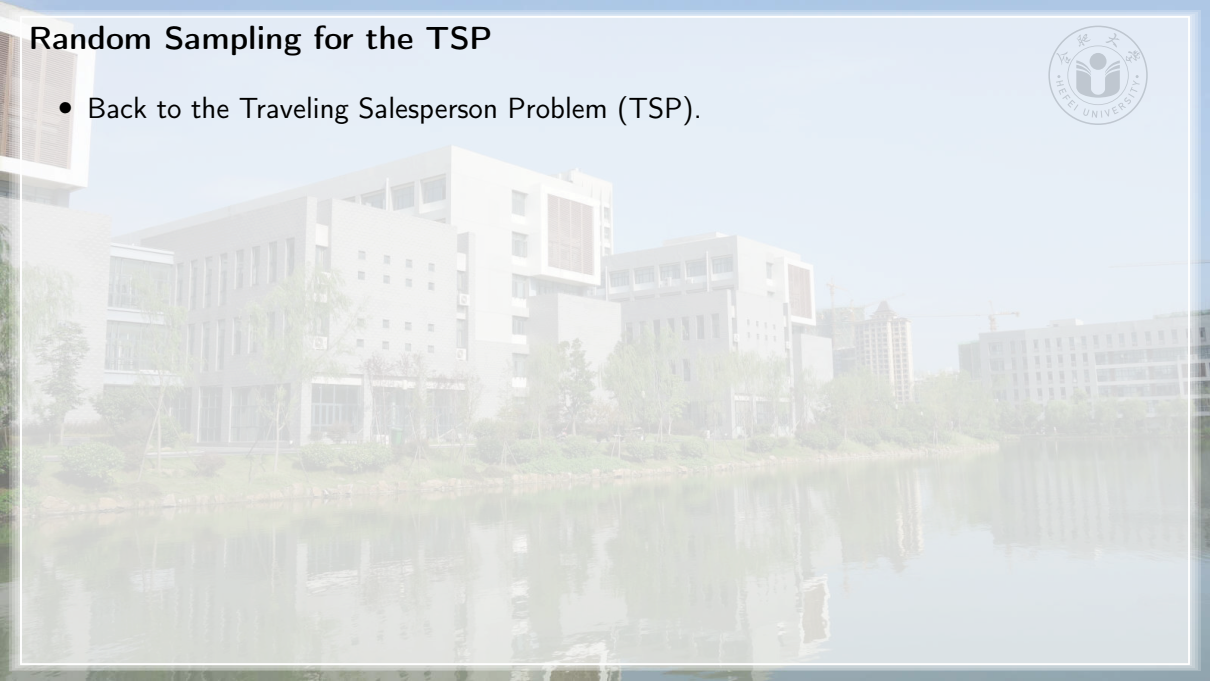


- For many problems, it is very easy to “guess” a solution.
- This solution will (very likely) not be optimal.
- It will very likely be very bad.
- But it will be better than nothing.



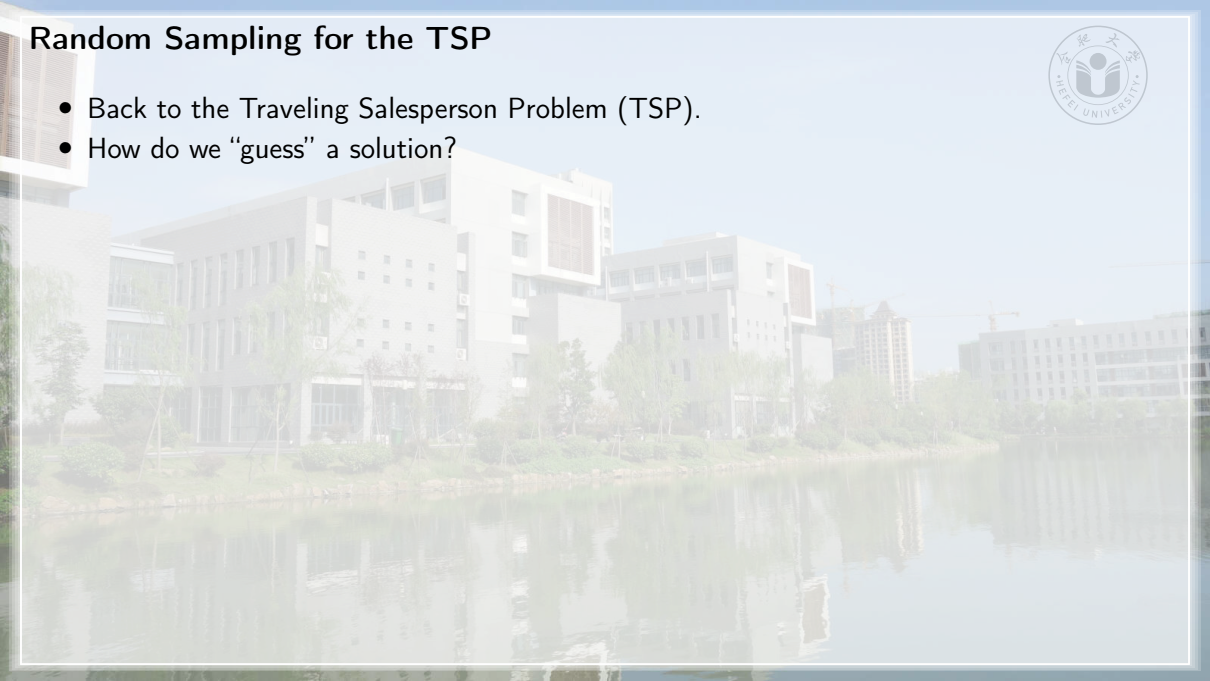
Random Sampling for the TSP

- Back to the Traveling Salesperson Problem (TSP).



Random Sampling for the TSP

- Back to the Traveling Salesperson Problem (TSP).
- How do we “guess” a solution?



Random Sampling for the TSP

- Back to the Traveling Salesperson Problem (TSP).
- How do we “guess” a solution?
- We could write down the cities in a random order.



Random Sampling for the TSP

- We could write down the cities in a random order.



Random Sampling for the TSP

- We could write down the cities in a random order.



Random Sampling for the TSP

- We could write down the cities in a random order.



Random Sampling for the TSP

- We could write down the cities in a random order.



Random Sampling for the TSP

- We could write down the cities in a random order.



Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.



Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.



Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.



Random Sampling for the TSP



- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.
- For this small problem, my computer can do this *millions* of times in a few seconds.

Random Sampling for the TSP

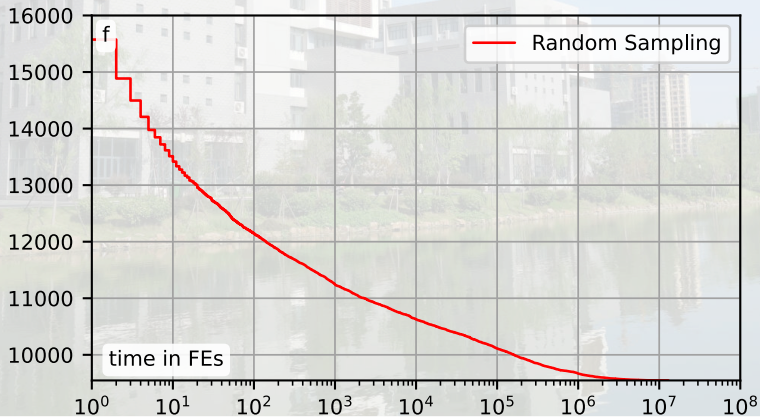


- We could write down the cities in a random order.
- If we do this, we will get a bad result.
- But we will get it quickly.
- And we can do this often.
- For this small problem, my computer can do this *millions* of times in a few seconds.
- We can let the algorithm run for as much time as we can wait ... and then simply take the best random tour we found.

Random Sampling for the TSP



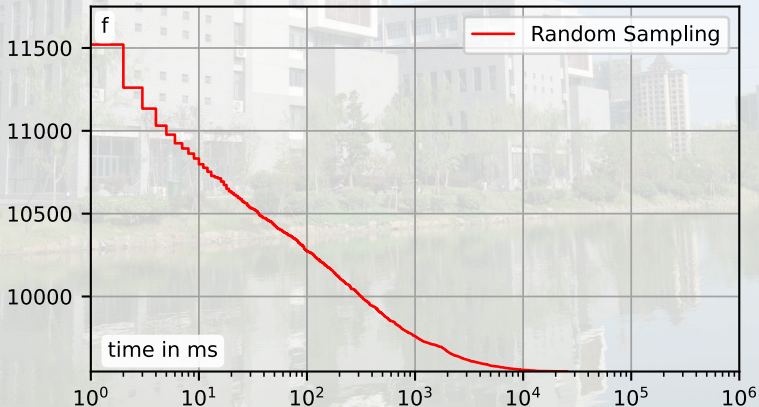
- We could write down the cities in a random order.
- This is the improvement of the best-so-far solution of this so-called Random Sampling algorithm over the number of solutions it has generated and tested (the so-called objective function evaluations (FEs)).



Random Sampling for the TSP



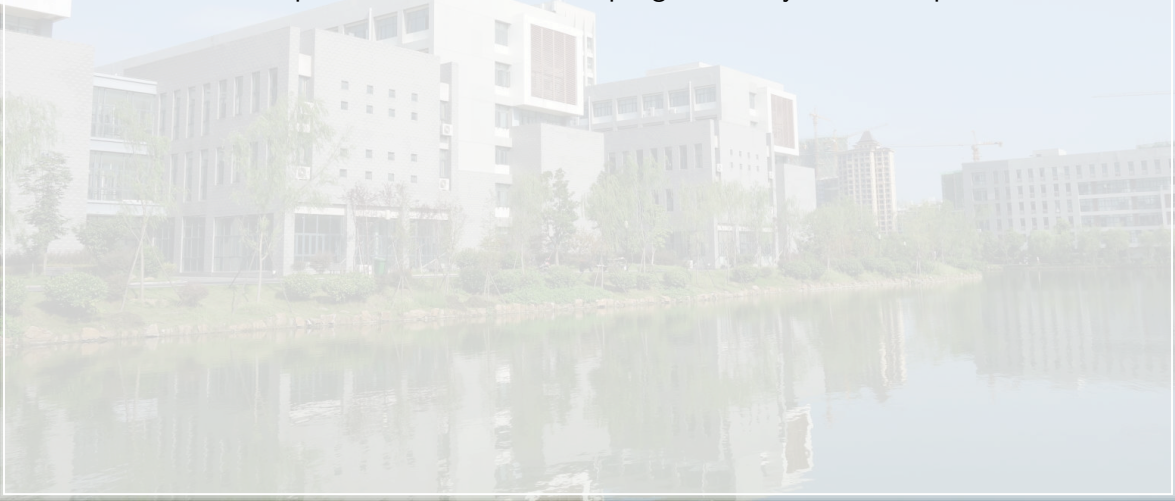
- We could write down the cities in a random order.
- This is the improvement of the best-so-far solution of this so-called Random Sampling algorithm over the consumed runtime in milliseconds.



Random Sampling for the TSP



- We could write down the cities in a random order.
- And for this small problem, this random sampling eventually finds the optimal tour.





Principle 1: We can randomly construct solutions.



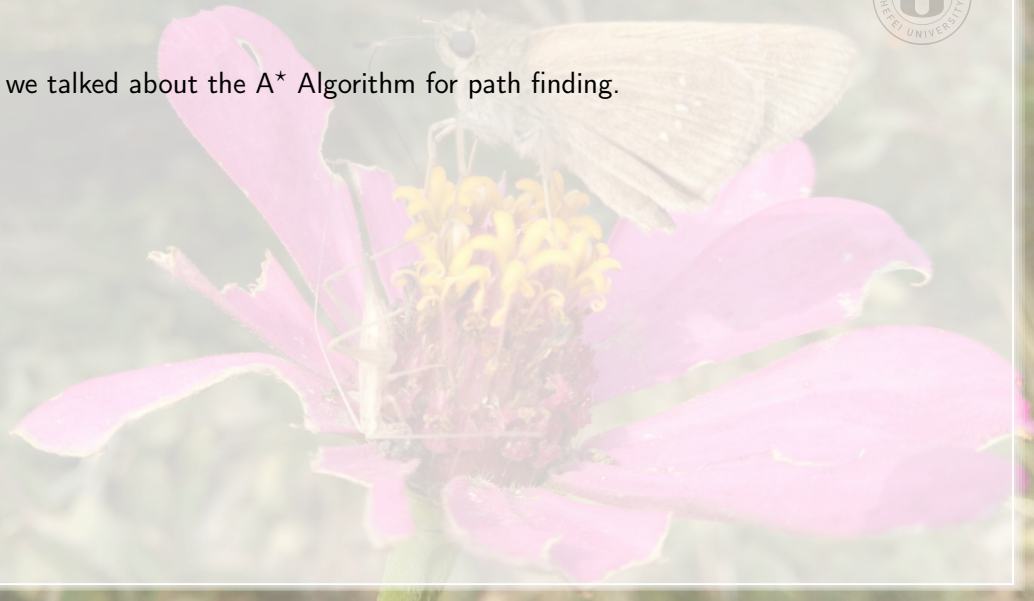
Principle 1: We can randomly construct solutions.

Problem 1: Guessing an optimal (or even just good) solution randomly is very unlikely.

Why don't we use the A^* Algorithm for the TSP?



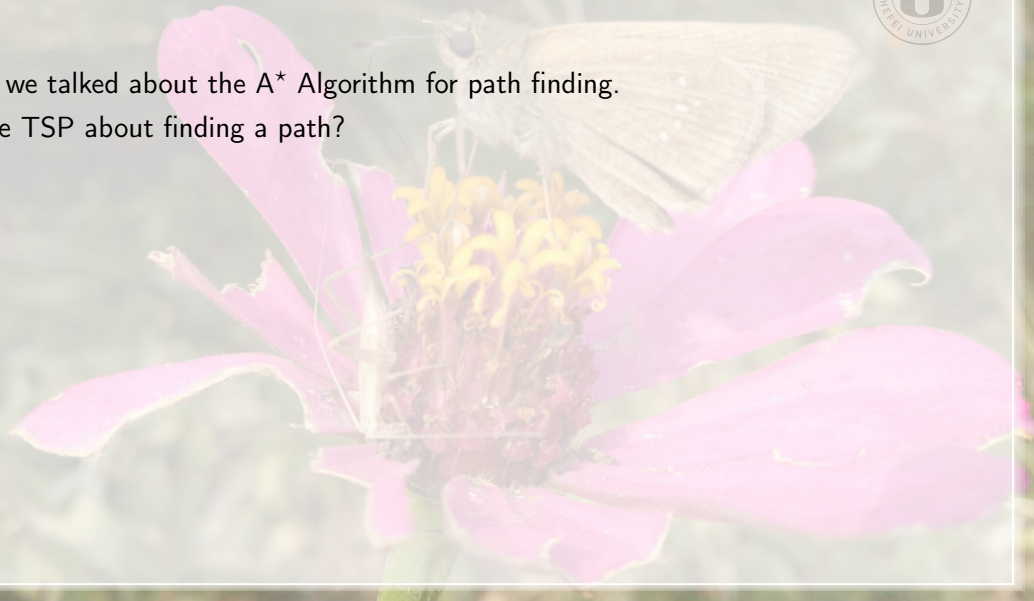
- Before, we talked about the A^* Algorithm for path finding.



Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?



Why don't we use the A^* Algorithm for the TSP?



- Before, we talked about the A^* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴.

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?

1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
 - Isn't the TSP about finding a path?
 - Why do we not use it for the TSP?
-
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...
 2. Also, it is a bit awkward: What are the heuristic and the goal state?

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
- Isn't the TSP about finding a path?
- Why do we not use it for the TSP?
- It will crash for any larger TSP.

1. The A* Algorithm needs **memory exponential** in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...
2. Also, it is a bit awkward: What are the heuristic and the goal state?

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
 - Isn't the TSP about finding a path?
 - Why do we not use it for the TSP?
 - It will crash for any larger TSP.
 - But it can be done and used for our small TSP.
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...
 2. Also, it is a bit awkward: What are the heuristic and the goal state?

Why don't we use the A* Algorithm for the TSP?



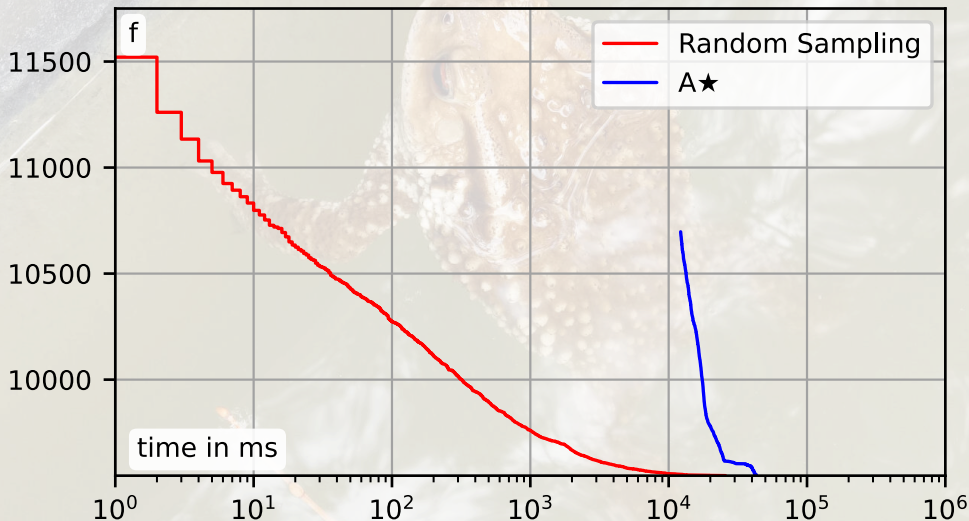
- Before, we talked about the A* Algorithm for path finding.
 - Isn't the TSP about finding a path?
 - Why do we not use it for the TSP?
 - It will crash for any larger TSP.
 - But it can be done and used for our small TSP.
 - So let's do it.
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...
 2. Also, it is a bit awkward: What are the heuristic and the goal state?

Why don't we use the A* Algorithm for the TSP?



- Before, we talked about the A* Algorithm for path finding.
 - Isn't the TSP about finding a path?
 - Why do we not use it for the TSP?
 - It will crash for any larger TSP.
 - But it can be done and used for our small TSP.
 - So let's do it. (I spare you all details.)
1. The A* Algorithm needs memory exponential in the number of nodes in the final solution⁴⁴. Which is the number of cities in the TSP...
 2. Also, it is a bit awkward: What are the heuristic and the goal state?

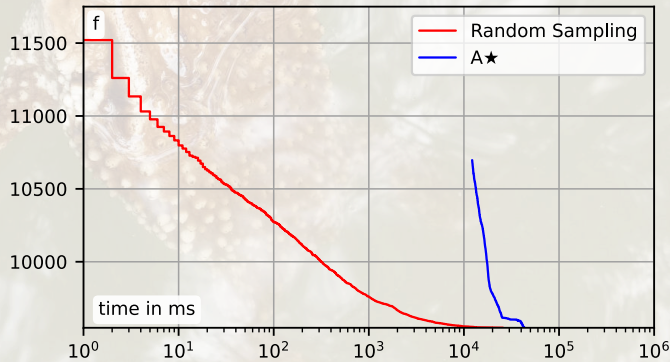
Why don't we use the A* Algorithm for the TSP?



Why don't we use the A* Algorithm for the TSP?



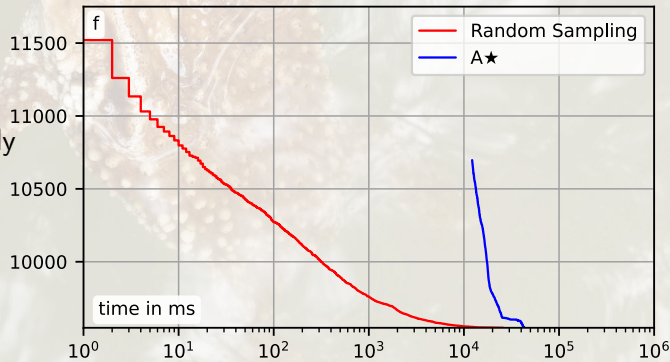
- Because not only will it use too much memory for larger instances...



Why don't we use the A* Algorithm for the TSP?



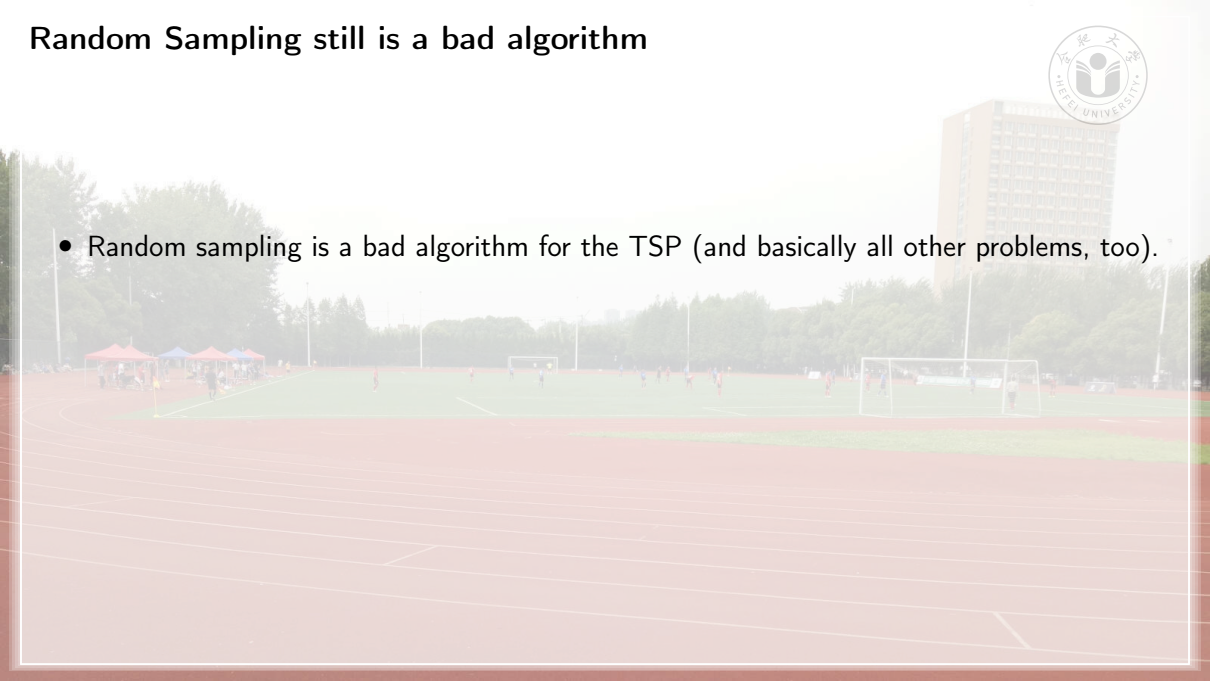
- Because not only will it use too much memory for larger instances...
- ...it is also slower than randomly guessing on smaller instances.



Random Sampling still is a bad algorithm



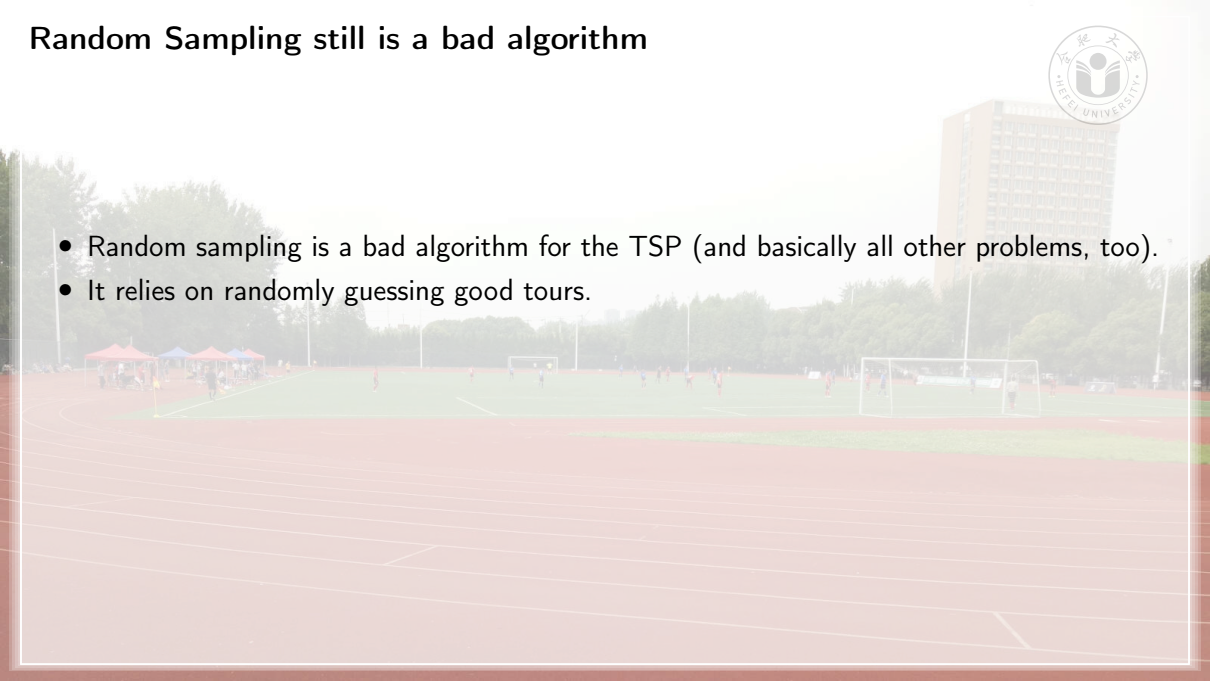
- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).



Random Sampling still is a bad algorithm



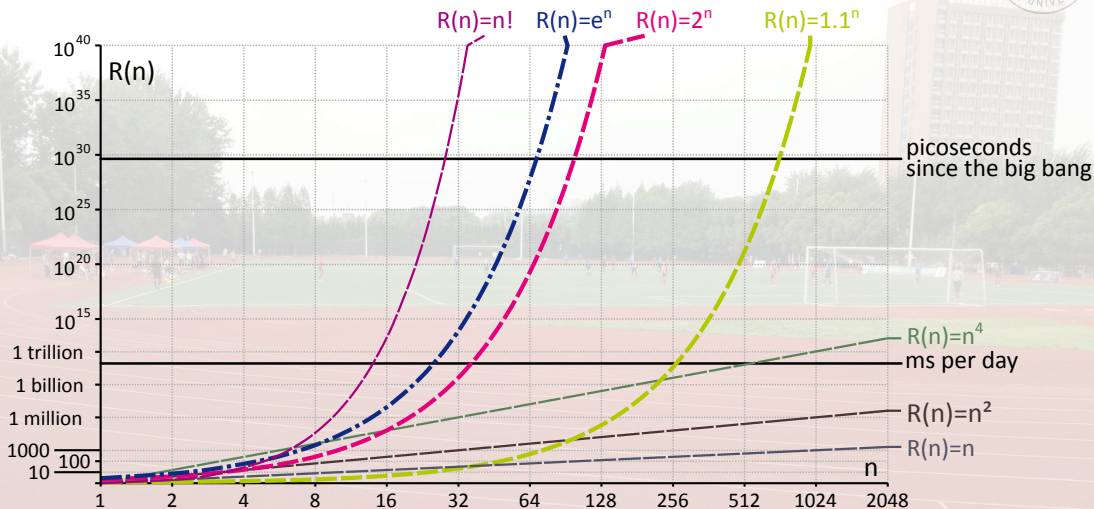
- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.



Random Sampling still is a bad algorithm



- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have n cities, then there are $1 * 2 * 3 * \dots * (n - 1) * n = n!$ tours.



Random Sampling still is a bad algorithm



- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have n cities, then there are $1 * 2 * 3 * \dots * (n - 1) * n = n!$ tours.
- Factorial growth ($n!$) is even worse than exponential growth...

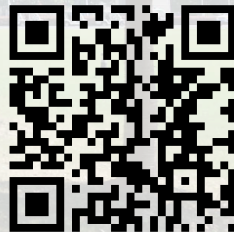
Random Sampling still is a bad algorithm



- Random sampling is a bad algorithm for the TSP (and basically all other problems, too).
- It relies on randomly guessing good tours.
- If we have n cities, then there are $1 * 2 * 3 * \dots * (n - 1) * n = n!$ tours.
- Factorial growth ($n!$) is even worse than exponential growth...
- So if we try to randomly sample the best possible tour, our chance is extremely small...



Local Search: Using Information



Random Sampling is a bad algorithm



- In each step, random sampling creates a completely random tour.

Random Sampling is a bad algorithm



- In each step, random sampling creates a completely random tour.
- It does not gather any information.

Random Sampling is a bad algorithm



- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.

Random Sampling is a bad algorithm



- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.
- Just guessing answers randomly is not a good method.

Random Sampling is a bad algorithm



- In each step, random sampling creates a completely random tour.
- It does not gather any information.
- It keeps the best tour, but does not use any information inside this tour.
- Just guessing answers randomly is not a good method.
- Clearly, seeing millions of tours, we should be able learn *something* and somehow use that to find better tours???

Randomized Local Search



- But what could we do?



Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”



Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.



Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.
- If we have a tour x , we could randomly pick two cities in the tour and swap them.



Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.
- If we have a tour x , we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.

Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.
- If we have a tour x , we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.

Randomized Local Search



- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.
- If we have a tour x , we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.
- Then we do the same with a better tour that we find, and with yet a better tour, and so on.

Randomized Local Search



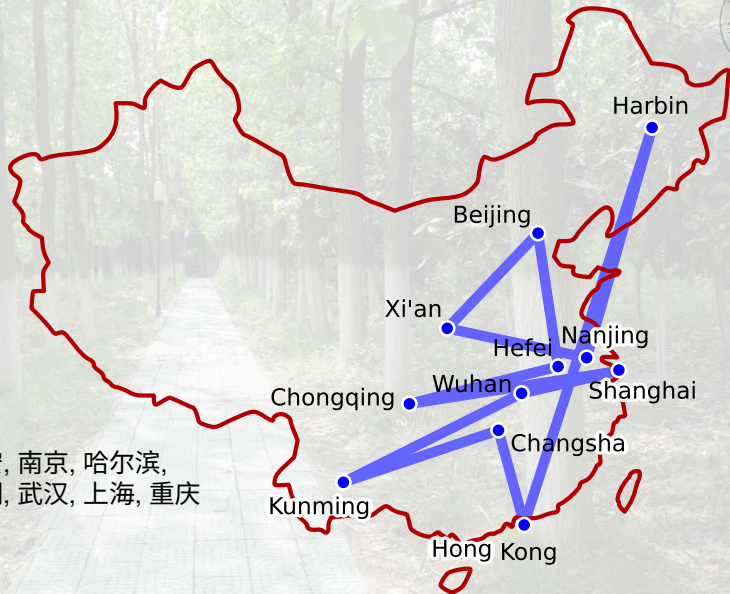
- But what could we do?
- “In the neighborhood of a solution, there will probably be better or worse solutions.”
- neighborhood = solutions that are similar.
- If we have a tour x , we could randomly pick two cities in the tour and swap them.
- If we do this, maybe we could get a better tour or a worse tour.
- We could keep a better tour, but throw away a worse one.
- Then we do the same with a better tour that we find, and with yet a better tour, and so on.
- This method is called randomized local search (RLS).

Randomized Local Search for the TSP



15983 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆



Randomized Local Search for the TSP



16080 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

合肥, 哈尔滨, 西安, 南京, 北京,
香港, 长沙, 昆明, 武汉, 上海, 重庆



Randomized Local Search for the TSP

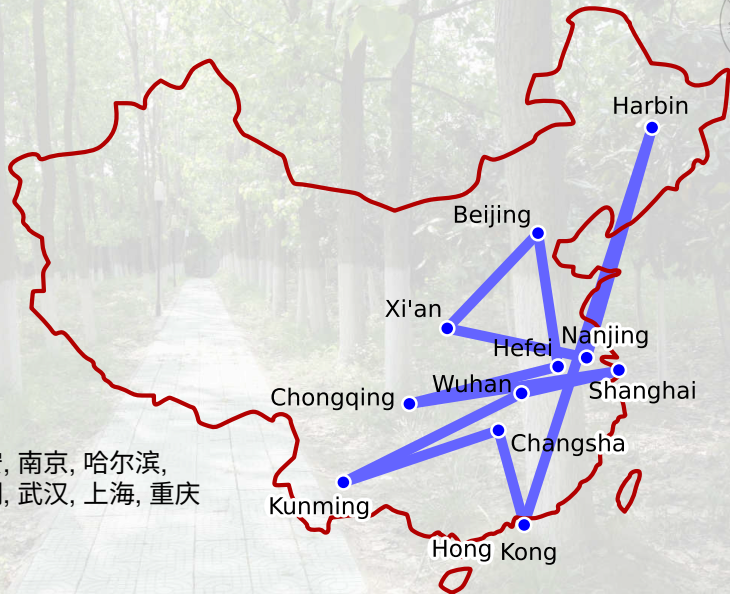


Randomized Local Search for the TSP



15983 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆



Randomized Local Search for the TSP



15128 km

合肥, 北京, 西安, 南京, 哈尔滨,
香港, 长沙, 昆明, 武汉, 上海, 重庆

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆



Randomized Local Search for the TSP



15128 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆



Randomized Local Search for the TSP



16435 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

合肥, 北京, 西安, 南京, 香港,
上海, 长沙, 昆明, 武汉, 哈尔滨, 重庆



Randomized Local Search for the TSP



Randomized Local Search for the TSP



15128 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆



Randomized Local Search for the TSP



12432 km

合肥, 北京, 西安, 南京, 上海,
香港, 长沙, 昆明, 武汉, 哈尔滨, 重庆

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆



Randomized Local Search for the TSP



12432 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆



Randomized Local Search for the TSP



14036 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 武汉, 上海,
香港, 长沙, 昆明, 南京, 西安, 重庆

Randomized Local Search for the TSP



14036 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 武汉, 上海,
香港, 长沙, 昆明, 南京, 西安, 重庆



Randomized Local Search for the TSP



12432 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆



Randomized Local Search for the TSP



10857 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 武汉, 西安, 重庆

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉

Randomized Local Search for the TSP



10857 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉



Randomized Local Search for the TSP



10620 km

合肥, 北京, 哈尔滨, 南京, 上海,
香港, 长沙, 昆明, 重庆, 西安, 武汉

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉



Randomized Local Search for the TSP



10620 km

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉



Randomized Local Search for the TSP



10567 km

合肥, 北京, 哈尔滨, 南京, 上海,
长沙, 香港, 昆明, 重庆, 西安, 武汉

合肥, 北京, 哈尔滨, 上海, 南京,
长沙, 香港, 昆明, 重庆, 西安, 武汉



Randomized Local Search for the TSP

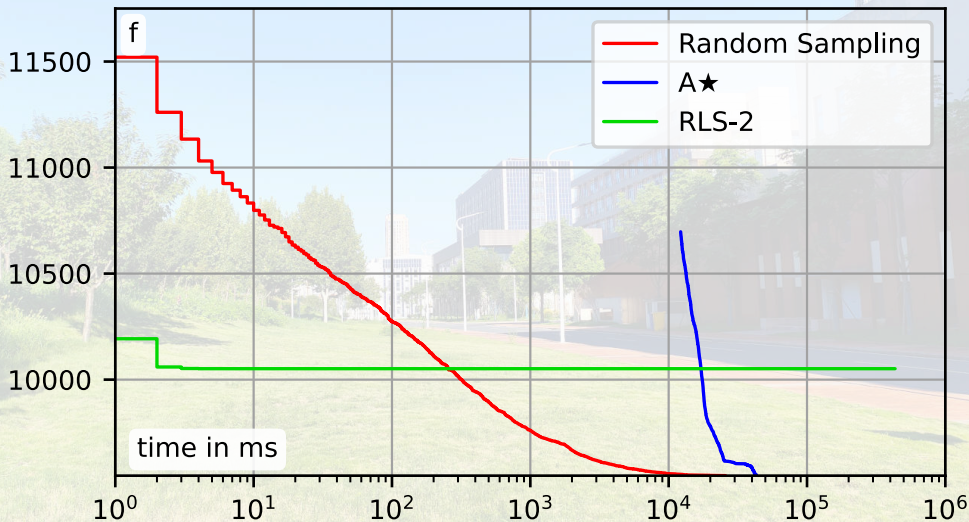


10567 km

合肥, 北京, 哈尔滨, 上海, 南京,
长沙, 香港, 昆明, 重庆, 西安, 武汉



Randomized Local Search for the TSP



Randomized Local Search for the TSP



- RLS is much faster than random sampling.

Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However...

Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However, if our algorithm is at a tour x , then it can only find other tours by swapping two cities.

Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However, if our algorithm is at a tour x , then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.

Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However, if our algorithm is at a tour x , then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.
- If all tours that we can reach in one step are worse than x , we will never leave x .

Randomized Local Search for the TSP



- RLS is much faster than random sampling.
- However, if our algorithm is at a tour x , then it can only find other tours by swapping two cities.
- There are some tours that it cannot reach in one step.
- If all tours that we can reach in one step are worse than x , we will never leave x .
- If x is not the optimum, then we are stuck and won't ever find the optimal/best tour.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.

Problem 2: Sometimes, a small change is not enough.



Principle 2: Applying a small random change to an existing solution sometimes can give us a better solution. This way, we can step-by-step get better solutions.

Problem 2: Sometimes, a small change is not enough. We can get stuck at a so-called *local* optimum.

Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .

Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .
- There are many things that we can do.

Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .
- There are many things that we can do, for example:
 1. We could restart the algorithm at a new starting point after some time.

Randomized Local Search for the TSP



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .
- There are many things that we can do, for example:
 1. We could restart the algorithm at a new starting point after some time.
 2. Or we could sometimes allow it swap more cities in the current tour.

Randomized Local Search with a Larger Neighborhood



15151 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



15195 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

合肥, 重庆, 武汉, 长沙, 北京, 南京,
上海, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



15195 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

合肥, 重庆, 武汉, 长沙, 北京, 南京,
上海, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



15151 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



14539 km

合肥, 重庆, 武汉, 长沙, 北京, 上海,
南京, 昆明, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



14539 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



16624 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

昆明, 南京, 武汉, 长沙, 北京, 上海,
合肥, 重庆, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



16624 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

昆明, 南京, 武汉, 长沙, 北京, 上海,
合肥, 重庆, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



14539 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



13882 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
昆明, 重庆, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



13882 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨



Randomized Local Search with a Larger Neighborhood



10688 km

合肥, 南京, 武汉, 长沙, 北京, 上海,
重庆, 昆明, 香港, 西安, 哈尔滨

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海



Randomized Local Search with a Larger Neighborhood



10688 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海



Randomized Local Search with a Larger Neighborhood



12111 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 哈尔滨, 武汉, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海



Randomized Local Search with a Larger Neighborhood



12111 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 哈尔滨, 武汉, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海



Randomized Local Search with a Larger Neighborhood



9547 km

合肥, 南京, 武汉, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 上海

合肥, 南京, 上海, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 武汉



Randomized Local Search with a Larger Neighborhood

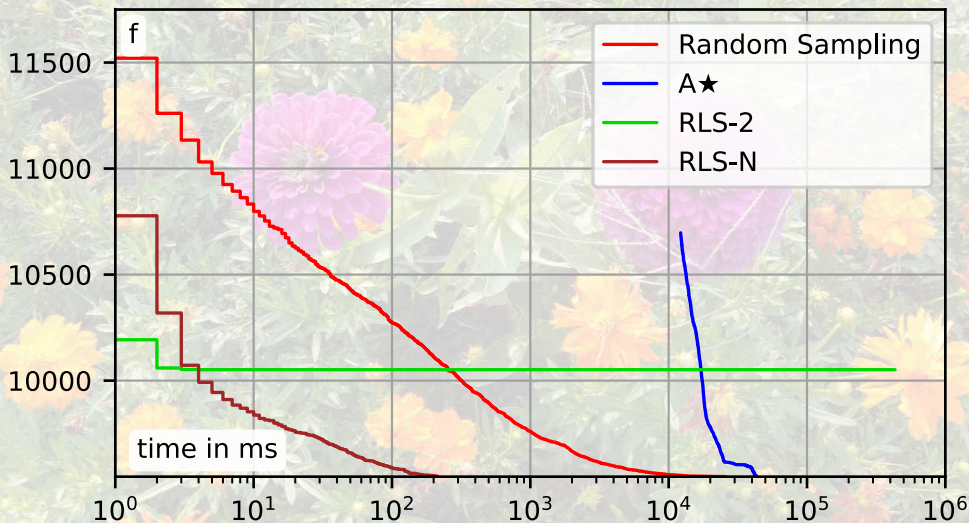


9547 km

合肥, 南京, 上海, 哈尔滨, 北京, 西安,
重庆, 昆明, 香港, 长沙, 武汉



Randomized Local Search with a Larger Neighborhood



Randomized Local Search with a Larger Neighborhood



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .
- There are many things that we can do, for example:
 1. We could restart the algorithm at a new starting point after some time.
 2. Or we could sometimes allow it swap more cities in the current tour.

Randomized Local Search with a Larger Neighborhood



- If all tours that we can reach in one step are worse than the current tour x , then our RLS will never leave x .
- There are many things that we can do, for example:
 1. We could restart the algorithm at a new starting point after some time.
 2. Or we could sometimes allow it swap more cities in the current tour.
- The larger neighborhood of solutions that can be reached in one step makes the search slower, but allows it to escape from local optima.



Principle 3: Larger changes can help us escape from local optima.





Principle 3: Larger changes can help us escape from local optima.

Problem 3: Larger changes are less likely to yield improvements (because the new solution is more different), so they slow down the search.

Metaheuristic Optimization



There are many more principles and problems surrounding (metaheuristic) optimization.



Metaheuristic Optimization

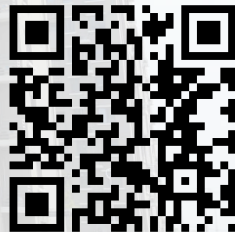


There are many more principles and problems surrounding (metaheuristic) optimization.

But we will leave it at what we have seen so far.

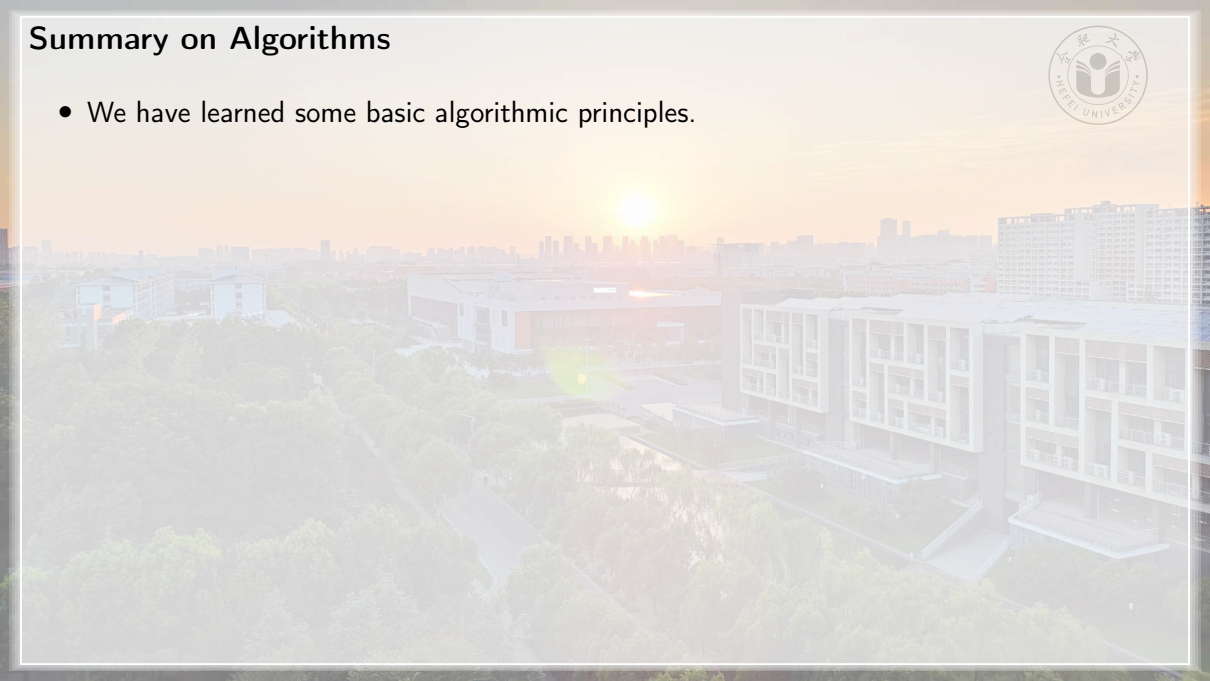


Summary



Summary on Algorithms

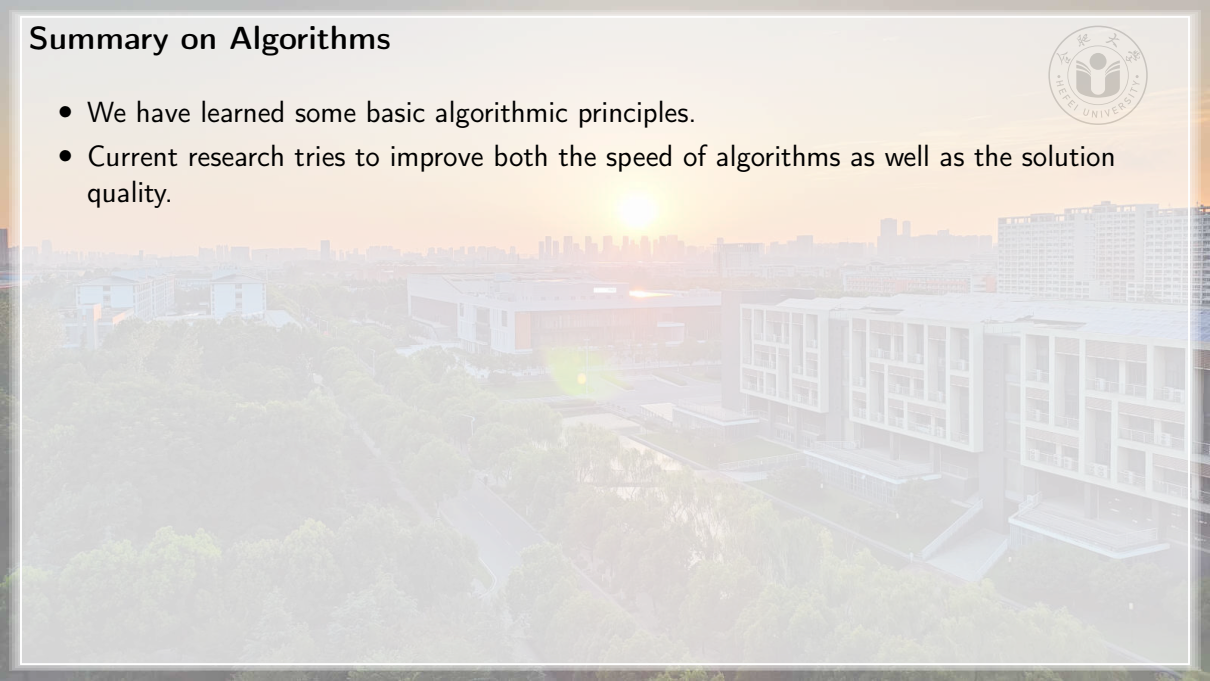
- We have learned some basic algorithmic principles.



Summary on Algorithms



- We have learned some basic algorithmic principles.
- Current research tries to improve both the speed of algorithms as well as the solution quality.



Summary on Algorithms



- We have learned some basic algorithmic principles.
- Current research tries to improve both the speed of algorithms as well as the solution quality.
- This requires carefully balancing the step-size of algorithms and to develop methods against getting stuck at local optima.

Summary on the TSP

- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}.



Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}.
- For this, we actually use principles similar to the A^* Algorithm, just a bit differently, in methods^{2,21} like branch and bound (B&B)⁶¹.

Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}.
- For this, we actually use principles similar to the A^* Algorithm, just a bit differently, in methods^{2,21} like branch and bound (B&B)⁶¹.
- We can get close-to-optimal solutions for TSPs with millions of cities.

Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}.
- For this, we actually use principles similar to the A^* Algorithm, just a bit differently, in methods^{2,21} like branch and bound (B&B)⁶¹.
- We can get close-to-optimal solutions for TSPs with millions of cities.
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm^{41,42}.

Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}. [but not all of them!]
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods^{2,21} like branch and bound (B&B)⁶¹.
- We can get close-to-optimal solutions for TSPs with millions of cities.
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm^{41,42}.

Summary on the TSP



- Today, we can actually solve TSPs with tens of thousands of cities to optimality^{2,20}. [but not all of them!]
- For this, we actually use principles similar to the A* Algorithm, just a bit differently, in methods^{2,21} like branch and bound (B&B)⁶¹.
- We can get close-to-optimal solutions for TSPs with millions of cities. [close-to-optimal, but not optimal (at least not always)!]
- For this, we use algorithms a bit similar to randomized local search (RLS), but with more targeted search steps, e.g., in the Lin-Kernighan Helsgaun (LKH) algorithm^{41,42}.

Summary



- Today, we discussed what optimization is.

Summary



- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.

Summary



- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.

Summary



- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.
- For example: As long as we can randomly construct and randomly modify a solution, we can attack the problem with randomized local search (RLS).

Summary



- Today, we discussed what optimization is.
- Today, you also saw some of the basic principles and methods that are inside of the algorithms used in optimization and operations research.
- You can imagine that the same principles that we tested on the TSP will work on many different kinds of problems.
- For example: As long as we can randomly construct and randomly modify a solution, we can attack the problem with randomized local search (RLS).
- Understanding the basic principles of optimization is not very hard.



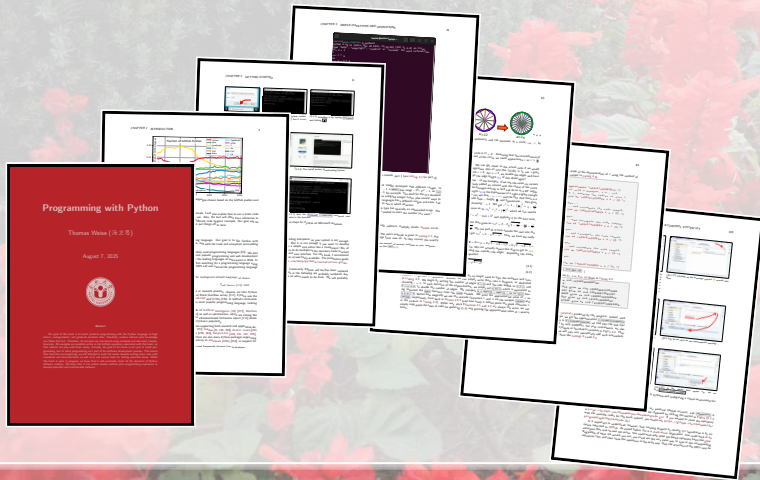
Advertisement



Programming with Python



We have a freely available course book on *Programming with Python* at <https://thomasweise.github.io/programmingWithPython>, with focus on practical software development using the Python ecosystem of tools⁹⁰.



Databases



We have a freely available course book on *Databases* at <https://thomasweise.github.io/databases>, with actual practical examples using a real database management system (DBMS)⁸⁷.



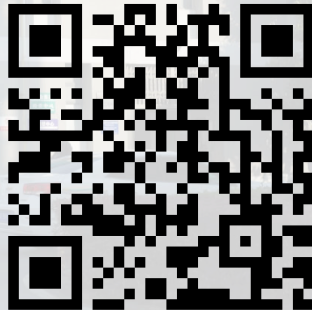
Metaheuristic Optimization in Python: moptipy



We offer moptipy⁹⁷, a mature open source Python package for metaheuristic optimization, which implements several algorithms, can run self-documenting experiments in parallel and in a distributed fashion, and offers statistical evaluation tools.

The collage displays various parts of the moptipy project:

- GitHub Repository:** Shows the repository structure, including the README, CONTRIBUTING, and LICENSE files.
- Documentation:** Shows the 'Quick search' bar, the 'Table of Contents', and the '1. Introduction' section, which describes the library's purpose and usage.
- Algorithms:** Shows the '4. Implemented Algorithms' section, listing various optimization algorithms available in the package.
- Progress Plots:** A line graph showing the progress of different algorithms over time, with the x-axis representing time and the y-axis representing the objective value.





谢谢你们！
Thank you!
Vielen Dank!



References I



- [1] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on p. 325).
- [2] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde TSP solver. (Cit. on pp. 136, 160–170, 290–295, 322, 327).
- [3] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: <http://gallica.bnf.fr/ark:/12148/bpt6k994750> (visited on 2023-12-13) (cit. on p. 329).
- [4] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on p. 322).
- [5] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel López-Ibáñez, Katherine Mary Malan, Jason Hall Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise (汤卫恩). "Benchmarking in Optimization: Best Practice and Open Issues". (abs/2007.03488), Dec. 18, 2020. doi:10.48550/arXiv.2007.03488. URL: <https://arxiv.org/abs/2007.03488> (visited on 2025-07-25). arXiv:2007.03488v2 [cs.NE] 16 Dec 2020 (cit. on p. 322).
- [6] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (visited on 2025-02-05) (cit. on p. 328).
- [7] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: <https://www.ietf.org/rfc/rfc1738.txt> (visited on 2025-02-05) (cit. on p. 328).

References II



- [8] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 323).
- [9] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 323).
- [10] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 326).
- [11] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: <https://arxiv.org/abs/1812.00493> (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on p. 321).
- [12] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 326).
- [13] Nouredine Chabini and Rachid Beguenane. "FPGA-Based Designs of the Factorial Function". In: *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE'2022)*. Sept. 18–20, 2022, Halifax, NS, Canada. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022, pp. 16–20. ISBN: 978-1-6654-8432-9. doi:10.1109/CCECE49351.2022.9918302 (cit. on p. 328).
- [14] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Cit. on pp. 323, 329).

References III



- [15] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫恩). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 323, 325, 326).
- [16] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: 978-0-471-94059-3 (cit. on p. 323).
- [17] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 324, 329).
- [18] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: <https://math.uwaterloo.ca/tsp/concorde> (visited on 2026-09-26) (cit. on p. 322).
- [19] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: 978-0-691-16352-9 (cit. on pp. 322, 324).
- [20] William John Cook. *World TSP*. Waterloo, ON, Canada: University of Waterloo, 2013–Oct. 25, 2025. URL: <http://www.math.uwaterloo.ca/tsp/world> (visited on 2026-01-03) (cit. on pp. 290–295, 324).
- [21] William John Cook, Daniel G. Espinoza, and Marcos Goycoolea. *Computing with Domino-Parity Inequalities for the TSP*. Tech. rep. Atlanta, GA, USA: Georgia Institute of Technology Industrial and Systems Engineering, 2005. URL: https://www.math.uwaterloo.ca/~bico/papers/DP_paper.pdf (visited on 2026-01-03). See also ²¹. (Cit. on pp. 290–295, 308).
- [22] *Definition of Operations Research*. University of Western Ontario, London, ON, Canada: International Federation of Operational Research Societies (IFORS), 2020. URL: <https://www.ifors.org/what-is-or> (visited on 2026-01-01) (cit. on p. 325).

References IV



- [23] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: 978-0-13-299045-5 (cit. on p. 328).
- [24] Slobodan Dimitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 322).
- [25] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on p. 325).
- [26] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 321, 323, 324, 327).
- [27] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-Z (cit. on p. 325).
- [28] Jacques Dutka. "The Early History of the Factorial Function". *Archive for History of Exact Sciences* 43(3):225–249, Sept. 1991. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany. ISSN: 0003-9519. doi:10.1007/BF00389433. Communicated by Umberto Bottazzini (cit. on p. 328).
- [29] Harald Dyckhoff and Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (cit. on pp. 8–13).
- [30] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on p. 325).
- [31] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/evco.2009.17.4.17401 (cit. on p. 327).

References V



- [32] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: 978-1-4503-5618-3. doi:10.1145/3205455.3205515 (cit. on pp. 323, 327).
- [33] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: 978-0-7167-1045-5 (cit. on p. 327).
- [34] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.1.3.190 (cit. on p. 327).
- [35] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.2.1.4 (cit. on p. 327).
- [36] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: 0884-8289. ISBN: 978-0-306-48056-0. doi:10.1007/b101874 (cit. on p. 324).
- [37] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: 0254-5330. doi:10.1007/BF02078647. Special issue on Tabu Search (TS). (Cit. on p. 327).
- [38] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (visited on 2025-08-01) (cit. on p. 329).
- [39] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on pp. 160–170, 327).
- [40] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". *IEEE Transactions on Systems Science and Cybernetics* 4:100–107, July 31, 1968. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0536-1567. doi:10.1109/TSSC.1968.300136 (cit. on pp. 122–127, 321).

References VI



- [41] Keld Helsgaun. "General k -opt Submoves for the Lin–Kernighan TSP Heuristic". *Mathematical Programming Computation* 1(2-3):119–163, July 2009. London, England, UK: Springer Nature Limited. ISSN: **1867-2949**. doi:[10.1007/s12532-009-0004-6](https://doi.org/10.1007/s12532-009-0004-6). URL: <https://www2.seas.gwu.edu/~simhaweb/champalg/tsp/papers/Helsgaun2.pdf> (visited on 2026-09-26) (cit. on pp. **290–295, 324**).
- [42] Keld Helsgaun. "An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic". 126(1):106–130, Oct. 2000. doi:[10.1016/S0377-2217\(99\)00284-2](https://doi.org/10.1016/S0377-2217(99)00284-2). URL: http://akira.ruc.dk/~keld/research/LKH/LKH-2.0/DOC/LKH_REPORT.pdf (visited on 2026-09-26) (cit. on pp. **290–295, 324**).
- [43] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: **978-1-55860-872-6**. doi:[10.1016/B978-1-55860-872-6.X5016-1](https://doi.org/10.1016/B978-1-55860-872-6.X5016-1) (cit. on pp. **324, 326**).
- [44] "Why is the Complexity of A* Exponential in Memory?" In: *Stack Overflow*. Ed. by Paul. New York, NY, USA: Stack Exchange Inc., Nov. 11, 2009–July 17, 2013. URL: <https://stackoverflow.com/questions/1715401> (visited on 2026-01-03) (cit. on pp. **194–203**).
- [45] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: **1863-7310**. ISBN: **978-3-031-35121-1**. doi:[10.1007/978-3-031-35122-8](https://doi.org/10.1007/978-3-031-35122-8) (cit. on p. **325**).
- [46] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. **319**).
- [47] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:[10.5555/800036.800944](https://doi.org/10.5555/800036.800944). See ⁴⁸ (cit. on p. **311**).
- [48] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: **1050-916X**. doi:[10.1145/1014194.800944](https://doi.org/10.1145/1014194.800944). See ⁴⁷ (cit. on pp. **311, 326**).

References VII



- [49] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: <https://www.researchgate.net/publication/6026283> (visited on 2025-08-08) (cit. on p. 326).
- [50] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on p. 329).
- [51] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (cit. on p. 329).
- [52] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (cit. on p. 326).
- [53] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (cit. on p. 329).
- [54] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (visited on 2023-12-06) (cit. on pp. 323, 329).
- [55] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (cit. on pp. 136, 160–170, 327).
- [56] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 325).

References VIII



- [57] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (cit. on pp. 160–170, 323, 325, 327).
- [58] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 323, 325).
- [59] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner, and Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (cit. on p. 325).
- [60] Shen Lin and Brian W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". *Operations Research* 21(2):498–516, Mar.–Apr. 1973. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0030-364X. doi:10.1287/opre.21.2.498. URL: <https://www.cs.princeton.edu/~bwk/btl.mirror/tsp.pdf> (visited on 2026-09-26) (cit. on p. 324).
- [61] John D. C. Little, Katta G. Murty, Dura W. Sweeney, and Caroline Karel. *An Algorithm for the Traveling Salesman Problem*. Sloan Working Papers 07-63. Massachusetts Institute of Technology (MIT): Massachusetts Institute of Technology (MIT), Sloan School of Management, Mar. 1963. URL: <http://dspace.mit.edu/bitstream/handle/1721.1/46828/algorithmfortrav001litt.pdf> (visited on 2026-01-03) (cit. on pp. 290–295, 322).
- [62] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 326).

References IX



- [63] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: **978-1-4920-5627-0** (cit. on p. **323**).
- [64] Peter Luschny. *A New Kind of Factorial Function*. Highland Park, NJ, USA: The OEIS Foundation Inc., Oct. 4, 2015. URL: <https://oeis.org/A000142/a000142.pdf> (visited on 2024-09-29) (cit. on p. **328**).
- [65] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: **978-1-0981-7130-8** (cit. on p. **325**).
- [66] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0000195000003837](https://doi.org/10.5220/0000195000003837).
- [67] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: <https://www.researchgate.net/publication/284805798> (visited on 2025-08-15) (cit. on p. **325**).
- [68] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2002.806169](https://doi.org/10.1109/TEVC.2002.806169) (cit. on p. **327**).
- [69] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: **978-1-4919-6336-4** (cit. on p. **325**).
- [70] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: **978-1-4613-7987-4**.
- [71] Amit Patel. *Amit's A* Pages*. Stanford, CA, USA: Stanford University, 1997–2025. URL: <https://theory.stanford.edu/~amitp/GameProgramming> (visited on 2026-01-02) (cit. on pp. **122–127**, **321**).
- [72] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0030-364X**. doi:[10.1287/opre.18.6.1225](https://doi.org/10.1287/opre.18.6.1225) (cit. on p. **326**).

References X



- [73] Alexander Podlich, Thomas Weise (汤卫恩), Manfred Menze, and Christian Gorltd. "Intelligente Wechselbrückensteuerung für die Logistik von Morgen". *Electronic Communications of the EASST (ECEASST)* 17(Communication in Distributed Systems), Feb. 27, 2009. The Netherlands: European Association of Software Science and Technology (EASST). ISSN: **1863-2122**. doi:[10.14279/tuj.eceasst.17.205](https://doi.org/10.14279/tuj.eceasst.17.205). In the proceedings of *Workshops der Wissenschaftlichen Konferenz Kommunikation in Verteilten Systemen 2009* (WowKiVS 2009), 3 6, 2009, Kassel, Hessen, Germany (cit. on pp. **8–13**).
- [74] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. **325**).
- [75] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/3IEC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (visited on 2024-06-29) (cit. on p. **322**).
- [76] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miltiades Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: **978-0-387-74758-3**. doi:[10.1007/978-0-387-74759-0_462](https://doi.org/10.1007/978-0-387-74759-0_462) (cit. on p. **329**).
- [77] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: **0302-9743**. ISBN: **978-3-540-65078-2**. doi:[10.1007/BFb0056920](https://doi.org/10.1007/BFb0056920) (cit. on p. **324**).
- [78] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:[10.1287/ijoc.3.4.376](https://doi.org/10.1287/ijoc.3.4.376) (cit. on p. **327**).
- [79] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: <https://github.com/shredderzwj/TSPLIB> (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite> of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on p. **327**).

References XI



- [80] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on p. 324).
- [81] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCOM.pdf> (visited on 2026-02-21) (cit. on p. 326).
- [82] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Vol. 263 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (cit. on pp. 8–13).
- [83] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: <https://arodes.hes-so.ch/record/8224/files/Preprint.pdf> (visited on 2026-02-20) (cit. on p. 323).
- [84] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 325).
- [85] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫恩). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23 (cit. on pp. 323, 325).
- [86] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on p. 327).

References XII



- [87] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: <https://thomasweise.github.io/databases> (visited on 2025-01-05) (cit. on pp. 303, 322).
- [88] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (visited on 2025-07-25) (cit. on pp. 322, 324).
- [89] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: <https://thomasweise.github.io/programmingWithC> (visited on 2026-05-10) (cit. on p. 322).
- [90] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: <https://thomasweise.github.io/programmingWithPython> (visited on 2025-01-05) (cit. on pp. 302, 325).
- [91] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew “Zbyszek” Michalewicz, and Xin Yao (姚新). “Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem”. *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101 (cit. on pp. 160–170, 322, 327).
- [92] Thomas Weise (汤卫思), Li Niu (牛力), and Ke Tang (唐珂). “AOAB – Automated Optimization Algorithm Benchmarking”. In: *12th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO’2010)*. July 7–11, 2010, Portland, OR, USA. Ed. by Martin Pelikan and Jürgen Branke. New York, NY, USA: Association for Computing Machinery (ACM), 2010, pp. 1479–1486. ISBN: 978-1-4503-0073-5. doi:10.1145/1830761.1830763 (cit. on p. 322).
- [93] Thomas Weise (汤卫思), Alexander Podlich, and Christian Gorltd. “Solving Real-World Vehicle Routing Problems with Evolutionary Algorithms”. In: *Natural Intelligence for Scheduling, Planning and Packing Problems*. Ed. by Raymond Chiong and Sandeep Dhakal. Vol. 250 of Studies in Computational Intelligence (SCI). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Sept. 2009. Chap. 2, pp. 29–53. ISSN: 1860-949X. ISBN: 978-3-642-04038-2. doi:10.1007/978-3-642-04039-9_2 (cit. on pp. 8–13).

References XIII



- [94] Thomas Weise (汤卫思), Alexander Podlich, Manfred Menze, and Christian Gorltd. "Optimierte Güterverkehrsplanung mit Evolutionären Algorithmen". *Industrie Management – Zeitschrift für industrielle Geschäftsprozesse* 10(3):37–40, June 2009. Berlin, Germany: GITO mbH Verlag (cit. on pp. 8–13).
- [95] Thomas Weise (汤卫思), Alexander Podlich, Kai Reinhard, Christian Gorltd, and Kurt Geihs. "Evolutionary Freight Transportation Planning". In: *Applications of Evolutionary Computing (EvoWorkshops 2009): Proceedings of EvoCOMNET, EvoENVIRONMENT, EvoFIN, EvoGAMES, EvoHOT, EvoIASP, EvoINTERACTION, EvoMUSART, EvoNUM, EvoSTOC, EvoTRANSLOG*. Apr. 15–17, 2009, Tübingen, Baden-Württemberg, Germany. Ed. by Mario Giacobini, Anthony Brabazon, Stefano Cagnoni, Gianni A. Caro, Anikó Ekárt, Anna Isabel Esparcia-Alcázar, Muddassar Farooq, Andreas Fink, and Penousal Machado. Vol. 5484 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Apr. 2009, pp. 768–777. ISSN: 2512-2010. doi:10.1007/978-3-642-01129-0_87 (cit. on pp. 8–13).
- [96] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885 (cit. on p. 323).
- [97] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306 (cit. on pp. 304, 322, 325).
- [98] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 321, 323–325, 327).
- [99] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698 (cit. on pp. 321, 323, 327).

References XIV



- [100] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on pp. 160–170, 327).
- [101] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: <https://www.researchgate.net/publication/2527551> (visited on 2025-08-08) (cit. on p. 324).
- [102] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on p. 325).
- [103] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system> (visited on 2025-01-11) (cit. on p. 322).
- [104] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: <https://www.ietf.org/rfc/rfc3629.txt> (visited on 2025-02-05). See Unicode and ⁴⁶ (cit. on p. 328).
- [105] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098 (cit. on pp. 321, 323–327).

References XV



- [106] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 8–13, 133–135, 325).
- [107] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 8–13, 133–135, 325).

Glossary I



- (1 + 1) EA** The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{11,26}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ evolutionary algorithm $((\mu + \lambda)$ EA) where $\mu = \lambda = 1$.
- (1 + 1) FEA** The (1 + 1) evolutionary algorithm $((1 + 1)$ EA) with Frequency Fitness Assignment (FFA) plugged in^{98,99,105}. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in RLS with FFA (FRLS) which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.
- $(\mu + \lambda)$ EA** The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- A* Algorithm** The A* Algorithm is an a greedy best-first-first search for finding the shortest path between two locations^{40,71}. The algorithm iteratively constructs the shortest path. It maintains a list of current candidate paths. For choosing the next candidate path whose end node should be expanded, it computes a value $f(p)$ of these paths p . $f(p)$ is the sum of the cost $g(p)$ so far incurred by the path as well as a heuristic $h(p)$ predicting the cost from the current end of the path to the goal. $h(p)$ must never overestimate that cost.

Glossary II



B&B *Branch and bound* is a family of optimization methods that combines systematic exploration of the search space with both and objective function f subject to minimization and a lower bound function ⁶¹. The idea is that we divide the search space into two sets. We then pick the set that seems more interesting and divide it again. We do this recursively until we have arrived at a set of size 1, which is our first solution y . We can compute its objective value $z = f(y)$. Now we continue with the sets that we have created and not yet explored. However, we can discard all sets for which the lower bound of the solution quality is $\geq z$, because they can never give us a better solution than y . If we have divided another set down to size 1 and it yields a better solution than y , of course we update these values. Eventually, we have either discarded or fully expanded all sets – and then the solution we got must be optimal.

C is a programming language, which is very successful in system programming situations ^{24,75,89}.

Concorde is an exact solver for the TSP ^{2,18,19}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.

DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases* ⁸⁷.

DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB ¹⁰³.

EA An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively) ^{4,88}.

FE *Objective function evaluations* are an implementation-independent measure of runtime for optimization algorithms ^{5,91,92,97}.
1 FE equals to one evaluated candidate solution during the optimization process.

Glossary III



FFA *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values^{96,98,99}. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the Maximum Satisfiability (MaxSAT)^{98,99}, the TSP^{57,58}, and the Quadratic Assignment Problem (QAP)^{15,85}, amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau^{98,99,105}. The most common realizations of FFA so far are the $(1+1)$ FEA and FRLS.

Java is another very successful programming language, with roots in the C family of languages^{9,63}.

JSSP The *Job Shop Scheduling Problem*^{8,16,54} is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are k machines and m jobs⁸³. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$ -complete^{14,54}.

Jump The Jump problem is a benchmark problem for bit-string based black-box search^{26,32,105}. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n . However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ²⁶. In experiments, the $(1+1)$ EA with FFA ($(1+1)$ FEA) achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{98,99,105}.

Glossary IV



- LeadingOnes** The LeadingOnes problem is a benchmark problem for bit-string based black-box search^{80,101,105}. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n \prod_{j=1}^i x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ ²⁶. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ⁹⁸.
- LK** The *Lin-Kernighan* (LK) algorithm⁶⁰ is a stochastic local search (SLS) method for the TSP. The LK algorithm performs iterative k -opt moves, i.e., in each step, it constructs the two sets X and Y of k edges such that removing the edges in X from the current tour T and then adding the edges from Y to T results in a better tour. The LK algorithm is one of the most classical approaches to the TSP.
- LKH** The *Lin-Kernighan-Helsgaun* algorithm^{19,41,42} is a SLS method for the TSP based on the Lin-Kernighan (LK) algorithm⁶⁰. LKH-1 was proposed in⁴² and LKH-2 in⁴¹. It holds the record in the *World TSP*²⁰ as of 2025.
- MaxSAT** The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0, 1\}^n \mapsto \{0, 1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)⁴³, F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x . If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$ -complete¹⁷ and exhibits low epistasis but deceptiveness⁷⁷, making it hard for local searches.
- metaheuristic** A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ ^{36,88}. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions or similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, Simulated Annealing (SA), TS, and EAs.

Glossary V



moptipy is the *Metaheuristic Optimization in Python* library⁹⁷. It has been used in several different research works, including^{15,57–59,85,102,106,107}. Algorithms implemented by moptipy include EAs, $(1 + 1)$ EA, RLS, SA, and the $(1 + 1)$ FEA, amongst others. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.

OneMax OneMax is the simplest possible benchmark problem for bit-string based black-box search^{27,67,105}. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1 + 1)$ EA on OneMax is in $\Theta(n \ln n)$ ^{25,67}. The average runtime $(1 + 1)$ FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ⁹⁸.

OR Operations Research (or Operational Research) is the application of sciences such as mathematics and computer science to the management and organization of systems, organizations, enterprises, factories, or projects. It encompasses the development and application of problem-solving methods and techniques (such as mathematical optimization, simulation, queueing theory and other stochastic models) with the goal to improve decision-making and efficiency²².

Plateau The Plateau problem is a benchmark problem for bit-string based black-box search^{1,105}. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1 + 1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus¹. The average runtime of the $(1 + 1)$ FEA on Plateaus is slower by a factor seemingly proportional in n .

PostgreSQL An open source object-relational DBMS^{30,69,74,84}. See <https://postgresql.org> for more information.

Python The Python programming language^{45,56,65,90}, i.e., what you will learn about in our book⁹⁰. Learn more at <https://python.org>.

Glossary VI



- QAP** The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{10,15,52,62}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete⁸¹.
- RLS** Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.
- FRLS** The RLS with FFA plugged in¹⁰⁵. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.
- SA** Simulated Annealing is a SLS that sometimes accepts a worse solution^{12,48,49,72}. The probability to do so decreases over time and with the difference in objective values, i.e., is the lower the worse the new solution is.
- SLS** *Stochastic local search or local search* for short, is one of the simplest metaheuristic search paradigms⁴³: A single “current” candidate solution is maintained and new solutions are sampled from its neighborhood. If these solutions are deemed interesting based on their objective value, they replace the “current”. The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

Glossary VII



Trap The Trap problem is a benchmark problem for bit-string based black-box search^{26,68,105}. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the “gradient” in the search space leads the search away from the global optimum. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 - 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²⁶. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{98,99,105}.

TS *Tabu Search* is one of the most successful SLSes^{34,35,37}. In order to avoid returning to previous solutions, it declares certain solutions or search moves as “tabu.” For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion).

TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{2,39,55,57,91,100}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{33,39}.

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)^{78,79}. Its original URL was available until 2025 and then became defunct. The repository <https://github.com/shredderzsj/TSP LIB> contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite>, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of symmetric TSPs with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric TSPs with scales $n \in 17..443$.

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search^{32,86,105}. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ ^{31,32}. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax^{98,99,105}.

Glossary VIII



URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet ¹⁰⁴. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples for URIs.

URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites ⁷. URLs are subsets of Uniform Resource Identifiers (URIs).

WWW World Wide Web ^{6,23}

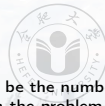
$i!$ The factorial $a!$ of a natural number $a \in \mathbb{N}_1$ is the product of all positive natural numbers less than or equal to a , i.e., $a! = 1 * 2 * 3 * 4 * \dots * (a - 1) * a$ ^{13,28,64}.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j . For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping.

Glossary IX



n We usually use n to denote the scale, i.e., the size of the problem to solve. For example, in a TSP, this may be the number of cities, in a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined, in a QAP, this may be the number of facilities and locations, and in a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a Job Shop Scheduling Problem (JSSP), we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n . And this is already the main issue: The larger n , the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms..

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$.

$\mathbb{N}_1$ the set of the natural numbers *excluding* 0, i.e., 1, 2, 3, 4, and so on. It holds that $\mathbb{N}_1 \subset \mathbb{Z}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) ³⁸.

$\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time ^{38,76}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.

$\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems ^{14,17,54} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time ³⁸.

$\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0 \forall x \geq x_0$ ^{50,51}. In other words, $\Omega(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x) \forall x \geq x_0$ ^{3,50,51,53}. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

$\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$ ^{50,51}. In other words, $\Theta(g(x))$ describes an exact order of function growth.

$\mathbb{R}$ the set of the real numbers.

Glossary X



$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated, like a Gantt chart. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

$\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y . These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.

y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.

$\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.